1(a)	Answers include: 1 They are tried and tested // Free from errors 2 They are readily available // Speed up development time 3 They will be updated automatically when improvements are made // No maintenance of routine is needed (as automatically updated) 4 The algorithm will be compliant with the international standard One mark per point **Max 3**	**3**

1(b)(i)	1 When a part of the algorithm performs a specific task 2 Part is repeated // performed in several places 3 Testing / debugging / maintenance is easier One mark for each point **Max 2**	**2**

1(b)(ii) **2**

Term	Description
`Pass2`	the name of the function
`Count`	The parameter / value passed to the function
`BOOLEAN`	The (data) **type** <u>returned</u> (by the function)

One mark per description (excluding first row)

1(c)		**3**

Expression	Evaluates to
`LENGTH(NUM_TO_STR(Multiplier))`	**3**
`MONTH(DoB) > 4`	**TRUE**
`15 + STR_TO_NUM(MID(AddressLine[1], 2, 1))`	**20**

One mark per row

Examiner report missing
9618_w25_qp_23 Question 1

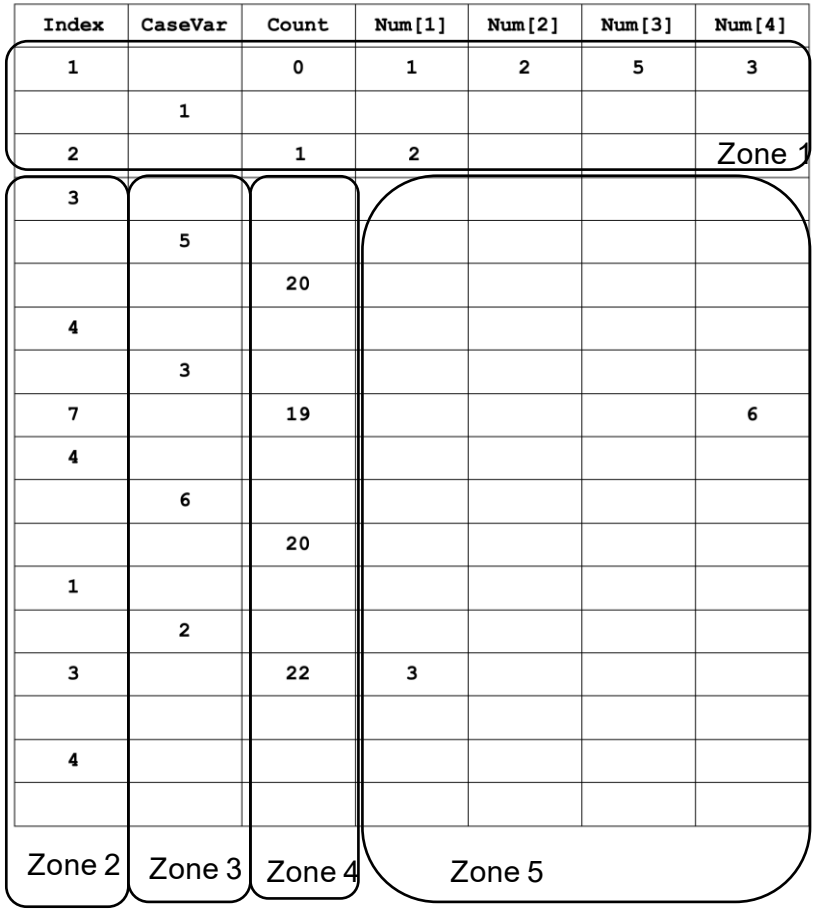
2(a)	One mark per point: 1 Open the file in append mode (and subsequently close) **before loop** 2 Initialise a variable to 1 / 0 **before loop** 3 Use the variable as an index to the array to access an element **in a loop** 4 Test if element value is blank / empty **in a loop** 5 If not blank then write the value to the file **in a loop** 6 Increment the variable **in a loop** 7 Repeat from MP3 until variable value is 66 / 65 / all elements (in the array) have been checked **Max 6**	**6**

2(b)	One mark per point: Construct: Selection Use: To test whether the (array) element is blank Alternative: Construct: Sequence Use: To specify the order in which the steps of the algorithm have to be followed so that the task is completed correctly	**2**

Examiner report missing
9618_w25_qp_23 Question 2

4	Example solution:
	
	`PROCEDURE Store()`
	`  DECLARE Index, StartPos, ThisNum : INTEGER`
	`  DECLARE ThisString : STRING`
	
	`  Index ← 1`
	`  StartPos ← 1`
	
	`  WHILE StartPos < LENGTH(NumString) AND Index < 21`
	`  ThisString ← MID(NumString, StartPos, 3)`
	`  ThisNum ← STR_TO_NUM(ThisString)`
	`  Data[Index] ← ThisNum`
	`  Index ← Index + 1`
	`  StartPos ← StartPos + 4`
	`  ENDWHILE`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	1 Declare all local variables used
	2 Conditional loop while end of string not reached
	3 ... and array not full
	4 Attempted use of `MID()` // Attempted use of `Right()` and `Left()`
	5 Correct extraction and use next three letter substring from `NumString` **in a loop**
	6 Convert to integer **and** assign value to array element **in a loop**
	7 Correct increment of array index by 1 **and** start position of substring by 4
	
	Max 6
	
	Alternative FOR loop solution:
	
	`PROCEDURE Store()`
	`  DECLARE Index, StartPos, ThisNum : INTEGER`
	`  DECLARE ThisString : STRING`
	`  StartPos ← 1`
	`  FOR Index ← 1 TO 20`
	`  ThisString ← MID(NumString, StartPos, 3)`
	`  ThisNum ← STR_TO_NUM(ThisString)`
	`  Data[Index] ← ThisNum`
	`  StartPos ← StartPos + 4`
	`  IF StartPos > LENGTH(NumString) THEN`
	`  Break`
	`  ENDIF`
	`  NEXT Index`
	`ENDPROCEDURE`
		6

Examiner report missing
9618_w25_qp_23 Question 4

5(a)	 Zones 1 to 4 : One mark for each set of values as shown Zone 5: Num[4] set to 6 when Index is 7 **and** Num[1] set to 3 when index is 3 for **second** time	**5**
5(b)(i)	1 and 2	**1**

5(b)(ii)	`1 TO 2 : Num[Index] ← Num[Index] + Index`
	`Index ← Index + CaseVar`
	`Count ← Count + CaseVar`
	
	One mark per point:
	• use of variable `CaseVar` rather than literal values
	• completely correct clause
		2

Examiner report missing
9618_w25_qp_23 Question 5

6(a)	One mark per point: 1 Create a list of numbers that should generate **all of the** possible check digit values 2 and check that each / the / a generated value is as expected	**2**

6(b)	Note 2 different solutions with different mark scheme
	
	Example solution – conversion to string and using a loop
	
	`FUNCTION CheckNumber(Number : INTEGER) RETURNS BOOLEAN`
	`  DECLARE Total, Index, CheckDigit : INTEGER`
	`  DECLARE NumString : STRING`
	
	`  Total ← 0`
	
	`  NumString ← NUM_TO_STR(Number)`
	`  CheckDigit ← STR_TO_NUM(RIGHT(NumString, 1))`
	`  // CheckDigit ← Number MOD 10`
	
	`  FOR Index ← 1 TO LENGTH(NumString) - 1`
	`  // 1 TO LENGTH(NUM_TO_STR(Number DIV 10))`
	`  Total ← Total + STR_TO_NUM(MID(NumString, Index, 1))`
	`  ENDFOR`
	
	`  IF Total MOD 10 <> CheckDigit THEN`
	`  RETURN FALSE`
	`  ENDIF`
	
	`  RETURN TRUE`
	
	`ENDFUNCTION`
	
	Mark as follows:
	1 Function Header including Identifier, parameters and return type
	2 Use of `Num_TO_STR` to convert parameter
	// Use of `Num_TO_STR` to convert parameter excluding least significant digit to a string
	3 Extract `CheckDigit` from `NumString`
	// Extract least significant digit from parameter
	4 Loop for length of `NumString - 1` following correct convesrion to string
	5 attempt to form sum of digits **in a loop**
	6 completely correct sum of digits **in a loop and** of sum initialised **before loop**
	7 Calculate check digit from sum
	8 Compare sum with final digit of parameter **and** return appropriate Boolean value
	
	Max 7
		7

Examiner report missing
9618_w25_qp_23 Question 6

8(a)	Example solution:
	
	`PROCEDURE LoanStatus(ThisStudentID, ThisBookID : STRING)`
	`  DECLARE Index : INTEGER`
	`  DECLARE Found : BOOLEAN`
	
	`  Index ← 1 / 0`
	`  Found ← FALSE`
	
	`  WHILE Index <= 8000 / 7999 AND NOT Found`
	`  IF Loan[Index].StudentID = ThisStudentID AND`
	`  Loan[Index].BookID = ThisBookID THEN`
	`  IF Loan[Index].OnLoan = FALSE THEN`
	`  OUTPUT "Loan has been returned."`
	`  ELSE`
	`  OUTPUT "Loan has not been returned."`
	`  ENDIF`
	`  Found ← TRUE`
	`  ENDIF`
	`  Index ← Index + 1`
	`  ENDWHILE`
	
	`  IF NOT Found THEN`
	`  OUTPUT "Warning - Loan not found."`
	`  ENDIF`
	`ENDPROCEDURE`
	
	Mark as follows:
	1 Procedure heading, parameters and ending
	2 Loop through **all elements** in `Loan` array
	3 ... or until record loan is found
	4 Test for correct `StudentID` **in a loop**
	5 ... and correct `BookID`
	6 ... check `OnLoan` data item
	7 ... and output <u>both</u> messages as appropriate regarding book loan status **once**
	8 If no matching loan found, Output 'not found' **after the loop**
		8

8(b)	Example solution:
	
	`FUNCTION LoansPerTutor(TutorID : STRING) RETURNS INTEGER`
	`  DECLARE Index, Count : INTEGER`
	
	`  Count ← 0`
	
	`  FOR Index ← 1 / 0 TO 8000 / 7999`
	`  IF LEFT(Loan[Index].StudentID, 3) = TutorID AND`
	`  Loan[Index].OnLoan = TRUE THEN`
	`  Count ← Count + 1`
	`  ENDIF`
	`  NEXT Index`
	
	`  RETURN Count`
	
	`ENDFUNCTION`
	
	Mark as follows:
	1 Initialise local integer for `Count`
	2 Loop through **all elements** in the `Loan` array
	3 Attempt at referencing a data item **in a loop**
	4 Extract `TutorID` from `Loan[Index].StudentID` **in a loop**
	5 Test for Matching Tutor **AND**
	test if the book has been returned using correct dot notation **in a loop**
	6 ... if true then increment `Count` **in a loop**
	7 Return count
		7

Examiner report missing
9618_w25_qp_23 Question 8

1(a)

Pseudocode example	Selection	Iteration	Subroutine

`IF Status = FALSE THEN`
`  FOR Count ← 1 TO 20`
`  CALL Reset(Count)`
`  NEXT Count`
`ENDIF`
	✓	✓	✓

`OTHERWISE : Status ← TRUE`	✓		
`WHILE AllDone() = TRUE`		✓	✓
`30 : NextChar ← 'X'`	✓		

One mark per row

4

1(b)

Variable	Example data value	Data type
`Result`	`5.42`	**REAL**
`MonthLetter`	`"JFMAMJJASOND"`	**STRING**
`Birthday`	`15/11/2009`	**DATE**

One mark per row

3

1(c)		**4**

Expression	Evaluates to
`INT(Result) + 1 > 6`	**FALSE**
`NUM_TO_STR(LENGTH(MonthLetter))`	**"12"**
`NUM_TO_STR(Result + "3.2")`	**ERROR**
`MID(MonthLetter, MONTH(Birthday) – 2, 1)`	**"S"**

One mark per row

Examiner report missing
9618_w25_qp_22 Question 1

4(a)	Example solution:
	
	`DECLARE CheckTotal : ARRAY [0:35] OF BOOLEAN`
	`DECLARE Index : INTEGER`
	
	`FOR Index ← 0 TO 35`
	`  CheckTotal[Index] ← FALSE`
	`NEXT Index`
	
	Mark as follows:
	
	MP1 Declaration of `CheckTotal` array
	MP2 Declaration of a loop counter
	MP3 Loop – with correct syntax and number of iterations
	MP4 Assignment of array element to `FALSE`
		4

4(b)	`DECLARE EasyQ, HardQ : INTEGER`
	
	`FOR EasyQ ← 0 TO 15 STEP 3`
	MP1 MP2
	
	`  FOR HardQ ← 0 TO 20 STEP 5`
	MP3 MP4
	
	`  CheckTotal[HardQ + EasyQ] ← TRUE`
	MP5
	
	`  NEXT HardQ`
	
	`NEXT EasyQ`
	
	Mark as follows:
	One mark per gap (boldened)
		5

4(c)	**MP1** **MP2** **MP3** **MP4**	Check that the parameter value is in the range 0 to 35 ... and if not, return `FALSE` Use the **parameter** value as **the index to array** `CheckTotal` Return the value from given index position	**4**

Examiner report missing
9618_w25_qp_22 Question 4

6	Example solution:
	
	`FUNCTION Compare(String1, String2 : STRING, __`
	`  CaseMatters : BOOLEAN, Position : CHAR) RETURNS BOOLEAN`
	`  DECLARE S1Length : INTEGER`
	
	`  S1Length ← LENGTH(String1)`
	`  IF LENGTH(String2) < S1Length THEN`
	`  RETURN FALSE // String2 is shorter than String1`
	`  ENDIF`
	
	`  IF CaseMatters = FALSE THEN`
	`  String1 ← TO_UPPER(String1)`
	`  String2 ← TO_UPPER(String2)`
	`  ENDIF`
	
	`  CASE Position`
	`  'e' : String2 ← RIGHT(String2, S1Length)`
	`  's' : String2 ← LEFT(String2, S1Length)`
	`  ENDCASE`
	
	`  IF String1 = String2 THEN`
	`  RETURN TRUE`
	`  ELSE`
	`  RETURN FALSE`
	`  ENDIF`
	
	`ENDFUNCTION`
	
	Mark as follows:
	
	MP1 Function heading **and** parameters **and** ending **and** return type
	MP2 Calculate length of `String1`
	MP3 **if** `String2` is shorter than `String1`, return `FALSE`
	MP4 Compare (modified) strings
	MP5 Test `CaseMatters` - if `FALSE` **and** convert two string(s) to same case
	MP6 Test for the value of ‘s’/’e’ **and** form modified string(s) **in one** or both cases
	MP7 Return appropriate `BOOLEAN` value in all cases
		7

Examiner report missing
9618_w25_qp_22 Question 6

7(a)	Example explanation: The arrow means that `Convert` **repeatedly** calls `Analyse` then `Update` and then `Sort` (in that order)10 **MP1** Reference to 'repetition'…' **MP2** Naming **all <u>four</u> modules** correctly **and** the sequence is stated or implied	**2**

7(b)	**MP1** `PROCEDURE Analyse (Count : INTEGER)` `FUNCTION Update (Key : STRING) RETURNS INTEGER` **MP2** **MP3** **MP4** `PROCEDURE Sort (BYREF P2 : BOOLEAN)` Mark as follows: One mark per underlined part	**4**

Examiner report missing
9618_w25_qp_22 Question 7

8(a)	Example solution:
	
	`PROCEDURE CountLoans(ThisStudentID : STRING)`
	`  DECLARE Index, LCount, RCount : INTEGER`
	
	`  LCount ← 0`
	`  RCount ← 0`
	
	`  FOR Index ← 1 TO 7000`
	`  IF Loan[Index].StudentID = ThisStudentID THEN`
	`  IF Loan[Index].OnLoan = FALSE THEN`
	`  RCount ← RCount + 1`
	`  ELSE`
	`  LCount ← LCount + 1`
	`  ENDIF`
	`  ENDIF`
	`  NEXT Index`
	
	`  OUTPUT "Student has ", LCount, " books on loan"`
	`  OUTPUT "and has returned ", RCount, " books"`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 Declaration of Index **and** two count variables
	MP2 Loop for 7000 iterations
	MP3 `Index` references an **indexed data** item with correct **.dot notation**
	MP4 Attempt to compare the `StudentID` field = **parameter in a loop**
	MP5 Test of the `OnLoan` field **and** correct logic for both outcomes
	MP6 Both count variables initialised **and** increment appropriate count **in a loop**
	MP7 Output includes both counts with a <u>suitable</u> message
		7

8(b)	Example solution:
	
	`FUNCTION NewLoan(ThisStudentID, ThisBookID : STRING) __ RETURNS BOOLEAN`
	`  DECLARE Index : INTEGER`
	`  DECLARE IsStored : BOOLEAN`
	`  CONSTANT Blank = ""`
	
	`  Index ← 1`
	`  IsStored ← FALSE`
	
	`  WHILE Index <= 7000 AND NOT IsStored`
	`  IF Loan[Index].StudentID = Blank THEN`
	`  Loan[Index].StudentID ← ThisStudentID`
	`  Loan[Index].BookID ← ThisBookID`
	`  Loan[Index].OnLoan ← TRUE`
	`  IsStored ← TRUE`
	`  ENDIF`
	`  Index ← Index + 1`
	`  ENDWHILE`
	
	`  RETURN IsStored`
	`ENDFUNCTION`
	
	Mark as follows:
	
	MP1 Loop through 7000 elements in `Loan` array – correct loop structure
	MP2 or until the first unused element is found // Correct `RETURN` inside a `FOR` loop
	MP3 Correct use of the indexed dot notation
	MP4 Test if record is unused (`.StudentID = ""`) **in a loop**
	MP5 Two item assignment statements correct
	MP6 `Loan[Index].OnLoan` assigned TRUE
	MP7 Correct logic to return `BOOLEAN` in both cases (position found / no position available)
	
	Alternative solution: loop only (no 'found' mechanism)
	
	`  FOR Index ← 1 TO 7000`
	`  IF Loan[Index].StudentID = Blank THEN`
	`  Loan[Index].StudentID ← ThisStudentID`
	`  Loan[Index].BookID ← ThisBookID`
	`  Loan[Index].OnLoan ← TRUE`
	`  RETURN TRUE`
	`  ENDIF`
	`  NEXT`
	`  RETURN FALSE`
		7

Examiner report missing
9618_w25_qp_22 Question 8

1(a)	**Expression**
	`MyString ← 'X' & MyString`
	`MyChar ← MID ("ABCD", 1 / 2 / 3 / 4 , 1)`
	`MyString ← NUM_TO_STR (DAY / MONTH / YEAR / DAYINDEX (MyDOB))`
	`MyInt ← INT (LENGTH (MyString) / 2)`
	One mark per row
		4

Examiner report missing
9618_w25_qp_21 Question 1

4(a)	Example solution - conditional:
	
	`PROCEDURE Store()`
	`  DECLARE ThisNum, LastNum : REAL`
	`  DECLARE Index : INTEGER // index to global array`
	
	`  INPUT ThisNum`
	`  LastNum ← ThisNum`
	`  Number[1] ← ThisNum`
	
	`  INPUT ThisNum`
	`  Index ← 2`
	
	`  WHILE Index < 21 AND ThisNum > LastNum`
	`  Number[Index] ← ThisNum`
	`  LastNum ← ThisNum`
	`  Index ← Index + 1`
	`  IF Index < 21 THEN`
	`  INPUT ThisNum`
	`  ENDIF`
	`  ENDWHILE`
	
	`  OUTPUT Index - 1, " values were stored in the array"`
	`ENDPROCEDURE`
	
	Alternative loop:
	`  // Loop without using LastNum`
	`  WHILE Index < 21 AND ThisNumNum > Number[Index - 1]`
	`  Number[Index] ← ThisNum`
	`  Index ← Index + 1`
	`  IF Index < 21 THEN  //skip input if array full`
	`  INPUT ThisNum`
	`  ENDIF`
	`  ENDWHILE`
	
	Mark as follows:
	1 Declare local variables `Index` as integer, `ThisNum` and `LastNum` as real
	2 Input value and assign to first element
	3 **Attempt** at conditional loop including **both** tests
	4 **Completely correct** conditional loop including **both** tests
	5 Assign input value to current element if greater than previous element **and** input next value **in a loop**
	6 Final output giving attempted count of elements stored plus message **after a loop**
		6

4(b)(i)	One mark per point: 1 The amount of values **stored** (in a text file) is **not fixed** / **not limited** (other than by secondary storage available) // The amount values **stored** is **not limited** to the size of the array 2 The data is stored **permanently** / **non-volatile** // Data can be restored / reused without **re-entering** values	**2**

4(b)(ii)	Each (real) value would have to be **converted** to a string	**1**

Examiner report missing
9618_w25_qp_21 Question 4

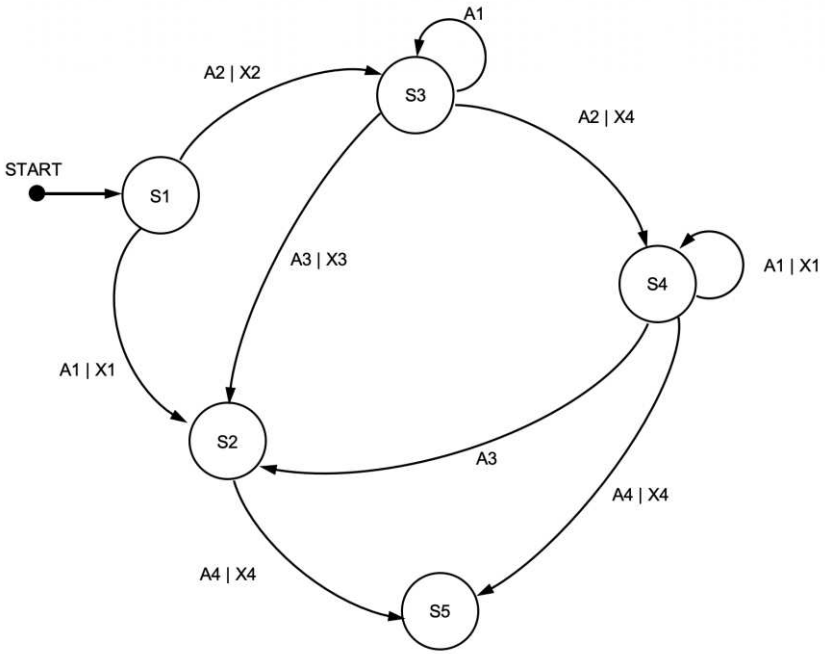
5(a)(i)	It is a value that is **invalid as an** (array) **index** // not in the **range** of valid (index) values	**1**
5(a)(ii)	To test whether the **current element / value at index** matches the **given / search** string / data / **parameter**	**1**
5(a)(iii)	The value of `FoundAt` is only updated to the (current) index if it currently stores the initial value / -1 // `FoundAt` is not updated when it currently stores any other value than -1	**1**
5(b)(i)	The loop continues after the (first instance) of the **search** value / string / **matching element** has been found Comparisons/tests continue to be made even if the **search** value **/ string / matching element** has been found	**1**
5(b)(ii)	One mark per point Construct: A conditional loop Justification: The loop / search can be terminated as soon as the (first occurrence of) **`SearchString`** / **required** value / **required** string / **matching** element is found	**2**

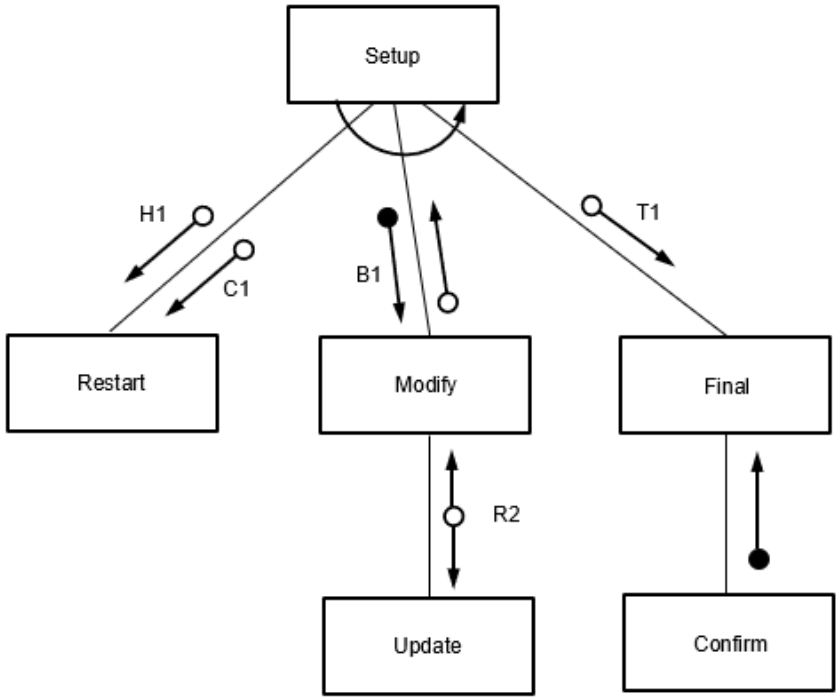
Examiner report missing
9618_w25_qp_21 Question 5

6(a)	Number:	Any number where the first two digits sum to 5 / 15 **and** the last three digits are 623	**2**
	Explanation:	The remainder is the same / 3 // The sum of the (first four) digits is the same / 13	

6(b)	Example solution – conversion of `Number` to a string
	
	`FUNCTION Generate(Number : INTEGER) RETURNS INTEGER`
	`  DECLARE Total, Index, CheckDigit : INTEGER`
	`  DECLARE NumString : STRING`
	
	`  Total ← 0`
	`  NumString ← NUM_TO_STR(Number)`
	
	`  FOR Index ← 1 TO LENGTH(NumString)`
	`  Total ← Total + STR_TO_NUM(MID(NumString, Index, 1))`
	`  NEXT Index`
	
	`  CheckDigit ← Total MOD 10`
	
	`  Number ← Number * 10 + CheckDigit`
	`  // NumString ← Numstring & NUM_TO_STR(CheckDigit)`
	`  RETURN Number // STR_TO_NUM(NumString)`
	
	`ENDFUNCTION`
	
	Mark as follows:
	
	1 Function heading, parameters return type and ending
	2 Initialise `Total`
	3 Convert `Number` to a string **and** loop for length of converted `Number`
	4 attempt to form sum of digits **in a loop**
	5 completely correct sum of digits **in a loop**
	6 Calculate `CheckDigit`
	7 Add `CheckDigit` to original number multiplied by 10 **and** return number
	//
	Convert `CheckDigit` to a character and appended to `NumString` **and** then convert to a number and return
		7

Examiner report missing
9618_w25_qp_21 Question 6

7(a)	 One mark for each: 1 Label A1 | X1 added on line from S1 to state labelled S2 2 States S3, S4 and S5 labelled 3 Line from S2 to S5 with event label A4|X4 4 Lines from S3 with correct event labels including 'loop' label 5 Lines from S4 with correct event labels including 'loop'	**5**

7(b)	 One mark per point: 1 All boxes correctly labelled 2 Iteration arrow 3 Parameter to Restart 4 Return value from Modify **and** parameter to Final 5 BYREF parameter to Update	**5**

Examiner report missing
9618_w25_qp_21 Question 7

8(a)	Example solution:
	
	`PROCEDURE OKToBorrow(ThisStudentID : STRING)`
	`  DECLARE Index, Count : INTEGER`
	
	`  CONSTANT Max = 5`
	`  Count ← 0`
	`  Index ← 1 / 0`
	
	`  WHILE Index <= 5000 / Index <= 4999 AND Count < Max`
	`  IF Loan[Index].StudentID = ThisStudentID AND __`
	`  Loan[Index].OnLoan =`
	`TRUE THEN`
	`  Count ← Count + 1`
	`  ENDIF`
	`  Index ← Index + 1`
	`  ENDWHILE`
	
	`  IF Count < Max THEN`
	`  OUTPUT "Student may borrow another book"`
	`  ELSE`
	`  OUTPUT "Student may not borrow another book"`
	`  ENDIF`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	1 Procedure heading, parameter and ending
	2 Loop through **all elements** in `Loan` array
	3 ... terminating if `Max` reached
	
	4 Test for correct `StudentID` field **in a loop**
	5 ... and that `OnLoan = TRUE` **in a loop**
	... if so, then increment `Count` **in a loop**
	6 Output an appropriate **message** in both cases **after the loop**
		7

8(b)	Example solution:
	
	`FUNCTION ReturnBook(ThisStudentID, ThisBookID : STRING)`
	`  RETURNS BOOLEAN`
	`  DECLARE Index : INTEGER`
	
	`  Index ← 1`
	
	`  WHILE Index < 5001`
	`  IF Loan[Index].StudentID = ThisStudentID  AND __`
	`  Loan[Index].BookID = ThisBookID THEN`
	`  IF Loan[Index].OnLoan = TRUE THEN // Not returned`
	`  Loan[Index].OnLoan ← FALSE // Mark as returned now`
	`  RETURN TRUE  //Found and not already returned`
	`  ELSE`
	`  RETURN FALSE  // Found and already returned`
	`  ENDIF`
	`  ENDIF`
	`  Index ← Index + 1`
	`  ENDWHILE`
	
	`  RETURN False // loan not found`
	`ENDFUNCTION`
	
	Mark as follows:
	1 Loop through **all elements** in `Loan` array
	2 ...and book loan not found
	3 Attempt to reference an individual data item **in a loop**
	4 Correct test for `StudentID` **and** correct `BookID` **in a loop**
	5 **...** If book not returned yet, then set `OnLoan` field to `FALSE`
	6 RETURN TRUE if book loan Found and not already returned
	7 RETURN FALSE if book loan Found and loan already returned
	8 RETURN FALSE if book loan not found
	
	Alterantive example solution – not immediate return
	
	`FUNCTION ReturnBook(ThisStudentID, ThisBookID : STRING)`
	`  RETURNS BOOLEAN`
	`  DECLARE Index : INTEGER`
	`  DECLARE Found, OK : BOOLEAN`
	
	`  Index ← 1`
	`  Found ← FALSE`
	`  OK ← TRUE`
		8

Examiner report missing
9618_w25_qp_21 Question 8

1(a)(i)	**MP1** **MP2** **MP3** **MP4** **MP5** **Max 2**	When a **task/module is repeated** / reused / called // performed in several places A **specific task** can be identified /coded as a module / subroutine / procedure / function. Reduces complexity of **program / code** // **Program / code** is simplified Module / subroutine / procedure / function **already available**. Testing / debugging / maintenance is easier	**2**
1(a)(ii)	**MP1** **MP2** **MP3** **Max 1**	Local variables are used within a module // are accessible only within the module (in which they are declared) Global variables are accessible to all parts of the program / Local variables use memory only while the module in is executing // Global variables use memory for the whole time the program is running	**1**
1(a)(iii)	**MP1** **MP2** **MP3** **MP4** **Max 2**	The **same variable name** can be reused in other parts of the program Using local variables makes modules **self-contained** // cannot accidentally change the same identifier outside of the module Using local variables aids **modularisation**. Memory allocated to local variables can be reused when module not in use	**2**

1(b)	Mark as follows: **Expression** / **Evaluates to** / **Mark** **CHR**(68) / 'D' / 1 **MONTH**(04/02/2025) // **-2 + DAY**(04/02/2025) // **-2023 + YEAR**(04/02/2025) / 2 / 1 **NOT** TRUE // **FALSE =** TRUE / FALSE / 1 **STR_TO_NUM(MID**("Court1.4Upper",**6,3**)) / 1.4 / 2 Mark Row 4 as follows: **STR_TO_NUM** 1 mark **MID**("Court1.4Upper",**6,3**) 1 mark	**5**

Examiner report missing
9618_s25_qp_23 Question 1

3	Example solution:
	
	`FUNCTION RollDice(NoOfTimes : INTEGER) RETURNS REAL`
	`  DECLARE RollValue : INTEGER`
	`  DECLARE Average : REAL`
	`  DECLARE Count : INTEGER`
	`  DECLARE Sum : INTEGER`
	
	`  Sum ← 0`
	
	`  FOR Count ← 1 TO NoOfTimes`
	`  RollValue ← INT(RAND(6)) + 1`
	`  OUTPUT RollValue`
	`  Sum ← Sum + RollValue`
	`  NEXT Count`
	
	`  Average ← Sum / NoOfTimes`
	`  RETURN Average`
	`ENDFUNCTION`
	
	MP1 Declare all variables used **and** initialise `Sum` to `0`
	MP2 Loop for 'number of times' parameter
	
	MP3 Use `RAND`() function with Integer parameter **in a loop**
	MP4 Use `INT`() function **in a loop**
	MP5 Generate a random integer between 1 and 6 **in a loop**
	
	MP6 Output each value generated i**n a loop**
	MP7 Calculate `Sum` **and** if `Average` used check declared as `REAL` **and** return `Average` **and** check function header returns `REAL`
		7

Examiner report missing
9618_s25_qp_23 Question 3

5	Example solution:
	
	`FUNCTION Parity(BitString : STRING) RETURNS STRING`
	`  DECLARE Index, Count: INTEGER`
	`  Count ← 0`
	
	`  FOR Index ← 1 TO LENGTH(BitString)`
	`  IF MID(BitString, Index, 1) = '1' THEN`
	`  Count ← Count + 1`
	`  ENDIF`
	`  NEXT Index`
	
	`  IF Count MOD 2 = 1 THEN`
	`  BitString ← BitString & '1'`
	`  ELSE`
	`  BitString ← BitString & '0'`
	`  ENDIF`
	
	`  RETURN BitString`
	`ENDFUNCTION`
	
	MP1 Function heading **and** ending **and** parameter (`STRING`) **and** return type `STRING`
	MP2 Loop for length of `BitString`
	MP3 Extract current character **in a loop**
	MP4 Compare current character to `'1'` or `'0'` **in a loop**
	MP5 Increment count **and** was Initialised earlier **in a loop**
	
	MP6 Check if even/odd number of 1s
	MP7 Append `'1'` or `'0'` as appropriate
	MP8 Return amended `BitString`
		8

Examiner report missing
9618_s25_qp_23 Question 5

6(a)	**MP1** A function returns a (single) **value and** a procedure does not **MP2** A procedure can pass parameters by ByRef **Max 1**	**1**

6(b)	Sub_Part1A:
	
	`PROCEDURE Sub_Part1A(R: INTEGER)` 1 Mark
	
	Sub_Part2A:
	
	`FUNCTION Sub_Part2A(T : REAL) RETURNS BOOLEAN`
	
	2 Marks
	
	Sub_Part3A:
	
	`PROCEDURE Sub_Part3A(V: INTEGER, W: BOOLEAN, BYREF U : STRING)`
	
	2 Marks
		5

Examiner report missing
9618_s25_qp_23 Question 6

7(a)(i)	Example solution:
	
	`PROCEDURE UpdateVisit(BYREF LastVisitDate : DATE,__`
	`BYREF LoyaltyPoints : INTEGER)`
	
	`IF MONTH(TODAY()) = MONTH(LastVisitDate) THEN`
	`LoyaltyPoints ← LoyaltyPoints + 4`
	`ELSE`
	`LoyaltyPoints ← LoyaltyPoints + 1`
	`ENDIF`
	
	`LastVisitDate ← TODAY()`
	
	`ENDPROCEDURE`
	
	MP1 Procedure heading **and** ending **and** two parameters of correct type
	MP2 … both passed `BYREF`
	
	MP3 Use of `TODAY()` function to obtain current date
	MP4 Use `MONTH` function (×2) **and** one parameter is the header parameter **and** 2nd parameter is `TODAY()` / assigned variable
	
	MP5 If visit is same month, increase `LoyaltyPoints` by 4 otherwise increase by 1
	MP6 `LastVisitDate` set to `TODAY()` / 'current date' variable (assigned or unassigned)
	Max 5
		5

7(a)(ii)	`DAYINDEX(LastVisitDate) = DAYINDEX(TODAY())`	**1**

7(b)(i)	Example solution:
	
	`FUNCTION (MondayCheck(CustomerID : STRING) RETURNS INTEGER`
	`  DECLARE LoyaltyPoints : INTEGER`
	`  DECLARE LoyaltyPointsString : STRING`
	`  DECLARE Line : STRING`
	
	`  Line ← FindCustomer(CustomerID)`
	`  LoyaltyPointsString ← MID(Line, 8, LENGTH(Line) - 16))`
	`  LoyaltyPoints ← STR_TO_NUM(LoyaltyPointsString)`
	
	`IF DAYINDEX(TODAY()) = 2 THEN`
	`  LoyaltyPoints ← LoyaltyPoints + 10`
	`  ENDIF`
	
	`  RETURN LoyaltyPoints`
	`ENDFUNCTION`
	
	MP1 Value is returned from `FindCustomer(CustomerID)` and stored
	
	MP2 Function `MID` used
	MP3 Attempt to extract `LoyaltyPoints` from the string returned by `FindCustomer()`
	MP4 Correct extraction of `LoyaltyPoints` from the string returned by `FindCustomer()`
	
	MP5 Conversion of `LoyaltyPoints` extracted to an integer
	
	MP6 Check - is today's date a Monday using **`TODAY()`**
	MP7 … increase `LoyaltyPoints` by 10
	MP8 Return `LoyaltyPoints`
		8

7(b)(ii)	Example solution:
	
	`DayInt ← STR_TO_NUM(LEFT(LastVisitDateString, 2))`
	`MonthInt ← STR_TO_NUM(MID(LastVisitDateString, 3, 2))`
	`YearInt ← STR_TO_NUM(RIGHT(LastVisitDateString, 4))`
	
	`LastVisitDate ← SETDATE(DayInt, MonthInt, YearInt)`
	
	Mark as follows:
	
	MP1 Use of one substring function
	MP2 `DayInt`, `MonthInt` and `YearInt` all correctly extracted
	
	MP3 Use of `STR_TO_NUM` x 3
	
	MP4 `SETDATE()` function used to convert to a date type **and** assigned to `LastVisitDate`
		4

Examiner report missing
9618_s25_qp_23 Question 7

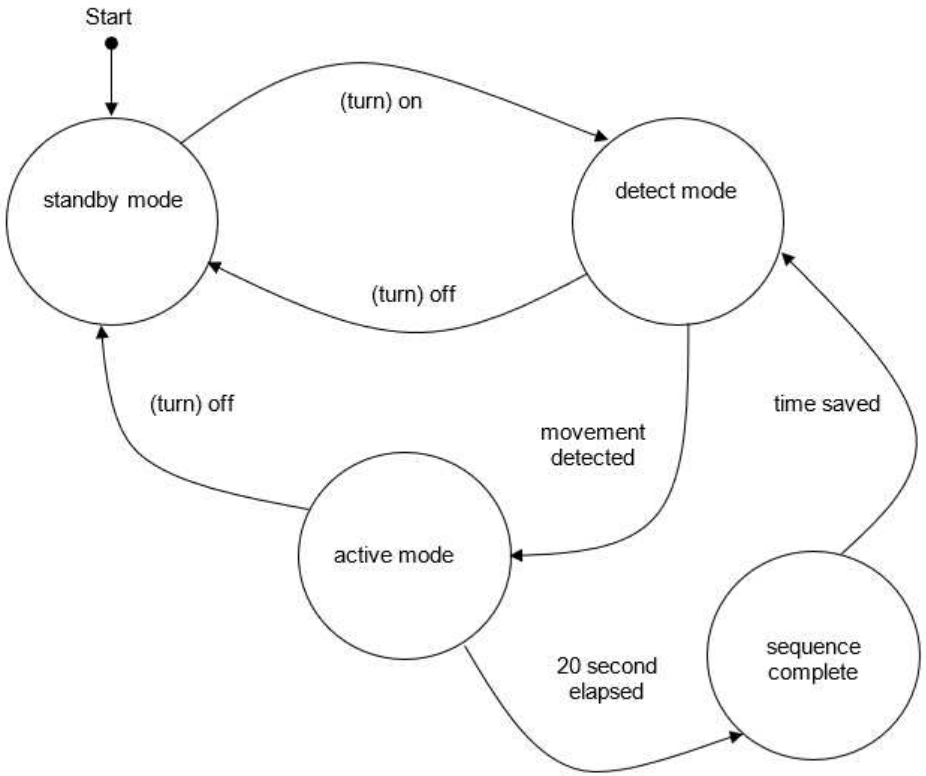
1(a)(i)	One mark for Choice of program development cycle depends on the **approach/method** required to produce the program **Or** One mark for a factor that would influence the choice of program development cycle, e.g. • Rapid development of program/prototypes required • Need for prototypes (at an early stage) • Complexity of problem • Skills / experience of development team / programmer(s) • Budget / Time / Resources available • Size of development team • Developer / programmer can return to earlier **stages** / any stage • Avoidance of repeating previous stages in development of program • How much involvement clients will have • Allows the (program) requirements to be changed during program development • (Program) requirements agreed at start of development of program **Max 1**	**1**

1(c)	1 mark for each correct row | Expression	Data type
---	---
`RIGHT(MachineCode, 4)`	**STRING**
`Speed * 2.5`	**REAL**
`NOT Status`	**BOOLEAN**
`IS_NUM(Check)`	**BOOLEAN**		**4**

Examiner report missing
9618_s25_qp_22 Question 1

3	Example solution
	
	`DECLARE FirstNumber : INTEGER`
	`DECLARE SecondNumber : INTEGER`
	`DECLARE ThirdNumber : INTEGER`
	
	`FirstNumber ← INT(RAND(21)) - 10`
	
	`OUTPUT FirstNumber`
	
	`REPEAT`
	`  SecondNumber ← INT(RAND(21)) - 10`
	`UNTIL FirstNumber <> SecondNumber`
	
	`OUTPUT SecondNumber`
	
	`IF FirstNumber < 0 AND SecondNumber < 0 THEN`
	`  ThirdNumber ← INT(RAND(6)) + 30`
	`  OUTPUT ThirdNumber`
	`ENDIF`
	
	Mark points:
	
	1. Declare all variables used
	2. Use `RAND()` function with any **integer** parameter
	3. Use `INT()` function with any **numeric** parameter
	4. Conditional loop until **two different** random numbers are generated
	5. Use `INT()` function using random number generated between –10 and 10 inclusive **in a loop**
	6. Output **two different** random integers / numbers
	7. Check if both random integers / numbers are negative
	8. then output a (third) random integer / number between 30 and 35 inclusive
		8

Examiner report missing
9618_s25_qp_22 Question 3

5(a)	 Mark as follows: 1 Mark: The three new states drawn and labelled. 1 Mark: Two of the events drawn with labels connecting correct states and correct line direction 1 Mark: Four of the events drawn with labels connecting correct states and correct line direction. 1 Mark: All and only six events drawn with labels connecting correct states and correct line direction.	**4**

5(b)	Conditional Solution
	
	Example Solution
	
	`DECLARE Count : INTEGER`
	`DECLARE Line : STRING`
	`DECLARE NextHour, Hour : STRING`
	
	`OPENFILE "TimeTaken.txt" FOR READ`
	`Count ← 1`
	`READFILE "TimeTaken.txt", Line`
	`Hour ← LEFT(Line, 2)`
	
	`WHILE NOT EOF("TimeTaken.txt")`
	`  READFILE "TimeTaken.txt", Line`
	`  NextHour ← LEFT(Line, 2)`
	`  IF NextHour = Hour THEN`
	`  Count ← Count + 1`
	`  ELSE`
	`  OUTPUT "Hour : ", Hour, " Total : ", Count`
	`  Hour ← NextHour`
	`  Count ← 1`
	`  ENDIF`
	`ENDWHILE`
	
	`OUTPUT "Hour : ", Hour, " Total : ", Count`
	
	`CLOSEFILE "TimeTaken.txt"`
	
	Mark as follows:
	
	1. Any Initialisation of count for number of pictures taken each hour
	2. Open `"TimeTaken.txt"` for read and subsequently close
	3. Conditional loop until EOF
	4. Read a line from `"TimeTaken.txt"` **in a loop**
	5. Extract Hour from line read **in a loop**
	6. A mechanism to compare current hour extracted from file with last one read from file **in a loop**
	7. If hours same increment count **in a loop**
	8. If **not** same output count (with a suitable message) **and** update `Hour ← NextHour` **and** set Count to 1 **in a loop**
	9. Output final `Count` **and** `Hour` (with a suitable message) **once only**
	
	Note: max 8
		8

Examiner report missing
9618_s25_qp_22 Question 5

6(a)(i)		**4**

Value	Start	Unused	New	Last	Current	
1043	2	8	8	−1	2	Region 1
				2	3	Region 2
				3	1	Region 3
				1	7	
				7	4	Region 4

Award 1 mark per region

Examiner report missing
9618_s25_qp_22 Question 6

7(a)	Example solution Conditional Loop
	
	`FUNCTION FindCustomer(CustomerID : INTEGER) RETURNS INTEGER`
	
	`DECLARE Index : INTEGER`
	`DECLARE Found : BOOLEAN`
	`CONSTANT Unused = 99999`
	`CONSTANT Upper = 1000`
	`Found ← FALSE`
	
	`Index ← 1`
	
	`IF CustomerID < 10001 OR CustomerID > 11000 THEN`
	`RETURN -1 // Out of range value for customer ID`
	`ENDIF`
	
	`WHILE Found = FALSE AND Loyalty[Index, 1] <> 99999`
	`IF Loyalty[Index,1] = CustomerID THEN`
	`Found ← TRUE`
	`ELSE`
	`Index ← Index + 1`
	`ENDIF`
	`IF Index > Upper THEN`
	`RETURN -1`
	`ENDIF`
	`ENDWHILE`
	
	`IF Found THEN`
	`RETURN Loyalty[Index, 2]`
	`ELSE`
	`RETURN -1`
	`ENDIF`
	
	`ENDFUNCTION`
	
	Mark as follows:
	
	1. Create function header and ending with correct parameter and return type
	2. Check `CustomerID` is in range and if not return `-1`
	3. (Conditional) loop iterating through each element in array
	4. Check for current element in array contains required customer ID **in a loop**
	5. … If found set value to terminate loop
	6. Terminating loop when customer ID found
	7. Terminating loop when current customer ID is 99999
	8. Return either loyalty points for the customer found or `-1` if not found
		8

7(b)	Example solution
	
	`PROCEDURE PointsReport()`
	
	`DECLARE Count : INTEGER`
	`DECLARE Index : INTEGER`
	`DECLARE Sum : INTEGER`
	`DECLARE Average : REAL`
	
	`Index ← 1`
	`Count ← 0`
	`Sum ← 0`
	
	`WHILE Loyalty[Index, 1] <> 99999 AND Index <= 1000`
	`IF Loyalty[Index, 2] >= 11 THEN`
	`OUTPUT Loyalty[Index, 1]`
	`ENDIF`
	`Count ← Count + 1`
	`Sum ← Sum + Loyalty[Index, 2]`
	`Index ← Index + 1`
	`ENDWHILE`
	
	`Average ← Sum / Count`
	`OUTPUT "The average points of all the customers is ", Average`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	
	1. Create procedure header and ending
	2. Declare and initialise `Index`, `Sum` and `Count`
	3. Loop through all elements in `Loyalty` array
	4. … or terminates when column1 of `Loyalty` array equals `99999` **in a loop**
	5. Check if loyalty points greater than or equal to `11` **in a loop**
	6. If true output customer ID
	7 Sum points **and** Increment `Count` **in a loop**
	8 Calculate **and** output average with an appropriate message
	
	Max 7
		7

Examiner report missing
9618_s25_qp_22 Question 7

1(a)

Example value	Explanation	Variable name	Data type
"Fruit"	a category of stock that is sold in the shop	**Category**	**STRING**
20/02/2025	when an item was sold	**DateSold**	**DATE**
12.67	the cost of an item	**(Item)Cost/Price**	**REAL**
TRUE	to indicate if an item is in stock	**(Item)InStock**	**BOOLEAN**

Mark as follows:
1 mark per row for each variable name **and** data type

4

1(b)

Pseudocode extract	Assignment	Selection	Iteration
`Result ← CalculateTotal()`	✓		
`WHILE IsClosed`			✓

`REPEAT`
`  INPUT Value`
`UNTIL Sales[4] > Value`
	✓		✓

`IF Sales[Current] <= 150 THEN`
`  Discount ← TRUE`
`ENDIF`
	✓	✓	

`CASE OF Option`		✓	

Mark as follows:
One mark for each correct row

5

Examiner report missing
9618_s25_qp_21 Question 1

2(a)(i)	**MP1** The two decision boxes are in the wrong order // The **first decision** symbol is wrong **MP2** The first decision should check for values greater than or equal to 2000 **Max 1**	**1**

2(b)(i)	2000 / 4000 / 10 / 100	**1**

2(b)(ii)	**MP1** The value **cannot be changed** // avoids being different in two places **MP2** Makes the program easier to understand **MP3** Avoids repeatedly writing the same value throughout the program **MP4** A change to the value requires a change in only one statement **Max 2**	**2**

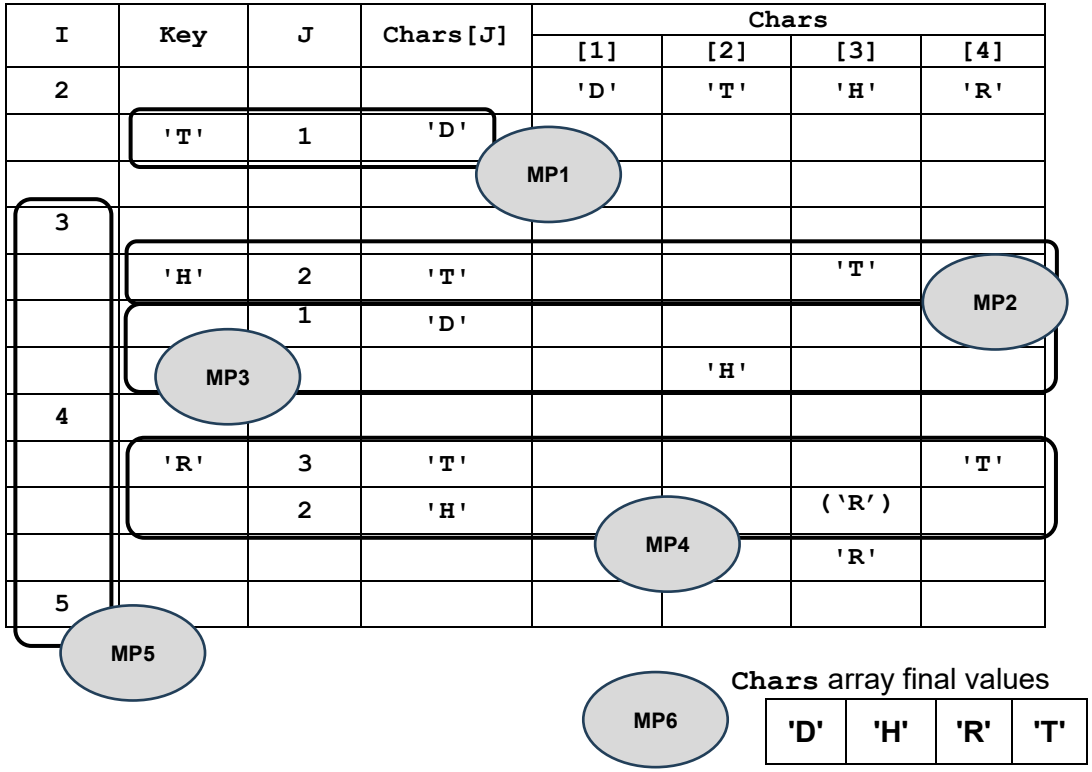
2(b)(iii)	**MP1** (The code is) tried and tested so error free **MP2** Provides functionality / code that the programmer may find difficult to code themselves **Max 1**	**1**

Examiner report missing
9618_s25_qp_21 Question 2

3	Example solution:
	
	`DECLARE GeneratedValue : INTEGER`
	`DECLARE Guess : INTEGER`
	
	`GeneratedValue ← INT(RAND(100)) + 1`
	
	`REPEAT`
	`  OUTPUT "Enter a guess between 1 and 100: "`
	`  INPUT Guess`
	
	`  IF Guess > GeneratedValue THEN`
	`  OUTPUT "Guess was too high"`
	`  ENDIF`
	`  IF Guess < GeneratedValue THEN`
	`  OUTPUT "Guess was too low"`
	`  ENDIF`
	`UNTIL Guess = GeneratedValue`
	
	`OUTPUT "Number guessed correctly"`
	
	MP1 Declare all variables used
	
	MP2 Uses `RAND()` function
	MP3 Uses `INT()` function
	MP4 Correct syntax for random number between 1 and 100
	
	MP5 Conditional loop until correct number guessed
	MP6 Prompt and input a guess **in a loop**
	
	MP7 Check if guess is too high **in a loop**
	MP8 Check if guess is too low **in a loop**
	
	MP9 Output appropriate message (x2) when the number is **not** guessed correctly (**in** the loop) **and** output correct guess message
	
	Max 8
		8

Examiner report missing
9618_s25_qp_21 Question 3

4(a)	**MP1** Count-controlled **MP2** The number of iterations required is known	**2**

4(b)	 Mark as follows: **MP1, MP2, MP3, MP4, MP5** 1 mark per enclosure	**6**

Examiner report missing
9618_s25_qp_21 Question 4

6(a)		**4**

Type of test data	Test data value	Expected outcome
normal	36	data item is accepted
boundary/ extreme/ normal	0 / 1	data item is accepted
boundary/ extreme/ normal	59 / 60	data item is accepted
abnormal	>= 61	data item is rejected
normal	15	data item is accepted

Mark as follows:
1 mark for each row with (Type **and** Value **and** Outcome)

6(b)	Example solution:
	
	`PROCEDURE Sort()`
	`  DECLARE Temp, J, Boundary : INTEGER`
	`  DECLARE NoSwaps : BOOLEAN`
	
	`  Boundary ← 1999`
	
	`  REPEAT`
	`  NoSwaps ← TRUE`
	`  FOR J ← 1 TO Boundary`
	`  IF Reading[J, 1] > Reading[J+1, 1] THEN`
	`  //first swap sensor value`
	`  Temp ← Reading[J, 1]`
	`  Reading[J, 1] ← Reading[J+1, 1]`
	`  Reading[J+1, 1] ← Temp`
	`  //now swap corresponding ID`
	`  Temp ← Reading[J, 2]`
	`  Reading[J, 2] ← Reading[J+1, 2]`
	`  Reading[J+1, 2] ← Temp`
	`  NoSwaps ← FALSE`
	`  ENDIF`
	`  NEXT J`
	`  Boundary ← Boundary - 1`
	`  UNTIL NoSwaps = TRUE`
	
	`ENDPROCEDURE`
	
	MP1 Procedure heading **and** ending
	MP2 Conditional loop correctly formed including Boolean flag declared and initialised
	
	MP3 An inner loop
	MP4 Correct range 1 to 1999 for inner loop
	
	MP5 Comparison of element J with J+1 in a loop
	MP6 Declare `Temp` variable **and** swap elements in a loop
	MP7 **Both** SensorID **and** Speed values were swapped in a loop
	
	MP8 'No-Swap' mechanism:
	• Conditional **outer** loop including flag reset
	• Flag set in **inner** loop to indicate swap
	
	MP9 Reducing `Boundary` in the <u>outer</u> loop
		8

Examiner report missing
9618_s25_qp_21 Question 6

7(a)	Example solution:
	
	`FUNCTION CustomerOrder(Number, Points : INTEGER) RETURNS INTEGER`
	
	`   DECLARE NewPoints, NumberFree: INTEGER`
	
	`   NewPoints ← Points + Number`
	`   NumberFree ← 0`
	`   WHILE NewPoints >= 11 AND Number > 0`
	`      NumberFree ← NumberFree + 1`
	`      NewPoints ← NewPoints -11`
	`       Number ← Number - 1`
	`   END WHILE`
	
	`   OUTPUT "Number of free coffees is: ", NumberFree`
	`   RETURN NewPoints`
	`ENDFUNCTION`
	
	MP1 Function header **and** ending **and** parameters **and** return type
	MP2 Number of coffees ordered added to points
	
	MP3 Correct calculation of one free coffee
	
	MP4 Attempted calculation of multiple free coffees **and** reduced `NewPoints`
	
	MP5 Output number of free drinks with suitable message
	
	MP6 Return of `NewPoints`
		6

7(b)(i)	Example solution:
	
	`PROCEDURE AddNewCustomers(NumToAdd : INTEGER)`
	
	`   DECLARE CustomerID : INTEGER`
	`   DECLARE Count : INTEGER`
	`   DECLARE Line : STRING`
	`   DECLARE NewLine : STRING`
	`   OPENFILE "Loyalty.txt" FOR READ`
	
	`   WHILE NOT EOF("Loyalty.txt")`
	`      READFILE "Loyalty.txt", Line`
	`   ENDWHILE`
	`   CLOSEFILE "Loyalty.txt"`
	
	`   OPENFILE "Loyalty.txt" FOR APPEND`
	
	`   CustomerID ← STR_TO_NUM((LEFT(Line, 6))`
	
	`   FOR Count ← 1 TO NumToAdd`
	`      CustomerID ← CustomerID + 1`
	`      OUTPUT CustomerID`
	`      NewLine ← NUM_TO_STR(CustomerID) & ",0"`
	`      WRITEFILE "Loyalty.txt", NewLine`
	`   NEXT Count`
	
	`   CLOSEFILE "Loyalty.txt"`
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 All variables used are declared using the correct type including a string **and** an integer
	MP2 Open file in read mode **and** close (before opening in append mode)
	
	MP3 Conditional loop with `EOF("Loyalty.txt")`
	MP4 Read `Line` from file // Count number of records in the file
	
	MP5 Open the file in append mode **and** subsequently close
	MP6 Extract `CustomerID` from `Line` **and** convert to integer // use count from MP4 to generate last `CustomerID` stored
	MP7 A (count controlled) loop for the number of customers to add
	MP8 Increment `CustomerID` **and** output the new `CustomerID` in loop
	
	MP9 Create string for new `CustomerID` **and** write to file in loop
	
	Max 8
		8

Examiner report missing
9618_s25_qp_21 Question 7

1(b)		**3**

Variable	Example data value	Data type
`Available`	`TRUE`	BOOLEAN
`Received`	`"18/04/2021"`	STRING
`Index`	`100`	INTEGER

One mark per row

1(c)		**3**

Expression	Evaluates to
`Available AND NOT(Index > 100)`	TRUE
`Index MOD 30`	10
`NUM_TO_STR(Index + "33")`	ERROR

One mark per row

Examiner report missing
9618_w24_qp_23 Question 1

2(a)	`DECLARE Count, Total, NextNumber : INTEGER`
	
	`Total ← 0`
	`FOR Count ← 1 TO 100`
	`  OUTPUT "Input an integer value"`
	`  INPUT NextNumber`
	`  IF NextNumber > 0 THEN`
	`  Total ← Total + NextNumber`
	`  ENDIF`
	`NEXT Count`
	
	`OUTPUT Total`
	
	Mark as follows:
	MP1 Declarations of all variables used
	MP2 Loop for 100 iterations
	MP3 Prompt **and** input a value **in a loop and**
	MP4 Test for value > 0 // >=1 **in a loop**
	MP5 Sum the Total **in a loop**
	MP6 Output of Total after the **loop**
	
	Max 5 marks
		5

Examiner report missing
9618_w24_qp_23 Question 2

4(a)	One mark per highlighted part:
	
	`FOR Index ← 1 TO 5`
	`  Lower ← GB[Index] - 2`
	`  Upper ← GB[Index] + 2  // Lower + 4`
	`  IF Mark >= Lower AND Mark <= Upper THEN`
	`//IF Mark <= Upper AND Mark >= Lower THEN`
	`  OUTPUT "Check this paper"`
	`  ENDIF`
	`NEXT Index`
		4

4(b)(i)	**MP1** Use `Mark` as the index to the `Check` array // to specify an array element **MP2** If value of indexed element is `TRUE`, then the paper will need to be checked	**2**

4(b)(ii)	Example:solution
	
	`PROCEDURE GBInitialise()`
	`  DECLARE GBIndex, GBMark, ThisIndex : INTEGER`
	
	`  FOR ThisIndex ← 0 TO 75`
	`  Check[ThisIndex] ← FALSE // initialise all values`
	`  NEXT ThisIndex`
	
	`  FOR GBIndex ← 1 TO 5`
	`  GBMark ← GB[GBIndex]`
	`  FOR ThisIndex ← GBMark - 2 TO GBMark + 2`
	`  Check[ThisIndex] ← TRUE //Set GBMark ± 2 to TRUE`
	`  NEXT ThisIndex`
	`  NEXT GBIndex`
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 Procedure heading and ending
	MP2 Initialise `Check` array to `FALSE`
	MP3 Loop through `GB` array
	MP4 Extract the `GB` mark
	MP5 Loop for 5 elements across `GBMark` ± 2 // Check if within `GBMark` ± 2
	MP6 Assign each element of `Check` array to `TRUE`
	
	ALTERNATIVE – Example using selection Version 1
	
	`FOR ThisIndex ← 0 TO 75`
	`  CASE ThisIndex`
	`  GB[1] - 2 TO GB[1] + 2 : Check[ThisIndex] ← TRUE`
	`  GB[2] - 2 TO GB[2] + 2 : Check[ThisIndex] ← TRUE`
	`  GB[3] - 2 TO GB[3] + 2 : Check[ThisIndex] ← TRUE`
	`  GB[4] - 2 TO GB[4] + 2 : Check[ThisIndex] ← TRUE`
	`  GB[5] - 2 TO GB[5] + 2 : Check[ThisIndex] ← TRUE`
	`  OTHERWISE: Check[ThisIndex] ← FALSE`
	`  ENDCASE`
	`NEXT ThisIndex`
		6

Examiner report missing
9618_w24_qp_23 Question 4

6	Loop Solution
	Example solution:
	
	`FUNCTION AdjustClock(ThisYear : INTEGER) RETURNS INTEGER`
	`  DECLARE ThisDayNumber, SundayCount : INTEGER`
	`  DECLARE ThisDate : DATE`
	
	`  ThisDayNumber ← 0`
	`  SundayCount ← 0`
	`  REPEAT`
	`  ThisDayNumber ← ThisDayNumber + 1`
	`  ThisDate ← SETDATE(ThisDayNumber, 3, ThisYear)`
	`  IF DAYINDEX(ThisDate) = 1 THEN`
	`  SundayCount ← SundayCount + 1`
	`  ENDIF`
	`  UNTIL SundayCount = 3`
	`  RETURN ThisDayNumber`
	`ENDFUNCTION`
	
	Mark as follows:
	MP1 Function heading, parameter, ending and return type
	MP2 Declare local integer variable that is used to create a date
	MP3 Loop until 3rd Sunday found
	MP4 Attempt to use both `SETDATE()` **and** `DAYINDEX()` **in a loop**
	MP5 Correctly generate value of type `DATE` using `SETDATE()` **in a loop**
	MP6 Test if value represents a Sunday using `DAYINDEX()` **in a loop**
	MP7 Increment Sunday count **in a loop and** initialised correctly before loop
	MP8 Return day number of **third** Sunday
	
	Max 7 marks
	
	Non-loop solution
	Example solution:
	
	`FUNCTION AdjustClock(ThisYear : INTEGER) RETURNS INTEGER`
	`  DECLARE FirstDayIndex: INTEGER`
	`  DECLARE ThisDate : DATE`
	
	`  ThisDate ← SETDATE(1, 3, ThisYear)`
	`  FirstDayIndex ← DAYINDEX(ThisDate)`
	`  IF FirstDayIndex = 1 THEN`
	`  ThirdSunday ← FirstDayIndex + 14 //First day is Sunday`
	`  ELSE`
	`  ThirdSunday ← 23 - FirstDayIndex // Other days`
	`  ENDIF`
	`  RETURN ThirdSunday`
	`ENDFUNCTION`
		7

Examiner report missing
9618_w24_qp_23 Question 6

8(a)	Example solution
	
	`PROCEDURE Count(ThisPlayer : INTEGER, ThisRole : STRING)`
	`  DECLARE Index, Num, Total : INTEGER`
	
	`  Num ← 0`
	`  Total ← 0`
	
	`  FOR Index ← 1 TO 45`
	`  IF Character[Index].Player = ThisPlayer AND __`
	`  Character[Index].Role = ThisRole THEN`
	`  Num ← Num + 1`
	`  Total ← Total + Character[Index].SkillLevel`
	`  NEXT Index`
	
	`  IF Num > 0 THEN`
	`  OUTPUT "Player ", ThisPlayer, __`
	`  " has ", Num, " characters with the role of`
	`  ", ThisRole, " and the total skill level is`
	`  ", Total`
	`  ELSE`
	`  OUTPUT "No characters with that role are`
	`  assigned to this player"`
	`  ENDIF`
	
	`ENDPROCEDURE`
	
	MP1 Initialisation of local integers for `Num` and `Total`
	MP2 Loop through 45 elements
	
	MP3 Attempt to check Player and Role fields **in a loop**
	MP4 Correctly compare `Player` field with parameter **in a loop**
	MP5 Correctly compare `Role` field with parameter **in a loop**
	
	MP6 ... if player and role found, increment `Num` **and** sum Skill Total **in a loop**
	MP7 Test for any matches **after the loop**
	MP8 **Both** possible OUTPUT statements correctly formed following an attempt at MP6 but outputting one only
	
	Max 7 marks
		7

8(b)	Example solution:
	
	`PROCEDURE Restore()`
	`  DECLARE Index : INTEGER`
	`  DECLARE Line  : STRING`
	
	`  OPENFILE "SaveFile.txt" FOR READ`
	`  FOR Index ← 1 TO 45`
	`  READFILE "SaveFile.txt", Line`
	`  Character[Index].Player ← STR_TO_NUM(Extract(Line, 1))`
	`  Character[Index].Role ← Extract(Line, 2)`
	`  Character[Index].Name ← Extract(Line, 3)`
	`  Character[Index].Skill ← STR_TO_NUM(Extract(Line, 4))`
	`  NEXT Index`
	
	`  CLOSEFILE "SaveFile.txt"`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 Open the file in read mode and subsequently close
	MP2 Loop through 45 elements
	MP3 Read a line from the file **in a loop**
	MP4 Attempt to use `Extract()` **in a loop**
	MP5 Correct use of `Extract()` for all fields **in a loop**
	MP6 Use of `STR_TO_NUM()` on Player and Skill **in a loop**
	MP7 Completely correct extraction and assignment of all fields **in a loop**
		7

Examiner report missing
9618_w24_qp_23 Question 8

2(a)	Example 'loop solution':
	
	`FUNCTION Conceal(CardNumber : STRING) RETURNS STRING`
	`  DECLARE MaskedString : STRING`
	`  DECLARE Count : INTEGER`
	`  CONSTANT Asterisk = '*'`
	
	`  MaskedString ← RIGHT(CardNumber, 4)`
	`  FOR Count ← 1 TO LENGTH(CardNumber) - 4`
	`  MaskedString ← Asterisk & MaskedString`
	`  NEXT Count`
	
	`  RETURN MaskedString`
	
	`ENDFUNCTION`
	
	Mark as follows:
	MP1 Function heading **and** parameter **and** ending **and** return type
	MP2 Declaration of all local variables used - including the loop counter
	MP3 Calculate number of digits to mask / number of asterisks required
	
	MP4 use of a 'relevant' Loop
	MP5 Correct number of iterations
	MP6 Concatenate asterisk to start/end of `MaskedString` **in a loop**
	
	MP7 Assign last four characters to `MaskedString //` Concatenate the retained original last four digits
	MP8 Return masked string
	
	Max 6 marks
	
	ALTERNATIVE 'non loop' solution:
	
	`FUNCTION Conceal(CardNumber : STRING) RETURNS STRING`
	`  DECLARE MaskedString : STRING`
	`  DECLARE Count : INTEGER`
	`  CONSTANT Asterisks = "********************" //20 asterisks`
	
	`  Count ← LENGTH(CardNumber) - 4`
	`  MaskedString ← LEFT(Asterisks, Count) & RIGHT(CardNumber, 4)`
	
	`  RETURN MaskedString`
	
	`ENDFUNCTION`
	
	Mark as follows:
	MP1 Function heading **and** parameter **and** ending **and** return type
	MP2 Declaration of all local variables used
	MP3 Calculate number of digits to mask / number of asterisks required
		6

2(b)(i)	`DECLARE CardNumber : ARRAY [1:100, 1:2] OF STRING` **MP1** Correct dimensions **MP2** All other parts of the statement correct	**2**

2(b)(ii)	Any reference to BYREF // 'by reference'	**1**

Examiner report missing
9618_w24_qp_22 Question 2

4	Example solution:
	
	`PROCEDURE Timer(Mins, Secs : INTEGER)`
	`  DECLARE WarningTick, EndTick : INTEGER`
	
	`  EndTick ← Tick + 1000 * ((Mins * 60) + Secs)`
	`  WarningTick ← EndTick - (30 * 1000)`
	
	`  REPEAT`
	`  //do nothing`
	`  UNTIL Tick = WarningTick`
	`  OUTPUT "30 seconds to go"`
	
	`  REPEAT`
	`  //do nothing`
	`  UNTIL Tick = EndTick`
	
	`  OUTPUT "The time is up!"`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 Procedure heading **and** parameters **and** ending
	MP2 'Attempt' to calculate 'total time'/`EndTick` // 'elapsed time' // `WarningTick`
	MP3 Correct calculation of `EndTick` **and** `WarningTick`
	
	MP4 **(Design mark)**
	• Two separate loops – checking warning time then the final time, **OR** …
	• Single loop checking the final time with an IF statement to check for warning time, **OR** …
	• Single loop with two IF statements checking the warning time and final time
	
	MP5 Completely correct MP4
	MP6 Output both messages (must be meaningful and follow successful MP4)
		6

Examiner report missing
9618_w24_qp_22 Question 4

5(a)	**MP1** `Count ← INT(100 / Number)` Number could be zero (giving a divide by zero) **MP2** `Index ← Data[Number]` Potential error: Value `Number` could be outside the range of array indices **MP3** `ReturnValue ← TO_UPPER(RIGHT(Label, Count))` Potential Error: Number to extract may be too big / negative / out of range for use in the `RIGHT` function // `Label` has insufficient characters **MP4** `RETURN RetVal` Potential Error: There is no value to be returned // there is no variable named `RetVal` Mark as follows: 1 mark for each statement **and** description **Max 3 marks**	**3**

5(b)	**MP1** Construct: A (pre/post) conditional loop **MP2** Explanation: The terminating condition is never satisfied	**2**

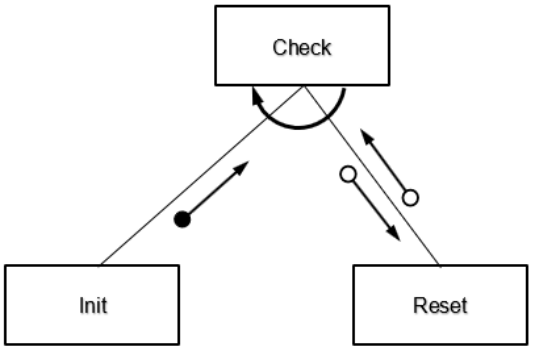
5(c)	Example solution*:*
	
	`IF Index Mod 2 = 0 THEN`
	`  ReturnValue ← TO_UPPER(RIGHT(Label, Count))`
	`ELSE`
	`  ReturnValue ← "****"`
	`ENDIF`
	
	Mark as follows:
	MP1 `IF...THEN...ELSE...ENDIF`
	MP2 Both correct assignments **and** the correct test/logic
		2

Examiner report missing
9618_w24_qp_22 Question 5

6(a)	Example solution:
	
	`PROCEDURE Special()`
	`  DECLARE Index : INTEGER`
	`  DECLARE Filling1, Filling2 : STRING`
	
	`  REPEAT`
	`  Index ← INT(RAND(35)) + 1`
	`  UNTIL Filling[Index] <> ""`
	
	`  Filling1 ← Filling[Index]`
	
	`  REPEAT`
	`  Index ← INT(RAND(35)) + 1`
	`  UNTIL Filling[Index] <> "" AND Filling1 <> Filling[Index]`
	
	`  Filling2 ← Filling[Index]`
	
	`  REPEAT`
	`  Index ← INT(RAND(10) + 1)`
	`  UNTIL Bread[Index] <> ""`
	
	`  OUTPUT "The daily special is ", Filling1, " and ", __ Filling2, " on ", Bread[Index], " bread."`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 Loop for Filling 1, avoiding unused elements
	MP2 Loop for Filling 2 avoiding unused elements
	MP3 Check Filling 2 is different from Filling 1 – could correctly compare either the indices or the array contents
	
	MP4 Loop for Bread, avoiding unused elements
	
	MP5 Using `RAND(10) / RAND(35)`
	MP6 Completely correct use of `RAND()` – including `INT()` and +1 in all cases
	
	MP7 Correct output - **once only** – following a reasonable attempt at selection of filings and bread
		7

Examiner report missing
9618_w24_qp_22 Question 6

7(a)	• Customer ID – to reference the other customer details • Email address – to send the email • Name – to personalise the email • Date of last visit – to select which customers should receive an email • Unique voucher code (or method of code generation) – to include in the email Mark as follows: One mark per item **and** justification **Max 3 marks**	**3**

7(d)	 **MP1** Three boxes correctly labelled **and** correct hierarchy **MP2** Parameter **and** return values **MP3** Iteration arrow	**3**

Examiner report missing
9618_w24_qp_22 Question 7

8(a)	Example solution:
	
	`PROCEDURE Assign(ThisRole : STRING, ThisPlayer : INTEGER)`
	`  DECLARE Index : INTEGER`
	`  DECLARE Done : BOOLEAN`
	
	`  Done ← FALSE`
	`  Index ← 1`
	
	`  WHILE Index < 46 AND Done = FALSE`
	`  IF Character[Index].Player = 0 AND __`
	`  Character[Index].Role = ThisRole THEN`
	`  Character[Index].Player ← ThisPlayer`
	`  Done ← TRUE`
	`  ELSE`
	`  Index ← Index + 1`
	`  ENDIF`
	`  ENDWHILE`
	
	`  IF Done = TRUE THEN`
	`  OUTPUT Character[Index].Name, " the ",__`
	`  Character[Index].Role, __`
	`  " has been assigned to player ", ThisPlayer`
	`  ELSE`
	`  OUTPUT "No characters with this role are available"`
	`  ENDIF`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 Loop until 'found' or all 45 elements considered
	MP2 Test of `Player` field – i.e. not value **in a loop**
	
	MP3 ... `AND Role` **–** i.e. match for **`ThisRole`** parameter **in a loop**
	
	MP4 If available character found, assign `ThisPlayer` to the character **in a loop**
	MP5 When character found set termination condition/flag
	MP6 **Both** OUTPUT messages logically correctly placed
	
	MP7 **Both** OUTPUT statements correctly formed
		7

8(b)	Example solution:
	
	`PROCEDURE Save()`
	`  DECLARE Index  : INTEGER`
	`  DECLARE Line  : STRING`
	`  CONSTANT SEP = '^'`
	
	`  OPENFILE "SaveFile.txt" FOR WRITE`
	`  FOR Index ← 1 TO 45`
	`  Line ← NUM_TO_STR(Character[Index].Player) & SEP`
	`  Line ← Line & Character[Index].Role & SEP`
	`  Line ← Line & Character[Index].Name & SEP`
	`  Line ← Line & NUM_TO_STR(Character[Index].Level)`
	`  WRITEFILE "SaveFile.txt", Line`
	`  NEXT Index`
	
	`  CLOSEFILE "SaveFile.txt"`
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 Declaration of local integer for `Index` (**and** string type for `Line`)
	MP2 Open `"SaveFile.txt"` in write mode **and** subsequently close
	MP3 Loop through 45 elements
	
	MP4 Attempt to form `Line` - four fields **in a loop**
	MP5 Correct use of `NUM_TO_STR()x2 (Player and Level)` **in a loop**
	MP6 Correct use of **<u>three</u>** &<separator>& `strings` **in a loop**
	
	MP7 Line from MP4 written to file **in a loop**
		7

Examiner report missing
9618_w24_qp_22 Question 8

1(a)	One mark per point: Error: The **calculation** of TaxPayable is incorrect Correction: TaxPayable ← (ItemCost * TaxRate) / 100	**2**

1(b)(i)	Use constants (to represent the tax rate values)	**1**

1(b)(ii)	One mark per bullet point (or equivalent to max 3): 1 **Tax rates** are entered once only 2 Avoids / Minimise (input) error(s) / changing the Tax rates accidentally // avoids different values for tax rates at different points in the program 3 When required, the constant value (representing a tax rate) is changed (once) // Easier to maintain / update the program (when the tax rates change) 4 Makes the program / code easier to understand	**3**

1(d)	OTHERWISE	**1**

Examiner report missing
9618_w24_qp_21 Question 1

2	Example solution:
	
	`PROCEDURE Tick()`
	`  SS ← SS + 1`
	`  IF SS = 60 THEN`
	`  SS ← 0`
	`  MM ← MM + 1`
	`  IF MM = 60 THEN`
	`  MM ← 0`
	`  HH ← HH + 1`
	`  IF HH = 24 THEN`
	`  HH ← 0`
	`  CALL NewDay()`
	`  ENDIF`
	`  ENDIF`
	`  CALL CheckAlarm()`
	`  ENDIF`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	1 Increment SS anywhere in the code
	2 Call `CheckAlarm()` when SS is set to zero
	3 Test if `SS` = 60 anywhere in the code …
	if so:
	4 set `SS` to 0
	5 increment `MM`
	6 When `MM` is set to 60 set `MM` to 0 **and** increment `HH`
	7 When `HH` is set to 24 set `HH` to 0 **and** call `NewDay()`
	
	Max 6 marks
		6

Examiner report missing
9618_w24_qp_21 Question 2

4(a)	One mark per point: **Type:** Run-time error **Cause:** A divide by zero operation is attempted // Attempt to convert a non-numeric string to a number	**2**

4(b)	Answers include: **Reason:** • This part of the algorithm performs a specific task // if the check or calculation is changed it is changed only once • A (similar) subroutine is already available // Library routine is already available • The program is simplified / easier to understand / easier to design / code / test / debug / maintain **Max 1 marks for 'Reason'** **Avoided:** • Unnecessary code duplication • Errors caused by differences where several copies of the check and calculation exist **Max 1 marks for 'Avoided'**	**2**

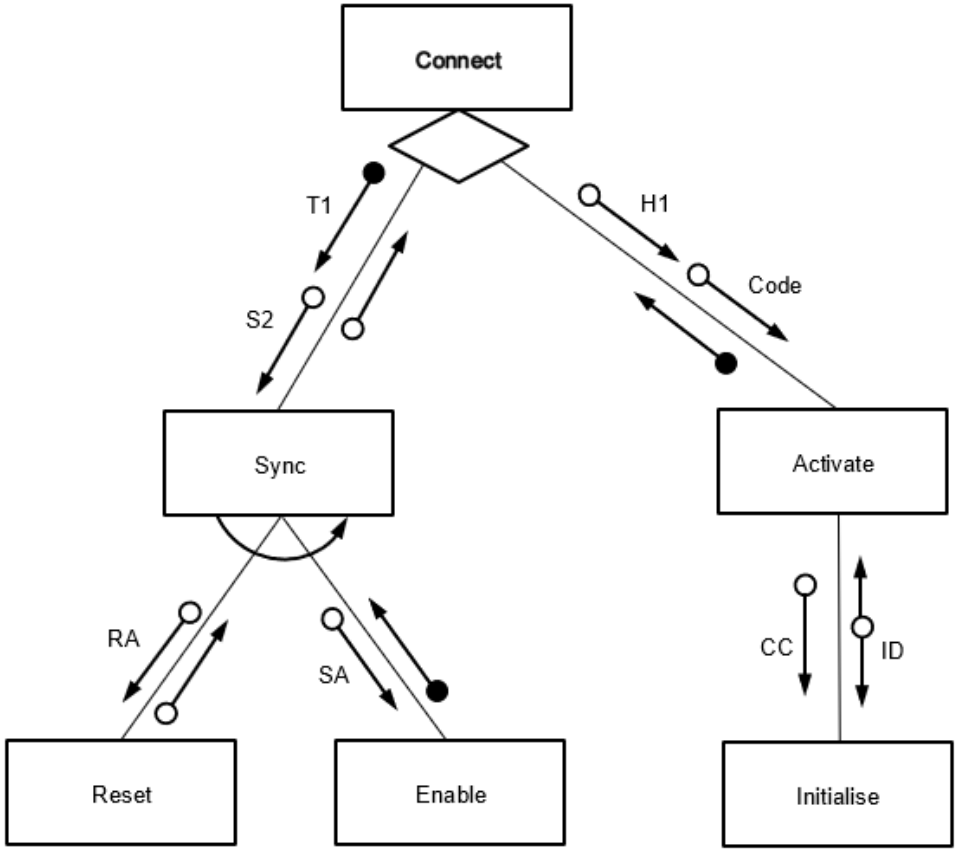
4(c)	Example solution:
	
	`FUNCTION Evaluate(NumStr1, NumStr2 : STRING) RETURNS Result`
	`  DECLARE RetVal : Result`
	`  DECLARE Num1, Num2 : REAL`
	
	`  RetVal.Done ← FALSE`
	
	`  IF IS_NUM(NumStr1) = TRUE AND IS_NUM(NumStr2) = TRUE THEN`
	`  Num1 ← STR_TO_NUM(NumStr1)`
	`  Num2 ← STR_TO_NUM(NumStr2)`
	`  IF Num2 <> 0 THEN`
	`  RetVal.Value ← (Num1 / Num2)`
	`  RetVal.Done ← TRUE`
	`  ENDIF`
	`  ENDIF`
	
	`RETURN RetVal`
	
	`ENDFUNCTION`
	
	Note that order of parameters is not specified so divisor could be either parameter.
	
	Mark as follows:
	
	1 Function heading, parameters, ending
	2 ... Correct return type of `Result`
	3 Local declaration of type `Result`
	4 Attempt at using `IS_NUM()` to check **both** parameters/strings
	5 Attempt at using of `STR_TO_NUM()` to convert **both** parameters/strings
	6 Check if the divisor, is non-zero ...
	7 … **and** if so correct calculation **and** assignment to `Value` field
	8 Return variable of type `Result` having assigned both fields
	
	Max 6 marks
		6

Examiner report missing
9618_w24_qp_21 Question 4

6(a)	One mark per point: 1 Fewer lines of code are needed 2 The program is simpler/ less complex // Program is easier to design / code / maintain / modify / test / debug 3 Direct access to days in a month / data (using month number as index) // Can use index / month to (directly) access days in month / data **Max 2 marks**	**2**

6(b)	Example Solution
	
	`FUNCTION GetDate(ProductionDate : DATE, ShelfLife : INTEGER) RETURNS DATE`
	`  DECLARE NewDate : DATE`
	`  DECLARE DD, MM, YY, LastDay : INTEGER`
	
	`  DD ← DAY(ProductionDate)`
	`  MM ← MONTH(ProductionDate)`
	`  YY ← YEAR(ProductionDate)`
	
	`  DD ← DD + ShelfLife`
	`  LastDay ← DaysInMonth[MM]`
	
	`  IF DD > LastDay THEN`
	`  MM ← MM + 1`
	`  DD ← DD – LastDay`
	`  IF MM = 13 THEN`
	`  MM ← 1`
	`  YY ← YY + 1`
	`  ENDIF`
	`  ENDIF`
	`  NewDate ← SETDATE(DD, MM, YY)`
	`  RETURN NewDate`
	
	`ENDFUNCTION`
	
	Mark as follows:
	
	1 Declare three variables of `TYPE INTEGER` to store `DD`, `MM` and `YY`
	2 Correct use of date functions from Insert to extract three integer values representing DD, MM and YY and use/assign to a variable
	3 Add parameter `ShelfLife` to `DD` and use / assign result to a variable
	4 Extract `LastDay` from `DaysInMonth` using `MM` as `Index`
	5 Test for new `DD` after end of month / `DaysInMonth[MM]`
	… and if `DD` > `DaysInMonth[MM]`:
	6 `MM` is incremented by 1 and `DD` is set to `DD – DaysInMonth[MM]`
	7 If `MM = 13 / MM > 12` then increment `YY` **and** set `MM` to 1
	8 Use of `SETDATE` to assign value to `NewDate`, `NewDate` must have been delared as a type `DATE`
	9 Return date
	
	Max 7 marks
		7

Examiner report missing
9618_w24_qp_21 Question 6

7(a)	 One mark per point: 1 All module identified in correct hierarchy 2 Parameters to both Sync and Activate **and** return values 3 Parameters to both Reset and Enable 4 Parameters to Initialise 5 Decision diamond	**5**

Examiner report missing
9618_w24_qp_21 Question 7

8(a)	Loop example solution
	
	`FUNCTION CheckMark(Mark : INTEGER) RETURNS BOOLEAN`
	`  DECLARE Index, Lower, Upper : INTEGER`
	
	`  FOR Index ← 1 TO 5`
	`  Lower ← GradeBoundary[Index] - 2`
	`  Upper ← GradeBoundary[Index] + 2`
	`  IF Mark >= Lower AND Mark <= Upper THEN`
	`  RETURN TRUE`
	`  ENDIF`
	`  NEXT Index`
	
	`  RETURN FALSE`
	`ENDFUNCTION`
	
	Mark as follows:
	1 Loop through all elements in `GradeBoundary` array
	2 Attempt to calculate range, both `Lower` and `Upper`, **in a loop**
	3 Completely correct range calculation **in a loop**
	4 Test if given mark / parameter, is within range **in a loop**
	5 Set variable / immediate RETURN if recheck required **in a loop**
	6 Return Boolean in both cases following a reasonable attempt
	
	Selection example solution
	
	`FUNCTION CheckMark(Mark : INTEGER) RETURNS BOOLEAN`
	
	`  CASE OF Mark`
	`  GradeBoundary[1] - 2 TO GradeBoundary[1] + 2 :`
	`  RETURN TRUE`
	`  GradeBoundary[2] - 2 TO GradeBoundary[2] + 2 :`
	`  RETURN TRUE`
	`  GradeBoundary[3] - 2 TO GradeBoundary[3] + 2 :`
	`  RETURN TRUE`
	`  GradeBoundary[4] - 2 TO GradeBoundary[4] + 2 :`
	`  RETURN TRUE`
	`  GradeBoundary[5] - 2 TO GradeBoundary[5] + 2 :`
	`  RETURN TRUE`
	`  OTHERWISE : RETURN FALSE`
	`  ENDCASE`
	
	`ENDFUNCTION`
	
	Mark as follows:
	1 Correct syntax used for selection structure(s)
	2 Correct checks for two ranges
	3 Correct checks for three ranges
	4 Correct checks for four ranges
	5 Correct checks for all five ranges
	6 Return Boolean in both cases following a reasonable attempt
		6

8(b)	Example Solution:
	
	`PROCEDURE CheckAll(CNum : INTEGER)`
	`  DECLARE Index, Count, ThisMark: INTEGER`
	
	`  Count ← 0`
	`  OPENFILE "GRList.txt" FOR WRITE`
	`  FOR Index ← 1 to CNum`
	`  ThisMark ← Result[Index, 1] // 2D array: mark + ID`
	`  IF CheckMark(ThisMark) = TRUE THEN`
	`  WRITE "GRList.txt", NUM_TO_STR(Result[Index, 2])`
	`  Count ← Count + 1`
	`  ENDIF`
	`  NEXT Index`
	`  CLOSEFILE "GRList.txt"`
	`  OUTPUT "There are ", Count, " papers to check"`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	1 Procedure heading, parameter and ending.
	2 Open file in write mode and subsequently close
	3 Loop through all `CNum` candidates
	4 Extract candidate mark from `Result` array **in a loop**
	5 Use of `CheckMark()` with candidate mark as parameter **in a loop**
	6 Test return value and if `TRUE` then increment `Count` **in a loop**
	7 ... write ID to file ...
	8 ... after conversion to a string **in a loop**
	9 Final output of message giving number of papers to check **not in a loop**
	
	Max 8 marks
		8

Examiner report missing
9618_w24_qp_21 Question 8

1(a)(i)	Two marks for the benefits: 1. The code can be called when needed 2. Any subsequent change to the algorithm / calculation needs to be made once only // Easier to manage / maintain (the program) 3. Less code / no code duplication 4. The algorithm / code / calculation can be designed / coded / tested **once** **Note: Max 2 marks**	**2**

1(a)(ii)	One mark per point:
	1. Create a module / function / procedure / subroutine **using the (old) code / algorithm**
	2. Replace the old code / algorithm wherever it appears with a call to the module / function / procedure / subroutine
	3. Pass the data as parameter(s) // Use of global variable(s)
	4. Return the result (of the calculation) // Assign result to global variable(s) / BYREF parameter(s)
	5. Create local variables as needed (to perform the calculation)
	
	Note: Max 3 marks
		3

1(b)	<table><tr><th>Pseudocode expression</th><th>Evaluates to</th></tr><tr><td>**MID**("Random", 2, 3)</td><td>"and"</td></tr><tr><td>5 + **DAY**(10/11/2023)</td><td>15</td></tr><tr><td>**IS_NUM**("45000")</td><td>TRUE</td></tr><tr><td>(20 **MOD** 3) + 1</td><td>3</td></tr></table> One mark per row	**4**

Examiner report missing
9618_s24_qp_23 Question 1

2(a)	One mark per point: 1. Open the file (in read mode and subsequently close) 2. Initialise an index variable (to 1 // 0) 3. Repeat (the next three steps) until the end of file is reached 4. Read a line from the file (into a string variable) 5. Store the line / variable in the array at the index 6. Increment the index **in a loop** **Note: max 5 marks**	**5**

2(b)	One mark per point: Construct: a conditional loop Use: To keep repeating until the **end of the file is reached** ALTERNATIVE: Construct: a selection statement Use: To test / check the value returned by the `EOF()` function	**2**

Examiner report missing
9618_s24_qp_23 Question 2

4(a)	Example solution:
	
	`PROCEDURE TwoParts()`
	`  DECLARE NextNum, Average : REAL`
	
	`  TotalA ← 0.0 // 0`
	`  TotalB ← 0.0 // 0`
	
	`  REPEAT`
	`  INPUT NextNum`
	`  TotalA ← TotalA + NextNum`
	`  UNTIL NextNum = 0`
	
	`  REPEAT`
	`  INPUT NextNum`
	`  TotalB ← TotalB + NextNum`
	`  UNTIL NextNum = 0`
	
	`  Average ← (TotalA + TotalB) / 2`
	`  OUTPUT "The average is ", Average`
	`ENDPROCEDURE`
	
	Mark as follows:
	1. Procedure heading and ending
	2. Declare all local variables
	3. Initialise `TotalA` and `TotalB`
	4. First conditional loop until zero entered, summing `TotalA` // Loop until both parts (sequences) have been entered
	5. Second conditional loop until zero entered, summing `TotalB` // Loop summing appropriate Totals
	6. Calculation of average and output with a message // Calculation of the average for the values making up the two totals and both output with a suitable message
		6

Examiner report missing
9618_s24_qp_23 Question 4

5(a)	One mark per point • Count-controlled loop • the number of iterations is known	**2**

5(b)	Two mark for Statement of Problem: 1 The functions will return the same value every time they are called 2 ... because `Label` / the parameter value does not change within the loop Two marks for Solution: 3 Assign `FormatA(Label)` and `FormatB(Label)` to two (local) variables <u>before</u> the loop 4 Use the (new) variables in place of the function calls / in the loop // Replace the references to `FormatA()` and `FormatB()` in the `CASE` clauses with the new variable names	**4**

Examiner report missing
9618_s24_qp_23 Question 5

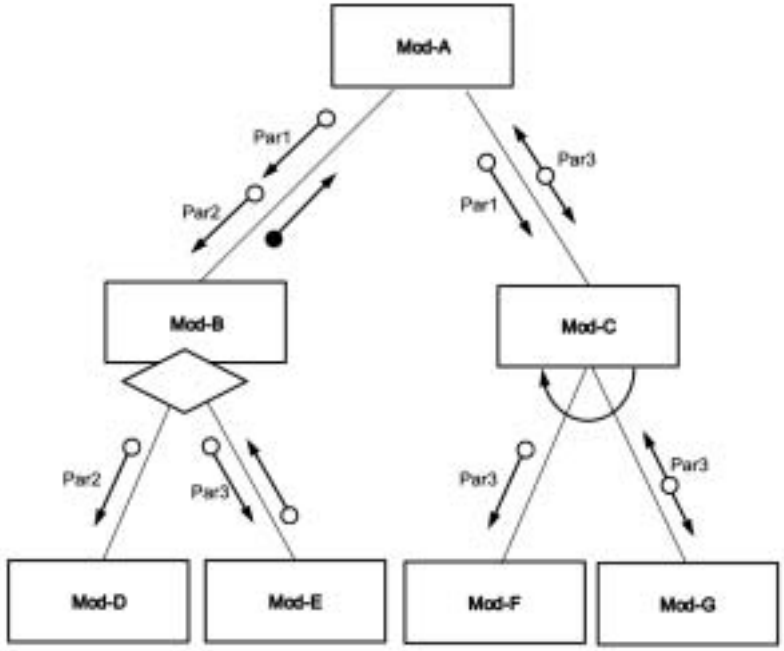
6(a)	Example Solutions:
	
	`PROCEDURE Progress(Percent : INTEGER, Root : STRING)`
	`  DECLARE StepValue : INTEGER`
	`  DECLARE Filename : STRING`
	
	`  StepValue ← (Percent DIV 10) + 1 // INT(Percent / 10) + 1`
	`  FileName ← Root & "-" & NUM_TO_STR(StepValue) & ".bmp"`
	`  CALL Display(Filename)`
	`ENDPROCEDURE`
	
	Alternative:
	
	`PROCEDURE Progress(Percent : INTEGER, Root : STRING)`
	`  DECLARE StepValue : INTEGER`
	`  DECLARE Filename : STRING`
	`  DECLARE Found : BOOLEAN`
	`  StepValue ← 1`
	`  Found ← FALSE`
	`  REPEAT`
	`  IF Percent < StepValue * 10 THEN`
	`  Found ← TRUE`
	`  ENDIF`
	`  StepValue ← StepValue + 1`
	`  UNTIL Found`
	`  Filename ← Root & "-" & NUM_TO_STR(StepValue - 1) & ".bmp"`
	`  CALL Display(Filename)`
	`ENDPROCEDURE`
	
	Mark as follows for use of DIV/INT or loop solution:
	1 Procedure heading, parameters and ending
	2 Calculate `StepValue` as integer value // Attempt at calculating file number
	3 Add 1 to obtain file number // Completely correct file number calculation
	4 Use of `NUM_TO_STR()` to convert file number to string and use
	5 Concatenate `Root`, hyphen, file number and ".bmp" suffix
	6 Call `Display()` with filename as parameter following a reasonable attempt
		6

6(b)(i)	Example answer: `FUNCTION Progress2(Percent, Steps : INTEGER, Root : STRING) RETURNS STRING` One mark for each: • Keyword `FUNCTION Progress2` **and** three parameters of correct type • Keyword `RETURNS` and type string	**2**

Examiner report missing
9618_s24_qp_23 Question 6

7(a)(i)	Mod-B() and Mod-E()	**1**

7(a)(ii)	Points required: 1 any change made to the parameter value / `Par3` within `Mod-G()` is reflected in the (subsequent) value in the calling module / `Mod-C()` (after `Mod-G()` terminates) 2 any change made to the parameter value / `Par3` within `Mod-F()` is NOT reflected in the (subsequent) value in the calling module / `Mod-C()` (after `Mod-F()` terminates) Mark as follows: 1 mark for a reasonable attempt to explain 2 marks for full explanation including context	**2**

7(b)	 One mark per bullet: 1 All modules correctly labelled and interconnected 2 Parameters between Mod-A and Mod-B and return value from Mod-B 3 Parameters between Mod-A and Mod-C 4 Diamond applied to Mod-B only 5 Iteration arrow applied to Mod-C only 6 All parameters at lower level and return value	**6**

Examiner report missing
9618_s24_qp_23 Question 7

8(a)	Example:
	
	`FUNCTION Header(Line : STRING) RETURNS STRING`
	
	`   IF LENGTH(Line) >= 13 THEN`
	`      IF TO_UPPER(LEFT(Line, 9)) = "FUNCTION " THEN`
	`         RETURN "F" & RIGHT(Line, LENGTH(Line) - 9)`
	`      ENDIF`
	
	`      IF TO_UPPER(LEFT(Line, 10)) = "PROCEDURE " THEN`
	`         RETURN "P" & RIGHT(Line, LENGTH(Line) - 10)`
	`      ENDIF`
	`   ENDIF`
	
	`   RETURN ""`
	
	`ENDFUNCTION`
	
	Mark as follows:
	
	1 Function heading, parameter, return type and ending
	2 Check that the line is at least 13 characters long before attempting to extract and return empty string
	3 Attempt at: Extract characters, 9 or 10 characters, corresponding to keyword plus space and compare with appropriate keyword plus space
	4 Completely correct MP3
	5 Use of type case conversion to allow for 'any case'
	6 Calculation of 'rest of line' and concatenation with 'P or 'F'
	7 Return string
		7

8(b)	Example:
	
	`PROCEDURE FindModules(FileName : STRING)`
	`  DECLARE Line : STRING`
	`  DECLARE Index, LineNum : INTEGER`
	
	`  OPENFILE FileName FOR READ`
	
	`  Index ← 1`
	`  LineNum ← 0`
	
	`  WHILE NOT EOF(FileName)`
	`  READFILE FileName, Line`
	`  LineNum ← LineNum + 1`
	`  Line ← Header(Line)`
	`  IF Line <> "" THEN`
	`  ModInfo[Index, 1] ← NUM_TO_STR(LineNum)`
	`  ModInfo[Index, 2] ← LEFT(Line, 1)`
	`  ModInfo[Index, 3] ← RIGHT(Line, LENGTH(Line) - 1)`
	`  Index ← Index + 1`
	`  ENDIF`
	`  ENDWHILE`
	
	`  CLOSEFILE FileName`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	
	1 Open file in `READ` mode **and** subsequently close
	2 Loop to `EOF(FileName)`
	3 Read a line from the file and maintain `LineNum` in **a loop**
	4 Call `Header()` and use return value **in a loop**
	5 Test return value for "" **in a loop**
	6 Attempt at all three-array assignment for all columns in correct row
	7 Correct values assigned to all columns of array
	8 Maintain correct array row index
		8

Examiner report missing
9618_s24_qp_23 Question 8

1(a)		**4**

Pseudocode example	Selection	Iteration	Input/Output

`FOR Index ← 1 TO 10`
`  Data[Index] ← 0`
`NEXT Index`
		✓	

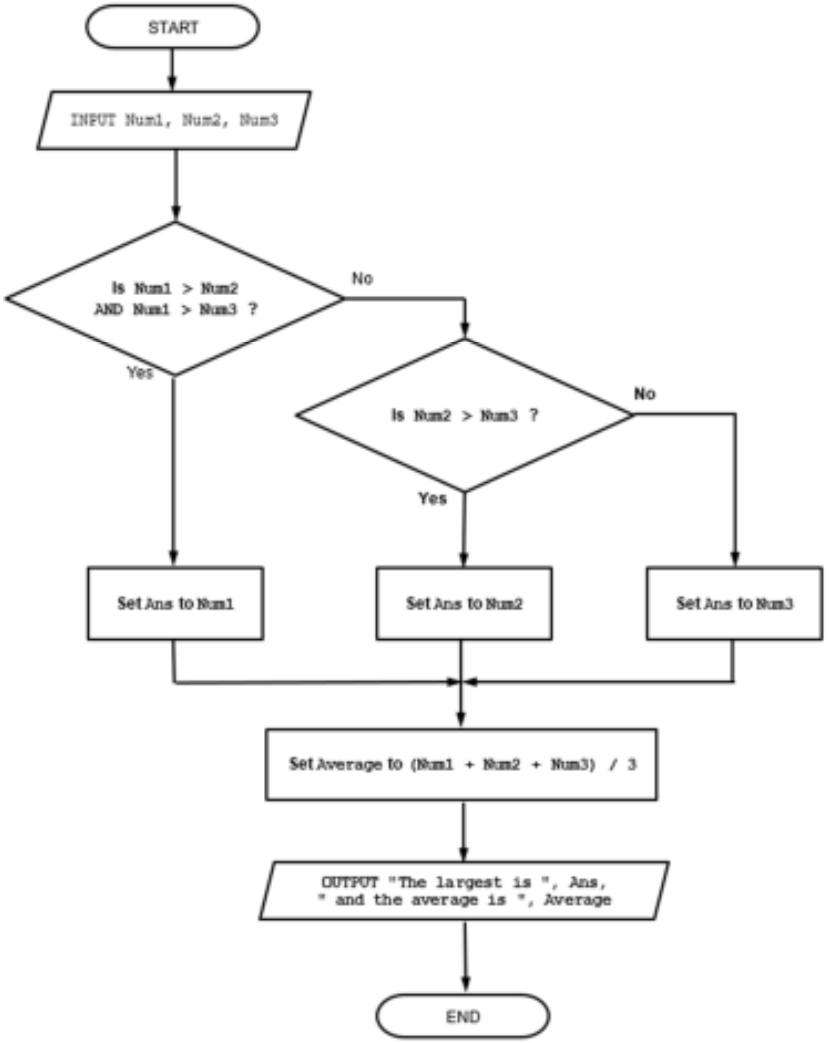
`WRITEFILE ThisFile, "*****"`			✓
`UNTIL Level > 25`		✓	

`IF Mark > 74 THEN`
`  READFILE OldFile, Data`
`ENDIF`
	✓		✓

One mark per row.

1(b)	**Expression**
	`MyInt ← INT (3.1415926)`
	`MyChar ← MID ("Elwood", 3, 1)`
	Any of:
	• `MyString ← NUM_TO_STR (INT (27.509))`
	• `MyString ← CHR (INT (27.509))`
	• `MyString ← TO_UPPER( NUM_TO_STR( 27.509))`
	• `MyString ← TO_LOWER( NUM_TO_STR( 27.509))`
	Any of:
	• `MyInt ← STR_TO_NUM ( RIGHT ("ABC123", 3))`
	• `MyInt ← LENGTH ( RIGHT ("ABC123", 3))`
	• `MyInt ← LENGTH ( LEFT ("ABC123", 3))`
	One mark per row
		4

Examiner report missing
9618_s24_qp_22 Question 1

2(a)	 Mark points 1 Condition for selecting one of the input numbers as largest value 2 … and assign to `Ans` 3 Condition for selecting largest number for all three number input and assigning to `Ans` 4 Average calculated using `Num1`, `Num2` and `Num3` and stored in a variable. Reject use of DIV 5 Output of `Ans` and average in output symbol / parallelogram	**5**

2(b)	Example solutions:
	
	`Flag ← GetStat()`
	`WHILE Flag <> TRUE`
	`  FOR Port ← 1 TO 3`
	`  CALL Reset(Port)`
	`  NEXT Port`
	`  Flag ← GetStat()`
	`ENDWHILE`
	
	Alternative:
	
	`REPEAT`
	`  Flag ← GetStat()`
	`  IF Flag <> TRUE THEN`
	`  FOR Port ← 1 TO 3`
	`  CALL Reset(Port)`
	`  NEXT Port`
	`  ENDIF`
	`UNTIL Flag = TRUE`
	
	One mark per point:
	1 (Outer) conditional loop testing `Flag`
	2 Correct assignment of `Flag` from `GetStat()` **in a loop**
	3 (Inner) loop checking / counting port // Check if `Port` is different to 4
	4 … loop for 3 iterations
	5 a call to `Reset()` **in a loop**
		5

Examiner report missing
9618_s24_qp_22 Question 2

4	Example solution:
	
	`PROCEDURE IsRA()`
	`  DECLARE a, b, c : INTEGER`
	
	`  OUTPUT "Input length of the first side"`
	`  INPUT a`
	`  OUTPUT "Input length of the second side"`
	`  INPUT b`
	`  OUTPUT "Input length of the third side"`
	`  INPUT c`
	
	`  IF (a * a = (b * b) + (c * c)) OR__`
	`  (b * b = (a * a) + (c * c)) OR__`
	`  (c * c = (a * a) + (b * b)) THEN`
	`  OUTPUT "It is right-angled"`
	`  ELSE`
	`  OUTPUT "Not right-angled"`
	`  ENDIF`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	
	1. Procedure heading and ending **and** declaration of all variables used
	2. Appropriate prompt and input for each length
	3. **One** correct length test
	4. **All three** length tests // selection of which test is required
	5. Output one of two messages following a reasonable attempt at MP3
		5

Examiner report missing
9618_s24_qp_22 Question 4

5(a)(i)	One mark per error: Syntax: 1. `NEXT Index` (should be `ENDWHILE)` 2. '&' used to concatenate an integer (in `OUTPUT` statement) Other: 3. Accesses element outside range // Accesses element 0	**3**

5(a)(ii)	One mark per point: Statement: • The `OTHERWISE` statement Explanation: • The result of MOD 2 can only be 0 or 1	**2**

Examiner report missing
9618_s24_qp_22 Question 5

6(a)	Example solution:
	
	`Function Trim(Name : STRING) RETURNS STRING`
	`  CONSTANT Dots = "..."`
	`  CONSTANT Space = " "`
	
	`  IF LENGTH(Name) <= 16 THEN`
	`  RETURN Name`
	`  ENDIF`
	
	`  // Otherwise it has to be trimmed`
	
	`  WHILE LENGTH(Name) > 13`
	`  REPEAT`
	`  Name ← LEFT(Name, LENGTH(Name) - 1) // strip last char`
	`  UNTIL RIGHT(Name, 1) = Space // back to SPACE`
	`  ENDWHILE`
	
	`  Name ← LEFT(Name, LENGTH(Name) - 1) // remove the space`
	`  Name ← Name & Dots`
	`  RETURN Name`
	
	`ENDFUNCTION`
	
	Mark as follows:
	1. Function heading, ending, parameter and return type
	2. If length of original string <= 16 then return original string
	3. Any Conditional loop...
	4. until string is short enough
	5. Inner conditional loop / second condition to identify word(s) to remove from the end of string // Check to identify word(s) to remove from the end of string
	6. Attempt to strip characters back to space
	7. Correct removal of word/words from end of string **and** remove final space
	8. Concatenate `Dots` and return result
	
	Max 7 marks
		7

Examiner report missing
9618_s24_qp_22 Question 6

8(a)	One mark for reason, one for benefit Reason: (Program is) easier to design / implement / test / debug / modify Benefit: Easier to check that **each stage** works as expected	**2**

8(b)	Example algorithm based on finding position of first non-space character and then using substring function:
	
	`FUNCTION DeleteSpaces(Line : STRING) RETURNS STRING`
	`  DECLARE NewLine : STRING`
	`  DECLARE EndOfLeading : BOOLEAN`
	`  DECLARE Count, NumSpaces : INTEGER`
	`  DECLARE NextChar : CHAR`
	`  CONSTANT Space = " "`
	
	`  NumSpaces ← 0`
	`  EndOfLeading ← FALSE`
	
	`  FOR Count ← 1 TO LENGTH(Line)`
	`  NextChar ← MID(Line, Count, 1)`
	`  IF NextChar <> Space AND EndOfLeading = FALSE THEN`
	`  NumSpaces ← Count - 1 // the number to trim`
	`  EndOfLeading = TRUE`
	`  ENDIF`
	`  NEXT Count`
	
	`  NewLine ← RIGHT(Line, LENGTH(Line) - NumSpaces)`
	
	`  RETURN NewLine`
	`ENDFUNCTION`
	
	Mark as follows:
	
	1 Loop to length of parameter `// Loop until first non-space character in Line`
	2 Extract a character **in a loop**
	3 Identify <u>first</u> non-space character **in a loop**
	4 Attempt at removing leading spaces in `Line`
	5 Leading spaces removed from `Line` / / Create new string without leading space
	6 Return a string following a reasonable attempt at removing leading spaces in `Line`
		6

8(c)	Example:
	
	`PROCEDURE Stage_2(F1, F2 : STRING)`
	
	`   DECLARE Line : STRING`
	`   DECLARE Count : INTEGER`
	
	`   Count ← 0`
	
	`   OPEN F1 FOR READ`
	`   OPEN F2 FOR APPEND`
	
	`   WHILE NOT EOF(F1)`
	`      READFILE F1, Line`
	`      Line ← DeleteSpaces(Line)`
	`      Line ← DeleteComment(Line)`
	`      IF Line <> "" THEN`
	`         WRITEFILE F2, Line  // skip blank lines`
	`      ELSE`
	`         Count ← Count + 1`
	`      ENDIF`
	`   ENDWHILE`
	
	`   CLOSEFILE F1`
	`   CLOSEFILE F2`
	
	`   OUTPUT Count, " blank lines were removed"`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	
	1 Procedure heading, parameters, ending
	2 Open both files in correct modes **and** subsequently close
	3 Loop to `EOF(F1)`
	4 Read a line from `F1` **in a loop**
	5 Assign return values from `DeleteComment()` and `DeleteSpaces()` **in a loop**
	6 Check return value following both MP5 function calls is not an empty string and if so write to `F2` **in a loop**
	7 Count the blank lines **in a loop**
	8 Output number of blank lines removed following a reasonable attempt **after the loop**
		8

Examiner report missing
9618_s24_qp_22 Question 8

1(a)		**4**

Assignment statement	Data type
`A ← LEFT(MyName, 1)`	CHAR / STRING
`B ← Total * 2`	INTEGER / REAL
`C ← INT(ItemCost) / 3`	REAL
`D ← "Odd OR Even"`	STRING

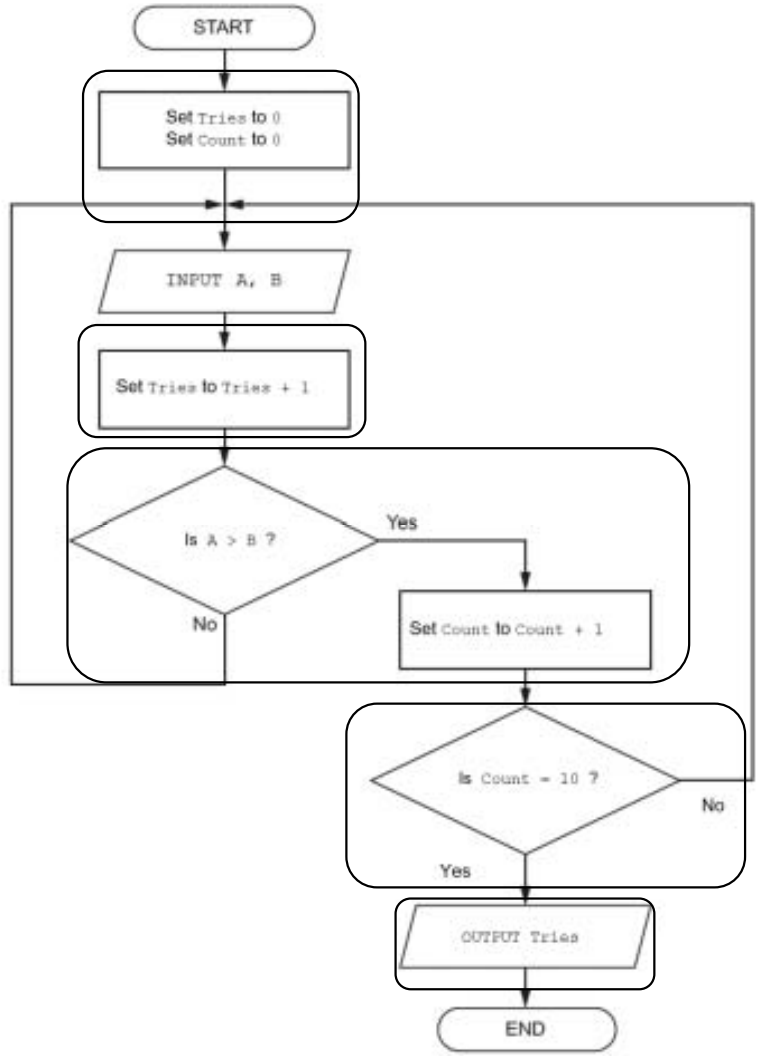
One mark per row

1(b)		**4**

Expression	Evaluates to
`Tries < 10 AND NOT Sorted`	TRUE
`Tries MOD 4`	1
`TO_LOWER(MID(ID, 3, 1))`	'a' // "a"
`LENGTH(ID & "xx") >= Tries`	TRUE

One mark per row

Examiner report missing
9618_s24_qp_21 Question 1

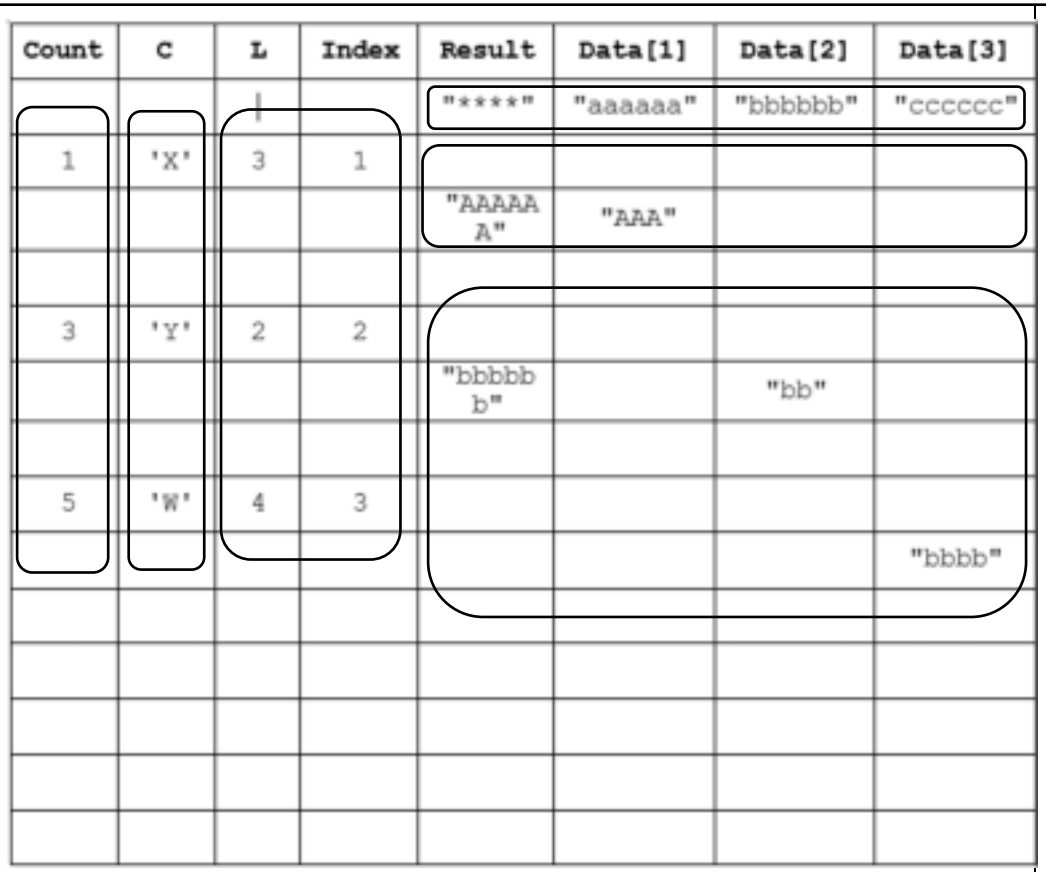
2(a)	 One mark per outlined region: 1 Initialise both counts 2 Increment `Tries` every time a pair is input 3 Compare A > B **and** increment `Count` if TRUE 4 Test for Count = 10 (10th time A > B) – MUST include Yes / No labels 5 If so output `Tries`, otherwise loop	**5**

2(b)	One mark per point: 1 A (variable of type) string will be input // by example e.g. “67,72” 2 A special / identified character would need to be used to separate each numeric value // all numbers are fixed length	**2**

Examiner report missing
9618_s24_qp_21 Question 2

4	Example:
	
	`FUNCTION Check() RETURNS STRING`
	`  DECLARE Odd, Even, Index : INTEGER`
	
	`  Odd ← 0`
	`  Even ← 0`
	
	`  FOR Index ← 1 TO 100`
	`  IF Index MOD 2 = 0 THEN`
	`  Even ← Even + Data[Index]`
	`  ELSE`
	`  Odd ← Odd + Data[Index]`
	`  ENDIF`
	`  NEXT Index`
	
	`ENDFUNCTION`
	
	Mark as follows:
	1. Function heading, ending and return type
	2. Declare local variables `Odd`, `Even` and `Index` as integers
	3. Initialise `Odd` and `Even`
	4. Loop for 100 `iterations` // through array
	5. Sum `Odd` and `Even` element values **in a loop**
	6. Compare `Odd` and `Even` after the loop and Return appropriate string
		6

Examiner report missing
9618_s24_qp_21 Question 4

5(a)	 One mark per zone.	**6**
5(b)(i)	OTHERWISE : CALL Error(C)	**1**
5(b)(ii)	After the 'Z' clause in the CASE construct // before the ENDCASE	**1**

Examiner report missing
9618_s24_qp_21 Question 5

6(a)	``` FUNCTION IsRA(x1, y1, x2, y2, x3, y3 : INTEGER) RETURNS BOOLEAN DECLARE Len1, Len2, Len3 : INTEGER Len1 ← (x1 - x2) ^ 2 + (y1 - y2) ^ 2 Len2 ← (x1 - x3) ^ 2 + (y1 - y3) ^ 2 Len3 ← (x2 - x3) ^ 2 + (y2 - y3) ^ 2 IF (Len1 = Len2 + Len3) OR__ (Len2 = Len1 + Len3) OR__ (Len3 = Len1 + Len2) THEN RETURN TRUE ELSE RETURN FALSE ENDIF ENDFUNCTION ``` Mark as follows: 1 Calculate the square of the length of at least one side 2 Calculation of all three lengths squared correctly 3 **One** correct comparison of square of three lengths 4 **All three** comparisons… 5 combined using logical operators / nested `IF` // completely correct selection 6 Return result correctly in both cases	**6**

Examiner report missing
9618_s24_qp_21 Question 6

7(a)(i)	To make the solution easier to design / implement / solve	**1**

7(b)(i)	Means that `Update` calls (one of) either `Sub-A, Sub-B` or `Sub-C` One mark for each point: • reference to selection / decision / if • naming all four modules correctly	**2**

7(b)(ii)	`PROCEDURE Sub-A (Name : STRING, BYREF P2 : BOOLEAN)`
	
	`FUNCTION Sub-B (P1 : REAL) RETURNS REAL`
	
	One mark per underlined part in each case
		4

Examiner report missing
9618_s24_qp_21 Question 7

8(a)	```
	FUNCTION DeleteComment(Line : STRING) RETURNS STRING
	   DECLARE NewLine, TwoChars : STRING
	   DECLARE Count, TrimTo : INTEGER
	   CONSTANT Comment = "//"
	
	   NewLine ← Line
	   TrimTo ← 0
	   Count ← 1
	   WHILE Count < LENGTH(Line) AND TrimTo = 0
	      TwoChars ← MID(Line, Count, 2) // extract 2 chars
	      IF TwoChars = Comment THEN
	         TrimTo ← Count
	      ENDIF
	      Count ← Count + 1
	   ENDWHILE
	
	   IF TrimTo <> 0 THEN
	      NewLine ← LEFT(Line, TrimTo - 1)
	   ENDIF
	
	   RETURN NewLine
	
	ENDFUNCTION
	``
	
	Mark as follows:
	
	1 Loop to length of `Line` (parameter) // length of `Line -1`
	2 Terminate loop on first double slash
	3 Attempt to extract one/two characters **in a loop**
	4 Check for "//" after attempt at extraction **in a loop**
	5 Record start position of comment // Calculate amount of trim required
	6 Attempt to trim `Line` from start of comment
	7 Completely correct trimmed `Line` from start of comment
	8 Return `Line` after a reasonable overall attempt
		8

8(b)	Example:
	
	`FUNCTION Stage_1(StudentName : STRING) RETURNS INTEGER`
	
	`   DECLARE OldFile, NewFile, Line : STRING`
	`   DECLARE Count : INTEGER`
	
	`   OldFile ← StudentName & "_src.txt"`
	`   NewFile ← StudentName & "_S1.txt"`
	`   OPENFILE OldFile FOR READ`
	`   OPENFILE NewFile FOR WRITE`
	`   Count ← 0`
	
	`   WHILE NOT EOF(OldFile)`
	`      READFILE OldFile, Line`
	`      Line ← DeleteComment(Line)`
	`      IF LENGTH(Line) <> 0 THEN`
	`         WRITEFILE NewFile, Line`
	`         Count ← Count + 1`
	`      ENDIF`
	`   ENDWHILE`
	
	`   CLOSEFILE OldFile`
	`   CLOSEFILE NewFile`
	
	`   RETURN Count`
	`ENDFUNCTION`
	
	Mark as follows:
	
	1 Generate filenames condone missing "_"
	2 Open both files in correct modes **and** subsequently close
	3 Loop to `EOF(OldFile)`
	4 Read a line from `OldFile` **and** execute `DeleteComment()` **in a loop**
	5 Skip blank lines after `DeleteComment()` **in a loop**
	6 Write `Line` to stage 1 file **and** increment Count
	7 Return the number of lines after a reasonable attempt at counting
		7

Examiner report missing
9618_s24_qp_21 Question 8

1(a)		**4**

Assignment statement	Data type
MyVar1 ← Total1 / Total2	REAL
MyVar2 ← 27/10/2023	DATE
MyVar3 ← "Sum1 / Sum2"	STRING
MyVar4 ← Result1 AND Result2	BOOLEAN

1(b)		**4**

Expression	Evaluates to
Fraction >= 0.2 AND NOT Active	**FALSE**
INT((Fraction1 * 100) + 13.3)	**33**
STR_TO_NUM(MID(Code, 4, 2)) + 5	**28**
LENGTH("TRUE" & Code)	**11**

1(c)	The use of a program library (routines)	**1**

Examiner report missing
9618_w23_qp_23 Question 1

4(a)	``` PROCEDURE RandList() DECLARE Count, BaseNum, ThisNum : INTEGER CONSTANT StepVal = 10 BaseNum ← 0 FOR Count ← 1 TO 25 ThisNum ← BaseNum + INT(RAND(StepVal)) OUTPUT ThisNum BaseNum ← BaseNum + StepVal NEXT Count ENDPROCEDURE ``` **MP1** Procedure heading and ending **MP2** Local loop counter `Count` as integer **MP3** Loop to iterate 25 times or more for each unique number **MP4** 'Attempt' to generate a random number including use of `INT()` **in a loop** **MP5** Ensure that number generated is greater than previous and change 'previous' **MP6** Output random number after an attempt at MP5 **in a loop**	**6**

4(b)	**One** mark for simplified logical expression: **MP1** `Result[x + 1] <= Result[x]` ALTERNATIVE SOLUTION: `Result[x] >= Result[x + 1]`	**1**

Examiner report missing
9618_w23_qp_23 Question 4

5		6

Index	Value	Total	Mix[1]	Mix[2]	Mix[3]	Mix[4]
2		0	4	2	3	5
	2	2				
				5		
3						
	3	5				
					8	
4						
	5	10				
						9
2						
	5	15				
				13		
2						
	13	28				
				21		
2						
			56			

MP1 Row 1 (initialisation)

Each iteration (1 – 5):
MP2 1 – Total 2
MP3 2 – Total 5
MP4 3 – Total 10
MP5 4 – Total 15
MP6 5 – Total 28 **and** final `Mix[1] =` 56

Examiner report missing
9618_w23_qp_23 Question 5

6	
	`FUNCTION TestNum(ThisNum : STRING) RETURNS INTEGER`
	
	`   IF LEFT(ThisNum,3) = RIGHT(ThisNum 3) THEN`
	`      RETURN 3`
	`   ENDIF`
	
	`   IF RIGHT(ThisNum, 3) = "000" THEN`
	`      RETURN 2`
	`   ENDIF`
	
	`   IF MID(ThisNum, 4, 1) = MID(ThisNum, 5, 1)__`
	`      AND MID(ThisNum, 5, 1) = MID(ThisNum, 6, 1) THEN`
	`      RETURN 1`
	`   ENDIF`
	
	`   RETURN 0`
	
	`ENDFUNCTION`
	
	MP1 Function heading and ending **including** parameter and return type
	MP2 Test for Condition 1
	MP3 Test for Condition 2
	MP4 Test for Condition 3
	MP5 Return the highest value if more than one condition is satisfied
	MP6 Return zero if no condition matched
		6

Examiner report missing
9618_w23_qp_23 Question 6

7(a)	**MP1** iteration / looping **MP2** naming all four modules correctly in the correct sequence // e.g. `Module-A` repeatedly calls `Sub-Y1`, then `SubY2` then `Sub-9`	**2**

7(b)(i)	`TYPE MyType`
	`  DECLARE RA  : INTEGER`
	`  DECLARE RB  : STRING`
	`  DECLARE RC  : BOOLEAN`
	`ENDTYPE`
	
	MP1 `TYPE MyType... ENDTYPE`
	MP2 `RA` as integer and `RB3` as string
	MP3 `RC` as Boolean
		3

7(b)(ii)	`PROCEDURE Sub-9(BYREF Param : MyType)` **MP 1** One mark for `BYREF` **MP 2** One mark for the rest of the statement	**2**

Examiner report missing
9618_w23_qp_23 Question 7

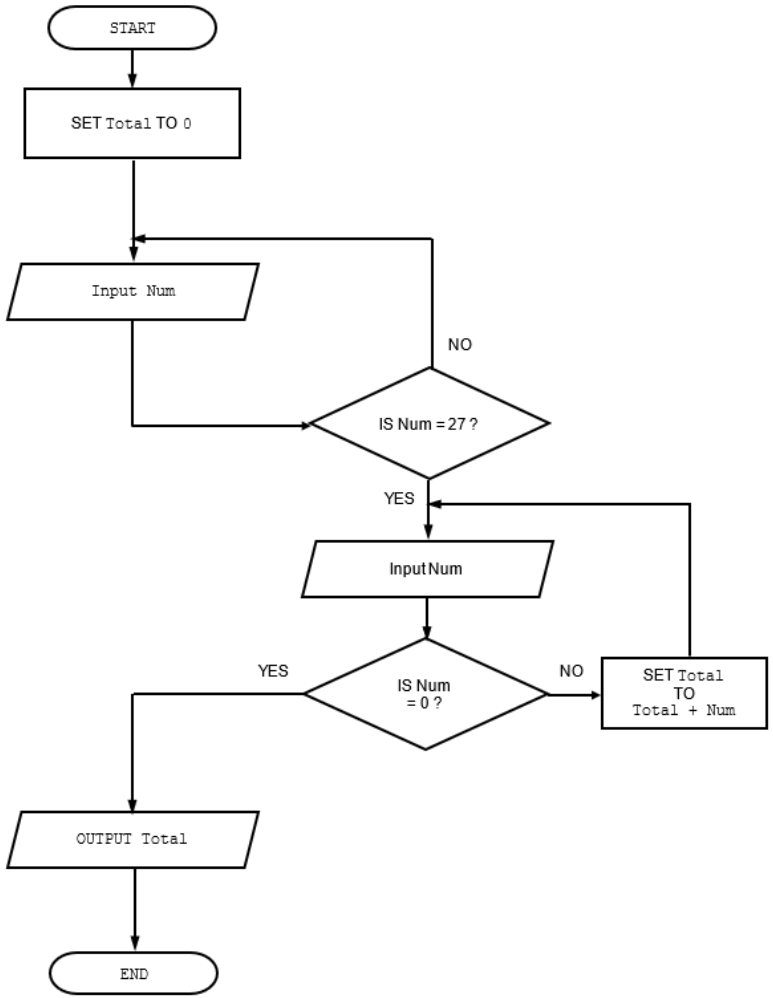
8(a)	 PROCEDURE ReceiveFile(FileName : STRING) DECLARE FileData : STRING DECLARE CharCount : INTEGER CONSTANT Terminator = "****" OPENFILE FileName FOR WRITE FileData ← GetData() CharCount ← 0 WHILE FileData <> Terminator WRITEFILE FileName, FileData CharCount ← CharCount + LENGTH(FileData) FileData ← GetData() ENDWHILE CLOSEFILE FileName OUTPUT CharCount (, " characters were written to ", FileName) ENDPROCEDURE **MP1** OPEN file in WRITE mode and subsequently CLOSE **MP2** Conditional loop until terminator received **MP3** 'Attempted' use of `GetData()` – Ignore CALL … **MP4** Fully correct use `GetData()` to return the data **in a loop** **MP5** Maintain `CharCount` **in a loop** **MP6** Write each line to the file - **except** the terminator **in a loop** **MP7** Final output of message giving number of characters written **outside loop**	**7**

8(c)	 PROCEDURE Chat(Destination : STRING, Port : INTEGER) DECLARE Data : STRING DECLARE Finished : BOOLEAN CONSTANT Terminator = "Bye" CONSTANT STX = CHR(2) CONSTANT ETX = CHR(3) Finished ← FALSE REPEAT Data ← GetData() OUTPUT Data IF Data = Terminator THEN Finished ← TRUE ENDIF IF NOT Finished THEN //about to reply INPUT Data Transmit(STX & Destination & MyID & Data & ETX, Port) IF Data = Terminator THEN Finished ← TRUE ENDIF ENDIF UNTIL Finished = TRUE ENDPROCEDURE Conditional loop **MP1 Conditional loop** **MP2** Test for terminator in both cases **MP3** Use `GetData()` to get the data from the message **MP4** `OUTPUT` the data **in a loop** **MP5** `INPUT` the data reply **MP6** 'Attempted ' use of `Transmit` to send it **in a loop** **MP7** Correct formation of parameters to `Transmit()`	**7**

Examiner report missing
9618_w23_qp_23 Question 8

1(a)	**One** mark for each row with appropriate variable name and data type **Example value** / **Explanation** / **Variable name** / **Data type** `"Mr Khan"` / The name of the customer / **`CustomerName`** / **`STRING`** `3` / The number of items in the order / **`NumItems`** / **`INTEGER`** `TRUE` / A value to indicate whether this is a new customer / **`NewCustomer`** / **`BOOLEAN`** `15.75` / The deposit paid by the customer / **`Deposit`** / **`REAL`**	**4**
1(b)	**One** mark per row **Expression** / **Evaluates to** `(Total * DepRate) + 1.5` / **`249.50`** `RIGHT(Description, 7)` / **`"(small)"`** `(LENGTH(Description) - 8) > 16` / **`TRUE`** `NUM_TO_STR(INT(DepRate * 10)) & '%'` / **`"20%"`**	**4**

Examiner report missing
9618_w23_qp_22 Question 1

2(a)	Example Solution  **One** mark per point: 1 Initialise `Total` to zero 2 Check for only **first** input of 27 **in a loop** then attempt to sum values 3 Loop until 0 input 4 Sum values **input** (after input of first 27) **in a loop** 5 Output `Total` after a reasonable attempt	**5**

2(b)	**One** mark per point: 1 Name: (pre / post) conditional loop 2 Justification: the number of iterations is not known // loop ends following a specific input (in the loop)	**2**

Examiner report missing
9618_w23_qp_22 Question 2

4(a)	Example Solution
	
	`PROCEDURE Count()`
	`  DECLARE COdd, CEven, ThisNum : INTEGER`
	
	`  COdd ← 0`
	`  CEven ← 0`
	
	`  INPUT ThisNum`
	
	`  WHILE ThisNum <> 99`
	`  IF ThisNum MOD 2 = 1 THEN`
	`  COdd ← COdd + 1`
	`  ELSE`
	`  CEven ← CEven + 1`
	`  ENDIF`
	`  INPUT ThisNum`
	`  ENDWHILE`
	
	`  OUTPUT "Count of odd and even numbers: ", COdd, CEven`
	
	`ENDPROCEDURE`
	
	Mark as follows **Max 6** marks:
	
	1 Procedure heading and ending
	2 **Local** `COdd`, `CEven` and `ThisNum` declared as integers
	3 Conditional loop while `ThisNum <> 99`
	4 Input `ThisNum` **in a loop**
	5 Check `ThisNum` is odd or even **in a loop**
	6 Increment appropriate count **in a loop**, both counts must have been initialised **before loop**
	7 **After the loop** output `COdd` and `CEven` **with a** suitable message following a reasonable attempt at counting
		6

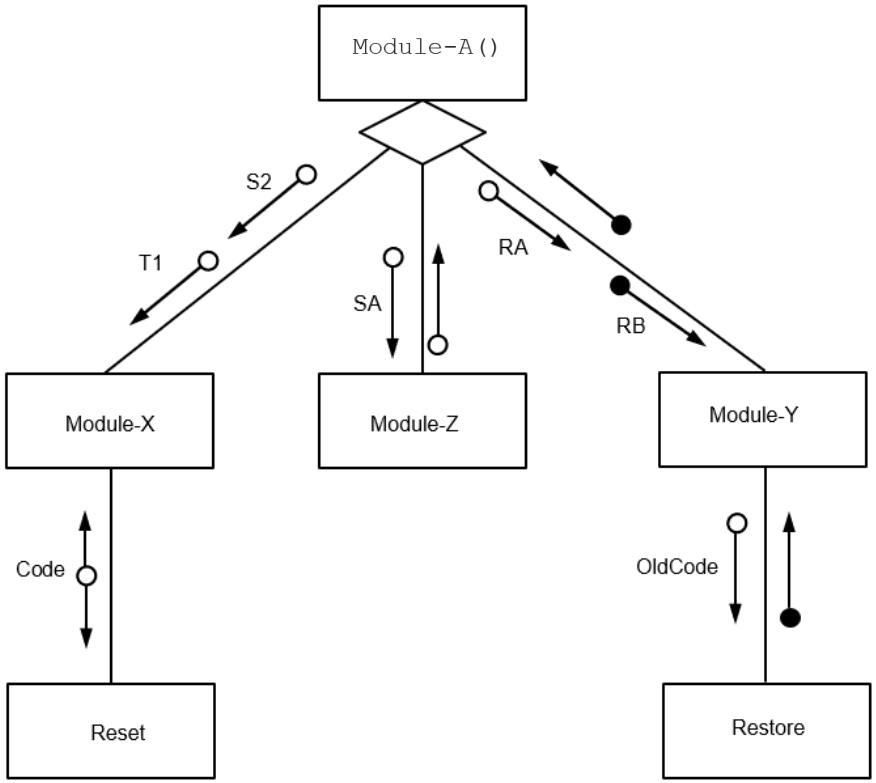
Examiner report missing
9618_w23_qp_22 Question 4

6(a)	Example Solution
	
	`PROCEDURE CreateFiles(NameRoot : STRING, NumFiles : INTEGER)`
	`  DECLARE FileName, Suffix : STRING`
	`  DECLARE Count : INTEGER`
	
	`  FOR Count ← 1 TO NumFiles`
	
	`  Suffix ← NUM_TO_STR(Count)`
	`  WHILE LENGTH(Suffix) <> 3`
	`  Suffix ← '0' & Suffix`
	`  ENDWHILE`
	
	`  FileName ← NameRoot & '.' & Suffix`
	`  OPENFILE FileName FOR WRITE`
	`  WRITEFILE FileName, "This is File " & FileName`
	`  CLOSEFILE FileName`
	
	`  NEXT Count`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	1 Procedure heading, including parameters, and ending
	2 Loop for `NumFiles` iterations
	3 Attempt to create **filename suffix** using `NUM_TO_STR()` **in a loop**
	4 Completely correct filename
	5 `OPENFILE` in `WRITE` mode and subsequent `CLOSE` **in a loop**
	6 `WRITE` initial first line to the file **in a loop**
		6

6(b)(i)	Function	**1**

6(b)(ii)	`FUNCTION CheckFiles(NameRoot : STRING) RETURNS INTEGER`	**1**

Examiner report missing
9618_w23_qp_22 Question 6

7(a)	 **One** mark per point: 1 Module names 2 Parameters with labels to Module-X and between Module-X and Reset 3 Parameter (with label) to Module-Z and return from Module-Z 4 Parameters with labels to Module-Y and Restore and return values from Module-Y	**4**

Examiner report missing
9618_w23_qp_22 Question 7

8(a)	Solution using a loop
	
	Example solution – using a loop
	
	`FUNCTION GetPort(ThisDest : STRING) RETURNS INTEGER`
	`  DECLARE Index, DNum, Port : INTEGER`
	
	`  DNum ← STR_TO_NUM(ThisDest)`
	`  Index ← 1`
	`  Port ← -1`
	
	`  REPEAT`
	`  IF RouteTable[Index, 1] <> -1 THEN`
	`  IF DNum >= RouteTable[Index, 1] AND __`
	`  DNum <= RouteTable[Index, 2] THEN`
	`  Port ← RouteTable[Index, 3]`
	`  ENDIF`
	`  ENDIF`
	`  Index ← Index + 1`
	`  UNTIL Index = 7 OR Port <> -1  // end of array or range found`
	
	`  RETURN Port`
	
	`ENDFUNCTION`
	
	Mark as follows **Max 7**:
	1.Function heading and ending **including** parameter and return type
	2 Convert **parameter** to a number
	3 (Conditional) loop through array
	4 Skip unused element **in a loop**
	5 Attempt to check one range with destination **in a loop**
	6 Test **all** ranges correctly with destination **in a loop**
	7 Store port value if destination matched **in a loop** (and exit loop)
	8 Return port value including -1 if no match found
	
	Solution using selection statement(s)
	
	Example solution
	
	`FUNCTION GetPort(ThisDest : STRING) RETURNS INTEGER`
	`  DECLARE Index, DNum, Port : INTEGER`
	
	`  DNum ← STR_TO_NUM(ThisDest)`
	`  Port ← -1`
	
	`  IF RouteTable[1, 1] <> -1 AND RouteTable[1, 1] >= DNum AND RouteTable[1, 2] <= DNum THEN`
		7

8(b)	Example solution
	
	`PROCEDURE ProcessMsg(ThisMsg : STRING)`
	`  DECLARE ThisDest : STRING`
	`  DECLARE Response : BOOLEAN`
	`  DECLARE StackNum : INTEGER`
	
	`  IF LENGTH(ThisMsg) >= 4 THEN`
	
	`  ThisDest ← LEFT(ThisMsg, 3)`
	
	`  IF ThisDest = MyID THEN  // It's for this computer`
	`  StackNum ← 1`
	`  ELSE`
	`  StackNum ← 2`
	`  ENDIF`
	
	`  Response ← StackMsg(ThisMsg, StackNum)`
	
	`  IF Response =  FALSE THEN`
	`  OUTPUT "Message discarded - no room on stack"`
	`  ENDIF`
	
	`  ENDIF`
	`ENDPROCEDURE`
	
	Mark as follows:
	1. Ignore message if data field is empty
	2. Extract `ThisDest` from `ThisMsg`
	3. Test if destination is this computer
	4. Attempt to use `StackMsg()`
	5. **Fully correct** use of `StackMsg()` for **both** cases / stacks
	6. Test `StackMsg()` return value for **both** cases
	7. Following a reasonable attempt at MP6 output warning if `StackMsg()` returns `FALSE`
		7

Examiner report missing
9618_w23_qp_22 Question 8

1(a)	**One** mark per row: |	Answer
---	---
The value assigned to `Level` when `ThisValue` is 40	**"Medium"**
The value assigned to Check when `ThisValue` is 36	**12**
The value assigned to Level when `ThisValue` is 18	**"Low"**
The number of elements in array Data that may be incremented	**11**		**4**
1(b)	**One** mark for identifying assignment: **MP1** `Level ← "Very Low"` // the level is assigned value “very low” Explanation points: **MP2** because CASE clauses are checked in sequence // because of the order of the clauses **MP3** a value < 30 satisfies the first clause // Clause '< 20' will never be tested	**3**
1(c)	**MP1** all of the possible values are addressed via all / four / three / the other clauses // there are no other possible values to map to `OTHERWISE`	**1**

Examiner report missing
9618_w23_qp_21 Question 1

2(a)	**Max 5** marks **MP1** Set total to zero **MP2** Input a number **MP3** Check if number greater than 29 and less than 71 **MP4** … if check is true - add number to total **MP5** Repeat from step 2 99 times // for a total of 100 iterations **MP6** Output the total	**5**

2(b)	**MP1** An iterative construct // a (count-controlled) loop **MP2** A selection construct // an IF statement	**2**

Examiner report missing
9618_w23_qp_21 Question 2

4(a)	 FUNCTION TooMany(Search : STRING, Max : INTEGER) RETURNS BOOLEAN DECLARE Count, Index : INTEGER Count ← 0 FOR Index ← 1 TO 150 IF Data[Index] = Search THEN Count ← Count + 1 ENDIF NEXT Index IF Count > Max THEN RETURN TRUE ELSE RETURN FALSE ENDIF ENDFUNCTION **MP1** Function heading, ending and return type **MP2** Declare Count and Index as integers **MP3** Initialise Count **MP4** Loop (any type) for 150 iterations **MP5** Compare Data element with parameter - if equal, increment Count in a loop **MP6** Compare Count with Max and return Boolean in both cases outside the loop	6

4(b)	**MP1** Test for row being even number
	MP2 Test for either column value equal to `Search`
	
	`IF Row MOD 2 = 0 AND __`
	
	`(Data[Row, 1] = Search OR Data[Row, 2] = Search) THEN`
	
	ALTERNATIVE using nested `IFs`:
	
	`IF Row MOD 2 = 0 THEN`
	
	`IF Data[Row, 1] = Search OR Data[Row, 2] = Search THEN`
	
	MP3 Selection structure is either:
	• Single IF statement using AND, or
	• Two nested IFs using AND, or
	• Single IF and the use of a two-iteration loop
	Either of these structures correctly formed scores the mark
	
	ALTERNATIVE SOLUTION: A FOR loop using 'STEP 2'
	
	`FOR Row ← 2 TO 150 / NEXT Row STEP 2`
	
	`Data[Row, 1] = Search OR Data[Row, 2] = Search) THEN`
		3

Examiner report missing
9618_w23_qp_21 Question 4

5(a)	**MP1** `Num > Min` should be `Num < Min` **MP2** `Count & 1` should be `Count + 1` **MP3** `NextInput ← "END"` should be `NextInput = "END"`	**3**

Examiner report missing
9618_w23_qp_21 Question 5

6(a)	`PROCEDURE MyOutput(NewString : STRING, EOL : BOOLEAN)`
	
	`  IF LENGTH(MyString) + LENGTH(NewString) > 255 THEN`
	`  OUTPUT MyString  // Resulting string would be too long`
	`  MyString ← NewString`
	`  ELSE`
	`  MyString ← MyString & NewString // Concat with MyString`
	
	`  IF EOL = TRUE THEN`
	`  OUTPUT MyString`
	`  MyString ← ""`
	`  ENDIF`
	`  ENDIF`
	
	`ENDPROCEDURE`
	
	MP1 Procedure heading, including parameters, and ending
	MP2 Produce concatenated string
	MP3 … Check whether resulting string would be too long
	
	MP4 If so, then output old `MyString`
	MP5 … and assign `NewString` to `MyString`
	MP6 Else concatenate `NewString` to `MyString`
	
	MP7 (test for length < 255) Test `EOL` – If `TRUE` then Output
	MP8 … and reset `MyString` to empty string
		7

6(b)	**MP1** A new (instance of) variable `MyString` is created each time the procedure is called / executed **MP2** So the previous contents are lost	**2**

Examiner report missing
9618_w23_qp_21 Question 6

8(a)	
	`PROCEDURE SendFile(FileName, DestID : STRING, Port : INTEGER)`
	`  DECLARE FileData : STRING`
	
	`  CONSTANT STX = CHR(02)`
	`  CONSTANT ETX = CHR(03)`
	
	`  OPENFILE FileName FOR READ`
	
	`  WHILE NOT EOF(FileName)`
	`  READFILE FileName, FileData`
	`  FileData ← STX & DestID & MyID & FileData & ETX`
	`  CALL Transmit(FileData, Port)`
	`  ENDWHILE`
	
	`  CLOSEFILE FileName`
	
	`  CALL Transmit(STX & DestID & MyID & "****" & ETX, Port)`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 OPEN file in READ mode – using parameter - and subsequently CLOSE
	MP2 Conditional loop to `EOF()`
	MP3 Use of `READFILE` to get a line from the file
	MP4 'Attempt' to form a message (minimum is `DestID, MyID, FileData`)
	MP5 Message formed is completely correct
	MP6 Call `Transmit()` with correct MP4 string **in a loop**
	MP7 Transmit the "****" message (all parts present) **after the loop**
		7

8(d)	`FUNCTION GetField(Msg : STRING, FieldNo : INTEGER)`
	`  RETURNS STRING`
	`  DECLARE RetString : STRING`
	
	`  CASE OF FieldNo`
	`  1 : RetString ← MID(Msg, 2, 3)`
	`  2 : RetString ← MID(Msg, 5, 3)`
	`  3 : RetString ← MID(Msg, 8, LENGTH(Msg) - 8)`
	`  OTHERWISE : RetString ← ""`
	`  ENDCASE`
	
	`  RETURN RetString`
	
	`ENDFUNCTION`
	
	MP1 Use of `CASE ... ENDCASE` or `IF ... THEN ... ENDIF`
	MP2 Field 1 and Field 2 extracted correctly
	MP3 Calculate a length of field 3
	MP4 Field 3 extracted correctly
	MP5 Return empty string in case of invalid parameter (via `OTHERWISE` or initialisation)
	MP6 Final `RETURN`, after a reasonable attempt
		6

Examiner report missing
9618_w23_qp_21 Question 8

1(a)	**Answer** The dimension of the array: **2** The name of the variable used as an array index: **PCount** The number of elements in the array: **100**	**3**

1(d)	One mark for each of rows 2 - 5	**4**

Pseudocode statement	Input	Process	Output
`INPUT MyChoice`	✓		
`OUTPUT FirstName & LastName`		✓	✓
`WRITEFILE OutputFile, TextLine`			✓
`READFILE MyFile, TextLine`	✓		
`Result ← SQRT(NextNum)`		✓	

Examiner report missing
9618_s23_qp_23 Question 1

2(a)	One mark for each underlined part `IF DAYINDEX(MyDOB)  = 5  THEN`	**2**

Examiner report missing
9618_s23_qp_23 Question 2

4	 FUNCTION MakeString(Count : INTEGER, AChar : CHAR) RETURNS STRING DECLARE MyString : STRING DECLARE Index : INTEGER IF Count < 1 THEN MyString ← "ERROR" ELSE MyString ← "" FOR Index ← 1 TO Count MyString ← MyString & AChar NEXT Index ENDIF RETURN MyString ENDFUNCTION MP1 Function heading and end including parameters and return type MP2 Declaration of locals `Index` and `MyString` MP3 Test for `Count` < 1 and if true, assign "ERROR" to `MyString` / Immediate RETURN MP4 Loop for `Count` iterations MP5 Use of concatenate – must have been initialised **in a loop** MP6 Return STRING (correctly in both cases) 	**6**

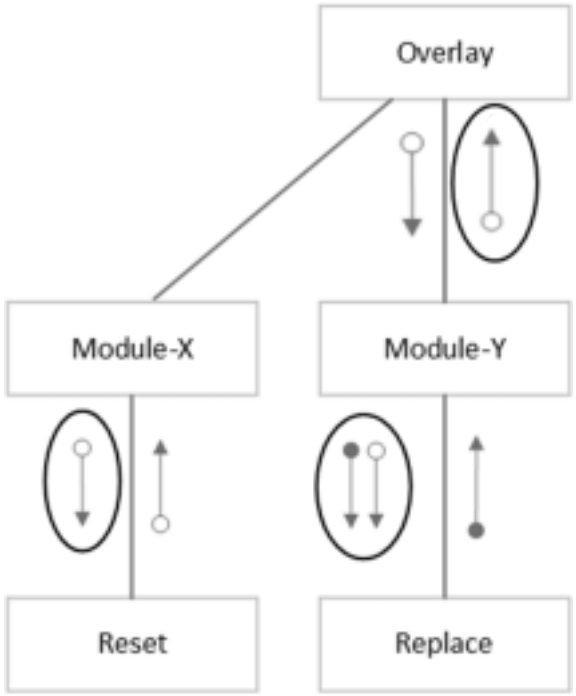
Examiner report missing
9618_s23_qp_23 Question 4

6(a)	**Max 7 marks**
	
	`PROCEDURE Select(Start, End : INTEGER)`
	`  DECLARE ThisNum, Total: INTEGER`
	`  DECLARE ThisString : STRING`
	`  DECLARE Char1, Char2 : CHAR`
	
	`  FOR ThisNum ← Start+1 TO End-1`
	`  ThisString ← NUM_TO_STR(ThisNum)`
	`  Char1 ← RIGHT(ThisString, 1)`
	`  Char2 ← LEFT(RIGHT(ThisString, 2), 1)`
	`  Total ← STR_TO_NUM(Char1) + STR_TO_NUM(Char2)`
	`  IF Total = 6 THEN`
	`  OUTPUT ThisString`
	`  ENDIF`
	`  NEXT ThisNum`
	
	`ENDPROCEDURE`
	
	MP1 Procedure heading and ending
	MP2 (Count-controlled) Loop
	MP3 …. with correct range from Start+1 to End-1
	MP4 Convert `ThisNum` (loop counter) to a string
	MP5 Extract the last two/first/second 'character digit(s)' required **in a loop**
	MP6 Extract the second individual 'character digit' required **in a loop**
	MP7 Calculate the sum of the last two digits
	MP8 If sum = 6 then OUTPUT the number (either string or integer) **in a loop**
		7

6(b)	**Max 4 marks** MP1 The function will take two integer parameters - the number and the (required) total MP2 … and return a Boolean OR: `CheckNum(Number,Total : INTEGER)` MP1 `RETURNS BOOLEAN` MP2 Two marks for the advantages: MP3 `CheckNum()` can be called repeatedly as and when required MP4 `CheckNum()` is designed and tested once (then used repeatedly) MP5 Any subsequent change to `CheckNum()` needs to be made once only // is easier to maintain/modify	**4**

Examiner report missing
9618_s23_qp_23 Question 6

7(a)(i)	• To filter out information (that is not necessary to solve the problem) // to include only essential information	**1**

7(b)	**Max 3 marks**  MP1 All modules correctly labelled and interconnected. MP2 Correct parameters and return type to Module-X and Reset MP3 Correct parameters and return type to Module-Y and Replace MP4 Correct parameters and return type to Overlay and Module-Y	**3**

Examiner report missing
9618_s23_qp_23 Question 7

8(a)		**8**

```
FUNCTION ChangeSupp(Code1, Code2 : STRING) RETURNS
                                                    INTEGER
  DECLARE Count : INTEGER
  DECLARE ThisLine, ThisCode : STRING

  OPENFILE "Stock.txt" FOR READ
  OPENFILE "NewStock.txt" FOR WRITE
  Count ← 0
  WHILE NOT EOF("Stock.txt")
     READFILE("Stock.txt ", ThisLine) // brackets
                                                    optional
     ThisCode ← MID(ThisLine, 5, 3)
     IF ThisCode = Code1 THEN
        ThisLine ← LEFT(ThisLine, 4) & Code2
                        & RIGHT(ThisLine,
                                  LENGTH(ThisLine) - 7)
        Count ← Count + 1
     ENDIF
     WRITEFILE("NewStock.txt", ThisLine) // brackets
                                                    optional
  ENDWHILE

  CLOSEFILE "NewStock.txt"
  CLOSEFILE "Stock.txt"

RETURN Count
ENDFUNCTION
```

MP1 Open both files, in correct modes, and subsequently close
MP2 Conditional loop until `EOF("Stock.txt")`
MP3 Read a line from `Stock.txt` **AND** extract `ThisCode` **in a loop**
MP4 Test `ThisCode = Code1` AND if true, increment `Count` (must have been Initialised) **in a loop**
MP5 Update `ThisLine` using substring functions and '&' **in a loop**
MP6 completely correct update of `ThisLine` **in a loop**
MP7 Write `ThisLine` to `NewStock.txt` **in a loop**
MP8 Return count after loop

8(b)		**6**

```
PROCEDURE Report_1(Supp : STRING)
  DECLARE Count : INTEGER
  DECLARE ThisItemNum, ThisDesc, ThisLine, ThisCode :
                                                    STRING

  Count ← 0

  OPENFILE "Stock.txt" FOR READ

  OUTPUT "Report for Supplier:" & Supp
  OUTPUT ""           //Blank line as per example
  OUTPUT "Item        Description"
  OUTPUT ""           //Blank line as per example

  WHILE NOT EOF("Stock.txt")
     READFILE("Stock.txt", ThisLine)
     ThisCode ← Mid(ThisLine, 5, 3)
     IF ThisCode = Supp THEN
        ThisItemNum ← LEFT(ThisLine, 4)
        ThisDesc ← RIGHT(ThisLine, LENGTH(ThisLine) - 7)
        OUTPUT ThisItem & "        " & ThisDesc
        Count ← Count + 1
     ENDIF
  ENDWHILE

  CLOSEFILE "Stock.txt"

  OUTPUT "" //Blank line as per example
  OUTPUT "Number of items listed: ", Count
ENDPROCEDURE
```

MP1 Output report header (blank lines optional) – Must contain the parameter code
MP2 Conditional loop until `EOF("Stock.txt")`
MP3 Read a line from `Stock.txt` **AND** extract `SupplierCode` **in a loop**
MP4 Test if `SupplierCode = Supp` then `increment count` (must have been Initialised)
MP5 Extract **AND** output item and description **in a loop**
MP6 Output the final line with count

Examiner report missing
9618_s23_qp_23 Question 8

1(a)(i)	Use of constants	**1**

1(a)(ii)	One mark per bullet point (or equivalent to max 3): 1 Postal rates are entered once only 2 Avoids input error / changing the cost accidentally // avoids different values for postal rates at different points in the program 3 When required, the constant representing the postal rate value is changed once only // easier to maintain the program when the postal rates change 4 Makes the program easier to understand **Note: Max 3 marks**	**3**

Examiner report missing
9618_s23_qp_22 Question 1

2(a)	`MyDOB ← SETDATE(17, 11, 2007)`	**1**

2(b)	`NumMonths ← 12 - MONTH(MyDOB)` One mark per underlined part	**2**

2(c)	One mark per array definition bullet: • A (1D) array containing 7 elements • of type STRING One mark per Step: Step1: Assign value "Sunday" to first element, "Monday" to second element etc. Step2: Use the function `DAYINDEX()` to return / find the day number from `MyDoB` Step3: Use the returned value as the array index / to access the element that contains the name / string Step4: Output the element / name / string **Note: Max 2 for Array definition, Max 4 for steps**	**6**

Examiner report missing
9618_s23_qp_22 Question 2

3(a)(i)	One mark per point: 1 Check that the queue is not full 2 `EoQ` pointer will move to point to location 9 3 Data item Orange will be stored in location referenced by EoQ pointer 4 `EoQ` pointer will move to point to location 0 5 Data item Yellow will be stored in location referenced by EoQ pointer **Note: max 4 marks**	**4**

3(b)	One mark per bullet: 1 Open file in `READ` mode 2 Loop to `EOF()` // read / process all the lines in file 3 Loop will end when return value from `AddToQueue()` is `FALSE` / queue is full 4 Read a line from the file in a loop 5 Pass string to `AddToQueue()` // `AddToQueue()` is executed with line as parameter	**5**

Examiner report missing
9618_s23_qp_22 Question 3

4	Function GetNum(ThisString : STRING, ThisChar : CHAR) RETURNS INTEGER DECLARE Index, Count : INTEGER Count ← 0 FOR Index ← 1 TO LENGTH(ThisString) IF MID(ThisString, Index, 1) = ThisChar THEN Count ← Count + 1 ENDIF NEXT Index RETURN Count ENDFUNCTION Mark as follows: 1 Function heading and end, including parameters and return type 2 Declare local Integers for Index and Count 3 Loop for length of ThisString 4 Extract a character and compare with parameter **in a loop** 5 Increment Count if match **in a loop** 6 Return Count **after loop**	**6**

Examiner report missing
9618_s23_qp_22 Question 4

5(a)	One mark per point: • parameter / `Num` has been passed by reference // should have been passed by value • so when the value / `ThisNum` is modified (in procedure `DisplaySqrt()`) • the new value will be used in the loop (lines 40–43) // `Num` will be changed to modified value	**3**

Examiner report missing
9618_s23_qp_22 Question 5

6	Example of iterative solution:
	
	`PROCEDURE Square(Dim : INTEGER)`
	`  DECLARE Count : INTEGER`
	`  DECLARE ThisChar : CHAR`
	`  DECLARE StringA, StringB : STRING`
	`  CONSTANT FILLER = '*'`
	
	`  StringA ← ""`
	
	`  ThisChar ← NUM_TO_STR(Dim)`
	
	`  FOR Count ← 1 TO Dim`
	`  StringA ← StringA & ThisChar //build up first & last line`
	`  NEXT Count`
	
	`  StringB ← ThisChar`
	`  FOR Count ← 1 TO Dim - 2`
	`  StringB ← StringB & FILLER //build up intermediate line`
	`  NEXT Count`
	`  StringB ← StringB & ThisChar // add final digit`
	
	`  OUTPUT StringA`
	`  FOR Count ← 1 TO Dim - 2`
	`  OUTPUT StringB`
	`  NEXT Count`
	
	`  IF Dim <> 1 THEN`
	`  OUTPUT StringA`
	`  ENDIF`
	
	`ENDPROCEDURE`
	
	For loop-based solutions, mark as follows:
	1 Procedure heading and ending including parameter
	2 Loop using parameter, containing attempt to construct first line / last line
	3 Construct first line / last line
	4 Attempt at loop to construct intermediate line
	5 Output first / last line of square when `Dim > 2`
	6 Output all intermediate lines **in a loop**
	7 Correct output of first two squares
	
	Note: Max 6 marks
		6

Examiner report missing
9618_s23_qp_22 Question 6

8(a)	 FUNCTION CheckInfo(NewLine: STRING) RETURNS BOOLEAN DECLARE ThisNum : STRING DECLARE Index : INTEGER IF LENGTH(NewLine) < 19 THEN RETURN FALSE ENDIF FOR Index ← 1 TO 4 IF NOT IS_NUM(MID(NewLine, Index, 1)) THEN RETURN FALSE ENDIF NEXT Index ThisNum ← LEFT(Newline, 4) IF ThisNum < "0001" OR ThisNum > "5999" THEN RETURN FALSE ENDIF IF NOT OnlyAlpha(MID(Newline, 5, 3)) THEN RETURN FALSE ENDIF RETURN TRUE ENDFUNCTION Mark as follows: 1 Test length of parameter 2 Extract first 4 characters of parameter (as ItemNum) 3 Test first four characters are **all** numeric 4 Test ItemNum in range "0001" to "5999" 5 Extract characters 5 to 7 of parameter (as SupplierCode) 6 Use of OnlyAlpha() with extracted SupplierCode 7 Return BOOLEAN value correctly in all cases, must have been declared as local	**7**

8(b)		**7**

```
PROCEDURE AddItem(NewLine : STRING)
  DECLARE NewItemNum, ThisItemNum : STRING

  OPENFILE "Stock.txt" FOR READ
  OPENFILE "NewStock.txt" FOR WRITE
  NewItemNum ← LEFT(NewLine, 4)

  WHILE NOT EOF("Stock.txt")
     READFILE("Stock.txt", ThisLine)
     ThisItemNum ← LEFT(ThisLine, 4)
     IF ThisItemNum > NewItemNum THEN
        WRITEFILE("NewStock.txt", NewLine) // write New
                                              Line...
        NewItemNum ← "9999" // ...once only
     ENDIF
     WRITEFILE("NewStock.txt", ThisLine)
  ENDWHILE

  IF NewItemNum <> "9999" THEN
     WRITEFILE("NewStock.txt", NewLine) //New last line
                                          in the file
  ENDIF

  CLOSEFILE "Stock.txt"
  CLOSEFILE "NewStock.txt"
ENDPROCEDURE
```

Mark as follows:

1 Open both files, in correct modes, and subsequently close
2 Conditional loop until end of file `Stock.txt`
3 Read a line from `Stock.txt` **AND** extract `ThisItemNum` **in a loop**
4 Test `ThisItemNum > NewItemNum` then write `NewLine` to `NewStock.txt`
5 ...including mechanism to only do this **once only**
6 Write line read from `Stock` to `NewStock`
7 Deal with the case where `NewLine` is the new last line

Examiner report missing
9618_s23_qp_22 Question 8

1(a)	For example: Could set a breakpoint to stop the program at a particular point / instruction then the value of variables could be checked using a report/watch window while single stepping can be used to execute one statement/line at a time. Marks available as follows: • One mark for each underlined term • One mark for an explanation of each term **Note: max 4 marks**	**4**

1(b)	One mark for correct description of: Error 1: Brackets mismatch // 2/value should be added after brackets/function // Addition between a string and a number is not valid // STR_TO_NUM / the function needs to be passed a string / not an integer Error 2: No Error Error 3: `MONTH()` returns an integer and this is being compared with a character/string // Integer cannot be compared to a string // 6 should not be in quotes	**3**

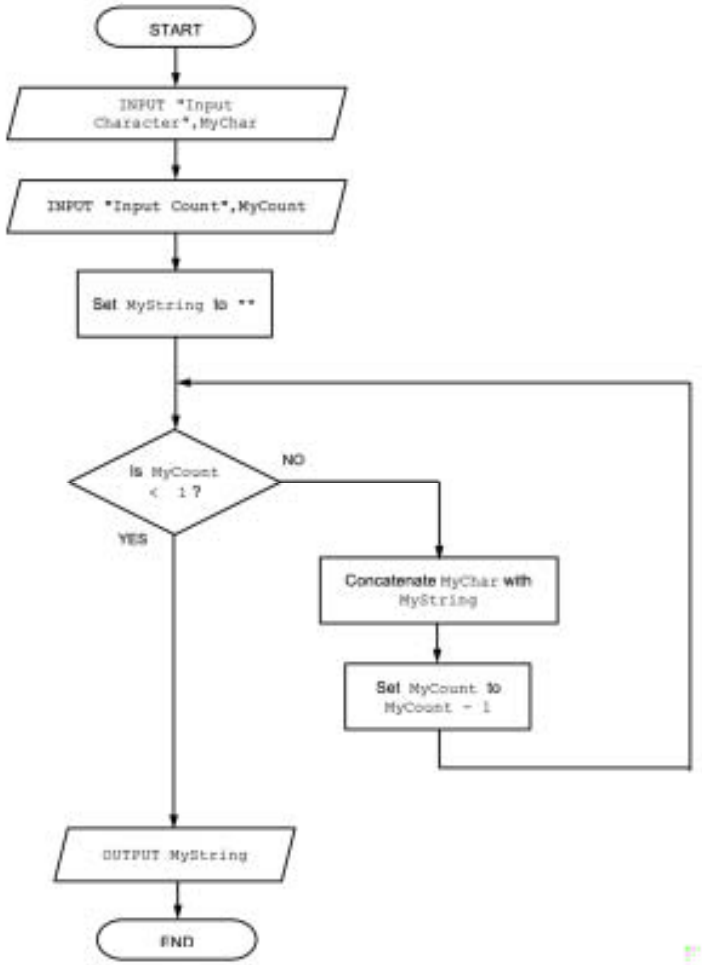
1(c)(i)		**2**

	Expression	**Evaluation**
1	`(Points > 99) OR Active`	TRUE
2	`(Points MOD 2 = 0) OR Exempt`	FALSE
3	`(Points <= 75) AND (Active OR Exempt)`	TRUE
4	`(Active OR NOT Active) AND NOT Exempt`	TRUE

One mark for any two rows correct
Two marks for all rows correct

1(c)(ii)	`NOT Exempt`	**1**

Examiner report missing
9618_s23_qp_21 Question 1

2(a)	 One mark per point: 1 Both prompts 2 Both inputs using correct identifiers as given in question, `MyChar` and `MyCount` 3 Initialise `MyString` to empty string **and** subsequent output **after loop** 4 Loop `MyCount` iterations including decrement `MyCount` 5 Use of Concatenate or equivalent pseudocode statement **inside loop** **Max 4 Marks**	**4**

2(b)	One mark for declaration: DECLARE StartDate : DATE One mark for each underlined part of assignment: StartDate ← SETDATE (15, 11, 2005)	**3**

Examiner report missing
9618_s23_qp_21 Question 2

4	``` Function Replace(OldString : STRING, Char1, Char2 : CHAR) __ RETURNS : STRING DECLARE NewString : STRING DECLARE ThisChar : CHAR DECLARE Index : INTEGER NewString ← "" FOR Index ← 1 TO LENGTH(OldString) ThisChar ← MID(OldString, Index, 1) IF ThisChar = Char1 THEN ThisChar ← Char2 ENDIF NewString ← NewString & ThisChar NEXT Index RETURN NewString ENDFUNCTION ``` Mark as follows: 1 Function heading and ending, including parameters and return type 2 Declaration of local variables used including loop counter 3 Loop for length of `OldString` 4 Extract char and test **in a loop** 5 Use of concatenate to build `NewString` replace char if necessary, **in a loop** 6 Return `NewString` after reasonable attempt	**6**

Examiner report missing
9618_s23_qp_21 Question 4

6	 PROCEDURE Mix() DECLARE Count, Total ThisNum : INTEGER DECLARE ThisUser, ThisSample : INTEGER FOR ThisSample ← 1 TO 128 Count ← 0 Total ← 0 FOR ThisUser ← 1 TO 6 IF Sample[ThisUser, ThisSample] > 10 THEN Count ← Count + 1 Total ← Total + Sample[ThisUser, ThisSample] ENDIF NEXT ThisUser Result[ThisSample] ← INT(Total / Count) NEXT ThisSample ENDPROCEDURE Mark as follows: 1 Declaration **and** initialisation before inner loop of `Count` and `Total` 2 Outer Loop for 128 iterations 3 Inner loop for six iterations 4 Test for sample > 10 **in a loop** 5 and if true sum `Total` and increment `Count` 6 Calculate average value and assign to `Result` array **after inner loop and within outer loop** 7 Use of `INT()/ DIV` to convert average to integer **Max 6 Marks**	**6**

Examiner report missing
9618_s23_qp_21 Question 6

8(a)	 ``` FUNCTION IsNewSupp(ThisString : STRING) RETURNS BOOLEAN DECLARE Index : INTEGER DECLARE ThisChar : CHAR IF LENGTH(ThisString) <> 5 THEN RETURN FALSE // invalid SupplierCode length ENDIF IF SuppExists(ThisString) THEN RETURN FALSE // SupplierCode already exists ENDIF FOR Index ← 1 TO 5 ThisChar ← TO_LOWER(MID(ThisString, Index, 1)) IF ThisChar < 'a' OR ThisChar > 'z'THEN RETURN FALSE ENDIF NEXT Index RETURN TRUE ENDFUNCTION ``` Mark as follows: 1 Check `ThisString` is exactly 5 characters in length 2 Use of `SuppExists()` with a string of 5 characters as a parameter 3 Loop for 5 iterations // loops for length of string parameter 4 Extract a character **in a loop** 5 Test if char is alphabetic **in a loop** 6 **…**catering for both upper and lower case 7 Return Boolean following a reasonable attempt	**7**

8(b)	
	``
	FUNCTION CheckNewItem(NewLine : STRING) RETURNS BOOLEAN
	  DECLARE NotFound : BOOLEAN
	  DECLARE NewItemNum, ThisItemNum, ThisLine : STRING
	
	  NotFound ← TRUE
	
	  OPENFILE "Stock.txt" FOR READ
	  NewItemNum ← LEFT(NewLine, 4)
	  ThisItemNum ← "0000" //rogue initial value
	
	  WHILE NOT EOF("Stock.txt") AND NotFound = TRUE AND__
	                                ThisItemNum < NewItemNum
	    READFILE("Stock.txt", ThisLine) //brackets optional
	    ThisItemNum ← LEFT(ThisLine, 4)
	    IF ThisItemNum = NewItemNum THEN
	      NotFound ← FALSE
	    ENDIF
	  ENDWHILE
	
	  CLOSEFILE "Stock.txt"
	
	  RETURN NotFound
	ENDFUNCTION
	``
	
	Mark as follows:
	
	1 Open `Stock.txt` in READ mode and subsequently close
	2 Extract `NewItemNum` from parameter
	3 Conditional loop until `EOF("Stock.txt")`
	4 ... **OR** `NewItemNum` found
	5 ... **OR** when `ThisItemNum < NewItemNum`
	6 Read a line from `Stock.txt` **AND** extract `ThisItemNum` **in a loop**
	7 If `ThisItemNum = NewItemNum` then terminate loop / set flag **in a loop**
	8 Return Boolean after reasonable attempt
	
	Max 7 marks
	
	Max 6 if function wrapper (heading and ending) missing or incorrect
		7

8(e)	One mark for each part: • The algorithm / search / iteration can stop /only iterates • if the current value read from the file // current line in file • is greater than the value being searched for	**3**

Examiner report missing
9618_s23_qp_21 Question 8

4(a)	The data type (of the item to be stored)	**1**

4(b)(i)	Operation: Add an item / Enqueue Check: There are unused elements in the array // The queue is not full Operation: Remove an item / Dequeue Check: There are items in the array // The queue is not empty One mark for reason and one mark for reason it could not be completed.	**4**

4(b)(ii)	One mark per point (**Max 5**): 1 Declare a (1D) array of size >= 10 2 …of data type `CHAR` 3 Declare integer variable for `FrontOfQueuePointer` 4 Declare integer variable for `EndOfQueuePointer` 5 Initialise `FrontOfQueuePointer` and `EndOfQueuePointer` to represent an empty queue 6 Declare integer variable or `NumberInQueue` 7 Declare integer variable for `SizeOfQueue` to count / limit the max number of items allowed // Reference to mechanism for defining 'wrap' of circular queue 8 Initialise `SizeOfQueue` // Initialise `NumberInQueue`	**5**

Examiner report missing
9618_w22_qp_23 Question 4

5(a)(i)	One mark per point:
	
	1 Procedure heading and ending including four parameters...
	2 ...and use of `BYREF` for the three extracted values
	3 Extract and assign `SID`
	4 Extract and assign `SDesc`
	5 Calculation of length of `SCost` (remainder of string)
	6 Extract and assign `SCost` following an attempt at MP5
	
	`PROCEDURE UnPack(BYVAL TLine : STRING, BYREF SID, SDesc, SCost : STRING)`
	`  SID ← LEFT(TLine, 5)`
	`  SDesc ← MID(TLine, 6, 32)`
	`  SCost ← RIGHT(TLine, LENGTH(TLine) - 37)`
	`ENDPROCEDURE`
		6

5(a)(ii)	One mark each (**Max 2**): 1 Provides a mechanism to allow calling program to pass data 2 Defines the four parameters of `Unpack()` 3 … giving their data type and order	**2**
5(b)(i)	`LineData.Cost ← 12.99`	**1**
5(b)(ii)	One mark per point (**Max 2**): • The new function will return an item of type `StockItem` • Need to declare/use a (local) variable of type `StockItem` • `Cost`field needs to be converted from a string to a real	**2**

Examiner report missing
9618_w22_qp_23 Question 5

6(a)	One mark per row Examples:	**5**

Test number	Component weight	Min	Max	Expected return value
1	300	290	315	'A'
2	> 317	290	315	'R'
3	317	290	315	'C'
4	288	290	315	'C'
5	< 288	290	315	'R'

6(b)	One mark per point:
	
	1 Function heading and ending **including** parameters
	2 Declaration of local variable for Result / alt mechanism
	3 Check for Reject
	4 Check for Accept
	5 Check for Recheck (or just default to third option)
	6 Return Result following a reasonable attempt
	
	`Function Status(Actual, Min, Max : INTEGER) RETURNS CHAR`
	`  DECLARE Result : CHAR`
	`  CONSTANT Accept = 'A'`
	`  CONSTANT Reject = 'R'`
	`  CONSTANT ReCheck = 'C'`
	
	`  Result ← ReCheck`
	
	`  //Check if reject`
	`  IF Actual > Max + 2 OR Actual < Min - 2 THEN`
	`  Result ← Reject`
	`  ENDIF`
	
	`  //Check if acceptable`
	`  IF Actual < Max - 2 AND Actual > Min + 2 THEN`
	`  Result ← Accept`
	`  ENDIF`
	
	`  RETURN Result`
	`ENDFUNCTION`
		6

Examiner report missing
9618_w22_qp_23 Question 6

7(a)	One mark per point (**Max 8**):
	
	1 Declaration and initialisation of local integer for `Count`
	2 Appropriate prompt and two inputs
	3 (Conditional) loop while error number input is in range // error code 999 reached
	4 …and not end of array
	5 Check if this `ErrCode` needs to be output **in a loop**
	6 if so check for blank error text **in a loop**
	7 Output in both cases
	8 ….and increment count **in a loop**
	9 OUTPUT of header and summary including count
	
	`PROCEDURE OutputRange()`
	`  DECLARE First, Last, Count, Index, ThisErr : INTEGER`
	`  DECLARE ThisMess : STRING`
	`  DECLARE PastLast: BOOLEAN`
	
	`  Count ← 0`
	`  Index ← 1`
	`  PastLast ← FALSE`
	
	`  OUTPUT "Please input first error number: "`
	`  INPUT First`
	`  OUTPUT "Please input last error number: "`
	`  INPUT Last`
	
	`  OUTPUT "List of error numbers from ", First, " to ", Last`
	
	`  WHILE Index < 501 AND NOT PastLast`
	`  ThisErr ← ErrCode[Index]`
	`  IF ThisErr > Last THEN`
	`  PastLast ← TRUE`
	`  ELSE`
	`  IF ThisErr >= First THEN`
	`  ThisMess ← ErrText[Index]`
	`  IF ThisMess = "" THEN`
	`  ThisMess ← "Error Text Missing"`
	`  ENDIF`
	`  OUTPUT ThisErr, " : ", ThisMess`
	`  Count ← Count + 1`
	`  ENDIF`
	`  ENDIF`
	`  Index ← Index + 1`
	`  ENDWHILE`
	
	`  OUTPUT Count, " error numbers output"`
	
	`ENDPROCEDURE`
		8

7(b)(i)	One mark per point:
	
	1 (Conditional) loop terminating when item added **OR** end of array reached
	2 Test for unused element **in a loop**
	3 Assignment of values to arrays // save index of first blank location and assign after loop
	4 Set loop termination if empty element found in a loop
	5 Call `SortArrays()` **once**
	6 Calculation of remaining unused elements **and** return Integer value (for both cases)
	
	`FUNCTION AddError(ErrNum : INTEGER, ErrMess : STRING)`
	`RETURNS INTEGER`
	`  DECLARE Index, Remaining : INTEGER`
	`  CONSTANT Unused = 999`
	
	`  Index ← 1`
	`  Remaining ← -1`
	
	`  REPEAT`
	`    IF ErrCode[Index] = Unused THEN`
	`      ErrCode[Index] ← ErrNum`
	`      ErrText[Index] ← ErrMess`
	`      CALL SortArrays()`
	`      Remaining ← 500 - Index`
	`    ENDIF`
	`    Index ← Index + 1`
	`  UNTIL Remaining <> -1 OR Index > 500`
	
	`  RETURN Remaining`
	
	`ENDFUNCTION`
		6

Examiner report missing
9618_w22_qp_23 Question 7

1(a)(i)	One mark for each point (**Max 2**): • When a task which is repeated / reused / performed in several places • When a part of an algorithm performs a specific task • Reduces complexity of program / program is simplified // subroutine already available • Testing / debugging / maintenance is easier	**2**

1(a)(ii)	One mark for each part: **Term**: Parameter(s) **Use**: to pass values / arguments to the procedure	**2**

1(c)	One mark per row: **Expression** / **Evaluation** `MID(CharList, MONTH(FlagDay), 1)` / 'D' `INT(Count / LENGTH(CharList))` / 4 `(Count >= 99) AND (DAY(FlagDay) > 23)` / FALSE	**3**

Examiner report missing
9618_w22_qp_22 Question 1

4(a)	One mark per point:
	
	1 Input `UserID` **and** use of `GetAverage()` **and** assignment
	2 Initialisation of `Total` to zero **and** `Index` to 4
	3 Conditional loop with `Index` from 4 to 6
	4 Assignment of `Last` from element `SameMonth[Index]` **in a loop**
	5 Structure: `IF...THEN...ELSE...ENDIF` **in a loop**
	6 Correct assignments **and** final call to `Update()` **after the loop**
	
	`INPUT UserID`
	`Average ← GetAverage(UserID)`
	`Total ← 0`
	`Index ← 4`
	
	`WHILE Index < 7 // REPEAT`
	`  Last ← SameMonth[Index]`
	`  IF Average > Last THEN`
	`  Total ← Total + Average`
	`  ELSE`
	`  Total ← Total + Last`
	`  ENDIF`
	`  Index ← Index + 1`
	`ENDWHILE // UNTIL Index = 7`
	
	`CALL Update(UserID, Total)`
	
	<u>Alternative solution using FOR loop</u>:
	
	One mark per point FOR loop solution:
	1 Input `UserID` **and** use of `GetAverage()` **and** assignment
	2 Initialisation of `Total` to zero
	3 loop `Index` from 4 to 6
	4 Assignment of `Last` from array `SameMonth` **in a loop**
	5 Comparison `IF...THEN...ELSE...ENDIF` **in a loop**
	6 Appropriate assignments **in a loop AND** final call to `Update()` **after the loop**
	
	`INPUT UserID`
	`Average ← GetAverage(UserID)`
	`Total ← 0`
	`FOR Index ← 4 TO 6`
	`  Last ← SameMonth[Index]`
	`  IF Average > Last THEN`
	`  Total ← Total + Average`
	`  ELSE`
	`  Total ← Total + Last`
	`  ENDIF`
	`NEXT Index`
	
	`CALL Update(UserID, Total)`
		6

4(b)	Pre-condition (loop) / count-controlled (loop)	**1**

Examiner report missing
9618_w22_qp_22 Question 4

5	One mark per `IF...THEN...ENDIF` clause: 1 ``` IF A AND B AND C THEN CALL Sub1() ENDIF ``` 2 ``` IF (A AND B) AND NOT C THEN CALL Sub2() ENDIF ``` 3 ``` IF (NOT A) AND (NOT C) THEN CALL Sub3() ENDIF ``` 4 ``` IF (NOT A) AND C THEN CALL Sub4() ENDIF ```	**4**

Examiner report missing
9618_w22_qp_22 Question 5

6(a)	Example by repeated multiplication:
	
	Mark as follows (multiplication solution), (**Max 7**):
	
	1 Function heading and ending **including** parameter and return type
	2 Declaration and initialisation of local `Num`
	3 Any conditional loop
	4 Conditional loop until `ThisValue` found or `Try` out of range
	5 Multiply `Try` by `Num` **in a loop**
	6 Compare `Try` with `ThisValue` **and** set termination if the same **in a loop**
	7 Increment `Num` and repeat **in a loop**
	8 Attempt to Return `Num` if `ThisValue` is a factorial or -1 otherwise
	
	`FUNCTION FindBaseNumber(ThisValue : INTEGER) RETURNS INTEGER`
	`  DECLARE Num, Try : INTEGER`
	`  DECLARE Found : BOOLEAN`
	
	`  Num ← 0`
	`  Found ← FALSE`
	`  Try ← 1`
	
	`   WHILE Try <= ThisValue AND Found = FALSE`
	`      Num ← Num + 1`
	`      Try ← Try * Num`
	`      IF Try = ThisValue THEN //BaseNumber found`
	`         Found ← TRUE`
	`      ENDIF`
	`   ENDWHILE`
	
	`   IF Found = TRUE THEN`
	`      RETURN Num`
	`   ELSE`
	`      RETURN -1`
	`   ENDIF`
	
	`ENDFUNCTION`
		7

Examiner report missing
9618_w22_qp_22 Question 6

7(a)	One mark per point (**Max 6**):
	
	1 Procedure heading and ending **including** parameters
	2 Conditional loop containing incrementing `Index`...
	3 ...terminating when `ErrNum` found
	4 ...terminating when `ErrCode[Index] > ErrNum` (i.e. `ErrNum` not found)
	5 ... **OR** after element 500 tested
	6 Test if found and OUTPUT 'Found' message
	7 ...otherwise OUTPUT 'Not Found' message
	
	`PROCEDURE OutputError(LineNum, ErrNum : INTEGER)`
	`  DECLARE Index : INTEGER`
	
	`  Index ← 0`
	
	`  // Search until ErrNum found OR not present OR end of array`
	
	`  REPEAT`
	`  Index ← Index + 1`
	`  UNTIL ErrCode[Index] >= ErrNum OR Index = 500`
	
	`  IF ErrCode[Index] = ErrNum THEN`
	`  OUTPUT ErrText[Index], " on line ", LineNum //Found`
	`  ELSE`
	`  OUTPUT "Unknown error on line ", LineNum //Not found`
	`  ENDIF`
	
	`ENDPROCEURE`
		6

7(b)	One mark per point (**Max 8**): 1 Procedure heading and ending as shown 2 Conditional loop correctly terminated 3 An inner loop 4 Correct range for inner loop 5 Comparison (element J with J+1) **in a loop** 6 Swap elements in **both** arrays **in a loop** 7 'No-Swap' mechanism: • Conditional **outer** loop including flag reset • Flag set in **inner** loop to indicate swap 8 Efficiency (this scenario): terminate inner loop when `ErrCode = 999` 9 Reducing `Boundary` **in the <u>outer</u> loop**	**8**

```
PROCEDURE SortArrays()
  DECLARE TempInt, J, Boundary : INTEGER
  DECLARE TempStr : STRING
  DECLARE NoSwaps : BOOLEAN

  Boundary ← 499

  REPEAT
     NoSwaps ← TRUE
     FOR J ← 1 TO Boundary
        IF ErrCode[J]> ErrCode[J+1] THEN
           //first swap ErrCode elements
           TempInt ← ErrCode[J]
           ErrCode[J] ← ErrCode[J+1]
           ErrCode[J+1] ← TempInt
           //now swap corresponding ErrText elements
           TempStr ← ErrText[J]
           ErrText[J] ← ErrText[J+1]
           ErrText[J+1] ← TempStr
           NoSwaps ← FALSE
        ENDIF
     NEXT J
     Boundary ← Boundary - 1
  UNTIL NoSwaps = TRUE

ENDPROCEDURE
```

Examiner report missing
9618_w22_qp_22 Question 7

1(a)	One mark per point: 1 They are tried and tested so free from errors 2 They perform a function that you may not be able to program yourself (for example **encryption**) 3 They are readily available / speed up development time	**3**

1(c)	One mark per row: **Statement** / **Error** `IF EMPTY ← "" THEN` — Should be "=" not ← `Status ← IS_NUM(-23.4)` — Parameter should be a string (or char) // should not be a real `X ← STR_TO_NUM("37") + 5` — NO ERROR `Y ← STR_TO_NUM("37" + "5")` — Wrong operator – should be & or Parameter is not a string	**4**

Examiner report missing
9618_w22_qp_21 Question 1

5	Mark as follows (**Max 5**):
	
	1 Procedure heading and ending **and** declaration of both indexes
	2 Loop to process all elements from `Array1`
	3 Sum (any) three consecutive values from `Array1` **and** divide by 3 **in a loop**
	4 Convert result to Integer
	5 Assign value to correct element of `Array2` **in a loop**
	6 Increment `Array2` index **in a loop**
	
	`PROCEDURE Summarise()`
	`  DECLARE Value : REAL`
	`  DECLARE IxA, IxB : INTEGER // Index variables`
	
	`  IxB ← 1`
	
	`  FOR IxA ← 1 TO 598 STEP 3`
	`  Value ← Array1[IxA] + Array1[IxA + 1] + Array1[IxA + 2]`
	`  Value ← Value / 3`
	`  Array2[IxB] ← INT(Value)`
	`  IxB ← IxB + 1`
	`  NEXT IxA`
	
	`ENDPROCEDURE`
		5

Examiner report missing
9618_w22_qp_21 Question 5

6(a)	One mark for **any** part correct (accept equivalent wording) (**Max 1**): • Condition evaluates to TRUE if bracket contents evaluate to FALSE: • Bracket contents evaluate to FALSE if: • Dots: zero / less than one or • Ats: not equal to one or • Others: less than nine	**1**

6(b)(i)	One mark for each area as outlined:	**5**

Index	NextChar	Dots	Ats	Others	Valid
		0	0	0	TRUE
1	'L'			1	
2	'i'			2	
3	'z'			3	
4	'.'	1			
5	'1'			4	
6	'2'			5	
7	'3'			6	
8	'@'		1		
9	'b'			7	
10	'i'			8	
11	'g'			9	
12	'@'		2		FALSE

6(b)(ii)	FALSE	**1**

Examiner report missing
9618_w22_qp_21 Question 6

7(a)(i)	One mark per point (**Max 7**) as follows:
	
	1 Declaration of local variables for `Par1` `Par2` and `Par3`
	2 Loop to end of (parameter) string // until operator is found
	3 Extract a character **in a loop**...
	4 Attempt at extraction of three parts of expression using substring functions
	5 Completely correct extraction of all three parts of expression
	6. Convert string to Integer using `STR_TO_NUM(<something sensible>)`
	7 Attempt to interpret at least two operators (`Par2`): + - * /
	8 Corresponding correct calculation (all operators) **and** final Output of result
	
	`PROCEDURE Calculate(Expression : STRING)`
	`  DECLARE Val1, Val2, Index : INTEGER`
	`  DECLARE Result : REAL`
	`  DECLARE Par1, Par2, Par3 : STRING`
	
	`  CONSTANT PLUS = '+'`
	`  CONSTANT MINUS = '-'`
	`  CONSTANT MULTIPLY = '*'`
	`  CONSTANT DIVIDE = '/'`
	
	`  FOR Index ← 1 TO LENGTH(Expression) //search for operator`
	`  ThisChar ← MID(Expression, Index, 1)`
	`  IF IS_NUM(ThisChar) = FALSE THEN`
	`  Par1 ← LEFT(Expression, Index - 1)`
	`  Par2 ← ThisChar`
	`  Par3 ← RIGHT(Expression, LENGTH(Expression) - Index)`
	`  ENDIF`
	`  NEXT Index`
	
	`  Val1 ← STR_TO_NUM(Par1)`
	`  Val2 ← STR_TO_NUM(Par3)`
	
	`  CASE OF Par2`
	`  PLUS : Result ← Val1 + Val2`
	`  MINUS : Result ← Val1 - Val2`
	`  MULTIPLY : Result ← Val1 * Val2`
	`  DIVIDE : Result ← Val1 / Val2`
	`  ENDCASE`
	
	`  OUTPUT Result`
	
	`ENDPROCEDURE`
		7

7(a)(ii)	`FUNCTION Calculate(Expression : STRING) RETURNS REAL`	**1**

7(b)	**Example string**: "23/0" (Any divide by zero example) **Reason**: The result is infinity / cannot be represented / is undefined // will cause the program to crash	**2**

Examiner report missing
9618_w22_qp_21 Question 7

8(a)	One mark for each point (**Max 7**) as follows: 1 Function heading and ending **including** parameter and return type 2 Declaration **and** initialisation of local Integer for `Count` 3 OPEN in READ mode and CLOSE 4 Conditional loop until `EOF()` 5 Read a line **in a loop** 6 If non-blank, increment count **in a loop** 7 Terminate loop when 10 non-blank lines have been read 8 Return Boolean in both cases	**7**

```
FUNCTION CheckFile(Thisfile : STRING) RETURNS BOOLEAN
   DECLARE Valid : BOOLEAN
   DECLARE ThisLine : STRING
   DECLARE Count : INTEGER

   Count ← 0
   Valid ← FALSE
   OPEN ThisFile FOR READ

   WHILE NOT EOF(ThisFile) AND Valid = FALSE
      READFILE ThisFile, ThisLine
      IF ThisLine <> "" THEN
         Count ← Count + 1
         IF Count > 9 THEN
            Valid ← TRUE
         ENDIF
      ENDIF
   ENDWHILE

   CLOSEFILE ThisFile
   RETURN Valid

ENDFUNCTION
```

8(b)	`CALL CountErrors("Jim01Prog.txt", 20)` One mark for each: 1 Module name, at least one parameter in brackets and one parameter correct 2 Completely correct statement	**2**

8(c)	Mark as follows:
	1 Procedure heading and ending **including** parameters
	2 Declaration and initialisation of local Integer value for `ErrCount`
	3 Use of `CheckFile()`, output message **and** terminate if it returns `FALSE`
	4 Conditional loop until `EOF()`
	5 ...or `ErrCount > MaxErrors`
	6 Read line and use as parameter to `CheckLine()` **in a loop**
	7 Test return value and increment `ErrCount` if non-zero **in a loop**
	8 Output either message once only as appropriate
	
	`PROCEDURE CountErrors(ThisFile : STRING, MaxErrors : INTEGER)`
	`  DECLARE ErrCount, ThisError : INTEGER`
	`  DECLARE ThisLine : STRING`
	
	`  ErrCount ← 0`
	
	`  IF CheckFile(ThisFile) = FALSE THEN`
	`  OUTPUT "That program file is not valid"`
	`  ELSE`
	`  OPEN ThisFile FOR READ`
	
	`  REPEAT`
	`  READFILE, ThisFile, ThisLine`
	`  ThisError ← CheckLine(ThisLine)`
	`  IF ThisError <> 0 THEN`
	`  ErrCount ← ErrCount + 1`
	`  ENDIF`
	`  UNTIL ErrCount > MaxErrors OR EOF(ThisFile)`
	
	`  IF EOF(ThisFile) = FALSE THEN`
	`  OUTPUT "Check terminated - too many errors"`
	`  ELSE`
	`  OUTPUT "There were ", ErrCount, " errors."`
	`  ENDIF`
	
	`  CLOSEFILE ThisFile`
	`  ENDIF`
	
	`ENDPROCEDURE`
		8

Examiner report missing
9618_w22_qp_21 Question 8

2(a)	One mark for reference to: 1 The use a variable as an index to the array 2 A loop to iterate through the array 3 An Inner loop (with a reducing range) 4 Test if current element is greater than next element 5 if so then swap elements 6 Description of swap 7 Attempt at efficient algorithm **Max 6** marks	**6**

2(b)	`Count ← 1`
	`Flag ← FALSE`
	`WHILE Flag = FALSE AND Count <= 5`
	`  CALL ReBoot()`
	`  Count ← Count + 1`
	`  Flag ← Check()`
	`ENDWHILE`
	
	`IF Flag = FALSE THEN`
	`  CALL Alert(27)`
	`ENDIF`
	
	One mark per point:
	
	1 Initialisation of `Count` **AND** `Flag`
	2 `WHILE ... ENDWHILE // REPEAT ... UNTIL` loop
	3 ... including both conditions
	4 Call `ReBoot()` **AND** increment `Count` **inside the loop**
	5 Assign return value from `Check()` to `Flag` **inside the loop**
	6 Final test of `Flag` **AND** call to `Alert(27)` **not in a loop**
		6

Examiner report missing
9618_w21_qp_23 Question 2

5(a)	One mark for the character and one for the corresponding reason. • Character: Any except alphabetic, numeric, ',' ':' or space • Reason: character doesn't occur in data to be recorded	**2**

5(c)		**7**

```
FUNCTION LogEvents(StudentID : STRING) RETURNS INTEGER

  DECLARE FileData : STRING
  DECLARE Index, Count : INTEGER

  CONSTANT LogFile = "LogFile"

  Count ← 0
  OPENFILE LogFile FOR APPEND
  FOR Index ← 1 TO 2000
     FileData ← LogArray[Index]

     IF LEFT(FileData, 6) = StudentID THEN
        WRITEFILE (LogFile, FileData) //brackets optional
        Count ← Count + 1
        LogArray[Index] ← ""        // clear the element
     ENDIF
  NEXT Index

  CLOSEFILE LogFile

  RETURN Count

ENDFUNCTION
```

1 mark for each of the following:

1 Function heading and ending including parameter **and** return type
2 `OPEN` file `LogFile` for `APPEND` and subsequent `CLOSE`
3 Loop for 2000 iterations
4 Extract first 6 characters from array element **in a loop**
5 Compare first 6 characters with parameter **in a loop**
6 If equal:
- write whole array element string to file **and**
- increment `Count` **and**
- clear array element **in a loop**

7 Return Count (must have been declared **and** initialised)

Examiner report missing
9618_w21_qp_23 Question 5

6(a)	```
	PROCEDURE SetRow(Row, SkipNum, SetNum : INTEGER)
	  DECLARE Col : INTEGER
	
	  // array is 1280 x 800
	
	  FOR Col ← SkipNum + 1 TO SkipNum + SetNum
	    Screen[Row, Col] ← 1
	  NEXT Index
	
	ENDPROCEDURE
	``
	ALTERNATIVE 1:
	``
	  FOR Col ← 1 TO SetNum
	    Screen[Row, SkipNum + Col] ← 1
	  NEXT Col
	``
	ALTERNATIVE 2:
	``
	  WHILE SetNum > 0
	    Screen[Row, SkipNum + SetNum] ← 1
	    SetNum ← SetNum - 1
	  ENDWHILE
	``
	Mark as follows:
	
	1 Procedure heading and ending including parameters
	2 Declaration of local Integer for `Col`
	3 Count-controlled loop with meaningful start number
	4 correct stop number
	5 Reference Screen Array element and set to 1 **in a loop**
		5

6(b)	```
	FUNCTION SearchInRow(ThisRow, StartCol : INTEGER) RETURNS
	        INTEGER
	   DECLARE ThisCol, Step : INTEGER
	   DECLARE Found: BOOLEAN
	
	   // array is 1280 x 800
	
	   Found ← FALSE
	   ThisCol ← StartCol
	
	   // first decide which way to search
	   IF StartCol = 1 THEN
	      Step ← 1
	      EndCol ← 1281
	   ELSE
	       Step ← -1
	       EndCol ← 0
	   ENDIF
	
	   WHILE ThisCol <> EndCol AND Found = FALSE
	      IF Screen[ThisRow, ThisCol] <> 1 THEN
	         ThisCol ← ThisCol + Step
	      ELSE
	         Found ← TRUE
	      ENDIF
	   ENDWHILE
	
	   IF Found = FALSE THEN
	      ThisCol ← -1
	   ENDIF
	
	   RETURN ThisCol
	
	ENDFUNCTION
	``
	
	Mark as follows:
	
	1 Interpreting `StartCol` parameter to determine direction of search
	2 An attempt at searching both up and down
	3 Conditional Loop / Count-controlled loop with use of `ThisCol` index
	4 Using correct values for `StartCol`, `EndCol` and `Step`
	5 Reference a `Screen` element and compare with 1 **in a loop**
	6 If equal save column or immediately Return column **in a loop**
	7 Return column number or −1
	Loop(s) terminate when element with value = 1 found
	
	Max 7 marks if function heading, including return type, and ending is incorrect or incomplete
		8

6(c)	`FUNCTION GetCentreCol(ThisRow : INTEGER) RETURNS INTEGER`
	`  DECLARE StartCol, EndCol, CentreCol : INTEGER`
	
	`  StartCol ← SearchInRow(ThisRow, 1)`
	`  IF StartCol = -1 THEN`
	`  CentreCol ← StartCol`
	`  ELSE`
	`  EndCol ← SearchInRow(ThisRow, 1280)`
	`  CentreCol ← INT((StartCol + EndCol)/2)`
	`  ENDIF`
	`  RETURN CentreCol`
	`ENDFUNCTION`
	
	Mark as follows:
	
	1 Declaration of local `INTEGER` for return value
	2 Use `SearchInRow()` with correct parameters and check for `-1`
	3 Use `SearchInRow(ThisRow, 1)` and `SearchInRow(ThisRow, 1280)`
	4 Calculate centre column
	5 Use of `INT()` function // use of `DIV`
	6 Return –1 or centre value
	
	Max 5 marks if function heading, including return type, and ending is incorrect or incomplete
		6

Examiner report missing
9618_w21_qp_23 Question 6

1(a)	The process involves: 1 Breaking down a problem / task into sub problems / steps / smaller parts 2 In order to explain / understand // easier to solve the problem 3 Leading to the concept of program modules // assigning problem parts to teams **Max 2**	**2**

1(c)	<table><tr><th>Expression</th><th>Evaluates to</th></tr><tr><td>ASC('C')</td><td>67</td></tr><tr><td>2 * STR_TO_NUM ("27")</td><td>54</td></tr><tr><td>INT(27 / 2)</td><td>13</td></tr><tr><td>"Sub" & MID("Abstraction" , 4 , 5)</td><td>"Subtract"</td></tr></table> One mark per row Function names must be exactly as shown	**4**

1(d)	<table><tr><th>Expression</th><th>Evaluates to</th></tr><tr><td>PressureOK AND HiFlow</td><td>FALSE</td></tr><tr><td>PumpOn OR PressureOK</td><td>TRUE</td></tr><tr><td>NOT PumpOn OR (PressureOK AND NOT HiFlow)</td><td>TRUE</td></tr><tr><td>NOT (PumpOn OR PressureOK) AND NOT HiFlow</td><td>FALSE</td></tr></table> 1 mark for any two rows correct 2 marks for all rows correct.	**2**

Examiner report missing
9618_w21_qp_22 Question 1

3(a)(i)	Pseudocode:
	
	`TYPE Student`
	`  DECLARE StudentID : STRING`
	`  DECLARE Email : STRING`
	`  DECLARE Club_1 : INTEGER`
	`  DECLARE Club_2 : INTEGER`
	`  DECLARE Club_3 : INTEGER`
	`ENDTYPE`
	
	Mark as follows:
	
	• One mark for `TYPE` and `ENDTYPE`
	• One mark for `StudentID` **and** `Email` fields as `STRING`
	• One mark for all Club fields as `INTEGER`
		3

3(a)(ii)	`DECLARE Membership : ARRAY [1:3000] OF Student` One mark per underlined phrase	**2**

3(b)	
	`PROCEDURE GetIDs()`
	`  DECLARE Index : INTEGER`
	`  DECLARE ThisClub, Count : INTEGER`
	
	`  OUTPUT "Please Input Club Number: "`
	`  INPUT ThisClub`
	
	`  Count ← 0`
	
	`  FOR Index ← 1 TO 3000`
	`  IF Membership[Index].Club_1 = ThisClub OR __`
	`  Membership[Index].Club_2 = ThisClub OR __`
	`  Membership[Index].Club_3 = ThisClub THEN`
	`  Count ← Count + 1`
	`  OUTPUT Membership[Index].StudentID`
	`  ENDIF`
	
	`  NEXT Index`
	
	`  OUTPUT "There are ", Count, " Students in the club"`
	`ENDPROCEDURE`
	
	Mark as follows:
	
	1 Declare and initialise `Count`
	2 Prompt and Input club number **before the loop**
	3 Loop through 3000 elements
	4 Compare **one** club field with number input
	5 Compare **all** `Club` fields with number input
	6 If number found, `OUTPUT` of `StudentID` field **and** increment `Count`
	7 Final `OUTPUT` of `Count` **outside the loop**
	
	Note:
	
	Max 6 if procedure heading and ending missing or incorrect (but allow array as parameter)
		7

Examiner report missing
9618_w21_qp_22 Question 3

4(a)	**Answer** A line number containing a variable being incremented: 19 / 21 / 23 The type of loop structure: pre-condition The number of functions used: 3 The number of parameters passed to function `STR_TO_NUM()`: 1 The name of a procedure other than `Check()`: Result	**5**
4(c)(i)	One mark per point: Structure: A count-controlled loop Justification: The number of iterations is known // repeats for the length of `InString`	**2**
4(c)(ii)	15, 23 One mark for **both** line numbers	**1**

Examiner report missing
9618_w21_qp_22 Question 4

5(a)	 ``` PROCEDURE MakeNewFile(OldFile, NewFile, Status : STRING) DECLARE Line1, Line2, Line3 : STRING DECLARE NumCopied, NumRecs : INTEGER NumRecs ← 0 NumCopied ← 0 OPENFILE OldFile FOR READ OPENFILE NewFile FOR WRITE WHILE NOT EOF(OldFile) READFILE OldFile, Line1 READFILE OldFile, Line2 READFILE OldFile, Line3 NumRecs ← NumRecs + 1 IF Line3 <> Status THEN WRITEFILE NewFile, Line1 WRITEFILE NewFile, Line2 WRITEFILE NewFile, Line3 NumCopied ← NumCopied + 1 ENDIF ENDWHILE OUTPUT "File " , OldFile , " contained " , NumRecs ,__ " employee details" OUTPUT Numcopied , " employee sets of details were __ written to file", NewFile CLOSEFILE OldFile CLOSEFILE NewFile ENDPROCEDURE ``` Mark as follows: 1 Procedure heading and ending, including parameters 2 `OPEN OldFile` for `READ` and `NewFile` for `WRITE` and subsequently `CLOSE` both files 3 Conditional loop until `EOF(OldFile)` 4 Read three lines from `OldFile` **in a loop** 5 Compare 3rd line read with `Status` parameter **and** if not equal write 3 lines to `NewFile` **in a loop** 6 Count number of sets read and those written **in a loop** 7 Final output including both counts and file names with suitable text **after a loop** Note: MP6: Both counts must have been declared and initialised	**7**

5(b)(i)	Store all three items on one line	**1**

5(b)(ii)	One mark per point: Advantage: Fewer file operations required Disadvantage: Algorithm to combine / extract individual data items is more complex	**2**

Examiner report missing
9618_w21_qp_22 Question 5

6(a)	```
	FUNCTION FirstRowSet() RETURNS INTEGER
	  DECLARE Row, Col : INTEGER
	  DECLARE Found : BOOLEAN
	
	  // array is 1280 × 800
	  Row ← 1
	
	  Found ← FALSE
	  WHILE Row <= 800 AND Found = FALSE // top to bottom
	  Col ← 1
	  WHILE Col <= 1280 AND Found = FALSE // left to right
	  IF Screen[Row,Col] = 1 THEN
	  Found ← TRUE // end function as soon as first
	  // found
	  ENDIF
	  Col ← Col + 1
	  ENDWHILE
	  Row ← Row + 1
	  ENDWHILE
	
	  IF Found = FALSE THEN // nothing found
	  Row ← 0
	  ENDIF
	  RETURN Row - 1
	ENDFUNCTION
	``
	
	Mark as follows:
	
	1 Function heading and ending **and** return type
	2 (Conditional) outer loop 1 to 800 (row)
	3 (Conditional) inner loop 1 to 1280 // 1280 to 1 (column)
	4 Reference `Screen` element and test for = 1 // <> 0
	5 and if true save row number **and** exit loops
	6 Increment index variables in both inner **and** outer loop
	7 Return Row number or −1, following a reasonable attempt
		7

6(b)	One mark for: • (A flag is used to) exit the loops // iteration is terminated • as soon as a Screen element with value 1 is found	**2**

6(c)(i)	One mark for: • Parameter(s) need to be passed to the module to identify the type of search • Search algorithm is controlled by (global) variables / parameters Alternative: • The search algorithms from the original modules are included in the new module • The new module needs to return / store the four values (the results of the four searches)	**2**

6(c)(ii)	One mark for advantage and one for disadvantage: Advantage: (max 1) • Only have to change one module if specification changes • Less repetitive code / fewer lines of code • Aids re-usability Disadvantage: (max 1) • Single module more complex / more error prone / more difficult to debug ... • Single module cannot be split among programmers / teams **Max 2**	**2**

6(d)	 ``` PROCEDURE GetCentre () DECLARE StartRow, EndRow, StartCol, EndCol : INTEGER StartRow ← FirstRowSet() IF StartRow = -1 THEN CentreRow ← -1 // no 'touch' detected ELSE EndRow ← LastRowSet() StartCol ← FirstColSet() EndCol ← LastColSet() CentreRow ← INT((StartRow + EndRow)/2) CentreCol ← INT((StartCol + EndCol)/2) ENDIF ENDPROCEDURE ``` Mark as follows: 1 Call <any `Set` function> and check for -1 // check for no element set 2 ...and if so set `CentreRow` to –1 3 Call all 4 `Set` functions to get 'extremity' values 4 Calculate centre row and centre column 5 Use of `INT()` function or `DIV` operator on values from MP4 6 Assign calculated values to `CentreRow` and `CentreCol` Note: Max 5 if procedure heading and ending missing or incorrect (ignore array if passed as a parameter) **or** any local variables are undefined or of incorrect type	**6**

Examiner report missing
9618_w21_qp_22 Question 6

2(a)	One mark for reference to: 1 The use a variable as an index to the array 2 A loop to iterate through the array 3 An Inner loop (with a reducing range) 4 Test if current element is greater than next element 5 if so then swap elements 6 Description of swap 7 Attempt at efficient algorithm **Max 6** marks	**6**

2(b)	`Count ← 1`
	`Flag ← FALSE`
	`WHILE Flag = FALSE AND Count <= 5`
	`  CALL ReBoot()`
	`  Count ← Count + 1`
	`  Flag ← Check()`
	`ENDWHILE`
	
	`IF Flag = FALSE THEN`
	`  CALL Alert(27)`
	`ENDIF`
	
	One mark per point:
	
	1 Initialisation of `Count` **AND** `Flag`
	2 `WHILE ... ENDWHILE // REPEAT ... UNTIL` loop
	3 ... including both conditions
	4 Call `ReBoot()` **AND** increment `Count` **inside the loop**
	5 Assign return value from `Check()` to `Flag` **inside the loop**
	6 Final test of `Flag` **AND** call to `Alert(27)` **not in a loop**
		6

Examiner report missing
9618_w21_qp_21 Question 2

5(a)	One mark for the character and one for the corresponding reason. • Character: Any except alphabetic, numeric, ',' ':' or space • Reason: character doesn't occur in data to be recorded	**2**

5(c)		**7**

```
FUNCTION LogEvents(StudentID : STRING) RETURNS INTEGER

  DECLARE FileData : STRING
  DECLARE Index, Count : INTEGER

  CONSTANT LogFile = "LogFile"

  Count ← 0
  OPENFILE LogFile FOR APPEND
  FOR Index ← 1 TO 2000
     FileData ← LogArray[Index]

     IF LEFT(FileData, 6) = StudentID THEN
        WRITEFILE (LogFile, FileData) //brackets optional
        Count ← Count + 1
        LogArray[Index] ← ""        // clear the element
     ENDIF
  NEXT Index

  CLOSEFILE LogFile

  RETURN Count

ENDFUNCTION
```

1 mark for each of the following:

1 Function heading and ending including parameter **and** return type
2 `OPEN` file `LogFile` for `APPEND` and subsequent `CLOSE`
3 Loop for 2000 iterations
4 Extract first 6 characters from array element **in a loop**
5 Compare first 6 characters with parameter **in a loop**
6 If equal:
- write whole array element string to file **and**
- increment `Count` **and**
- clear array element **in a loop**

7 Return Count (must have been declared **and** initialised)

Examiner report missing
9618_w21_qp_21 Question 5

6(a)	```
	PROCEDURE SetRow(Row, SkipNum, SetNum : INTEGER)
	  DECLARE Col : INTEGER
	
	  // array is 1280 x 800
	
	  FOR Col ← SkipNum + 1 TO SkipNum + SetNum
	    Screen[Row, Col] ← 1
	  NEXT Index
	
	ENDPROCEDURE
	``
	ALTERNATIVE 1:
	``
	  FOR Col ← 1 TO SetNum
	    Screen[Row, SkipNum + Col] ← 1
	  NEXT Col
	``
	ALTERNATIVE 2:
	``
	  WHILE SetNum > 0
	    Screen[Row, SkipNum + SetNum] ← 1
	    SetNum ← SetNum - 1
	  ENDWHILE
	``
	Mark as follows:
	
	1 Procedure heading and ending including parameters
	2 Declaration of local Integer for `Col`
	3 Count-controlled loop with meaningful start number
	4 correct stop number
	5 Reference Screen Array element and set to 1 **in a loop**
		5

6(b)	```
	FUNCTION SearchInRow(ThisRow, StartCol : INTEGER) RETURNS
	        INTEGER
	   DECLARE ThisCol, Step : INTEGER
	   DECLARE Found: BOOLEAN
	
	   // array is 1280 x 800
	
	   Found ← FALSE
	   ThisCol ← StartCol
	
	   // first decide which way to search
	   IF StartCol = 1 THEN
	      Step ← 1
	      EndCol ← 1281
	   ELSE
	       Step ← -1
	       EndCol ← 0
	   ENDIF
	
	   WHILE ThisCol <> EndCol AND Found = FALSE
	      IF Screen[ThisRow, ThisCol] <> 1 THEN
	         ThisCol ← ThisCol + Step
	      ELSE
	         Found ← TRUE
	      ENDIF
	   ENDWHILE
	
	   IF Found = FALSE THEN
	      ThisCol ← -1
	   ENDIF
	
	   RETURN ThisCol
	
	ENDFUNCTION
	``
	
	Mark as follows:
	
	1 Interpreting `StartCol` parameter to determine direction of search
	2 An attempt at searching both up and down
	3 Conditional Loop / Count-controlled loop with use of `ThisCol` index
	4 Using correct values for `StartCol`, `EndCol` and `Step`
	5 Reference a `Screen` element and compare with 1 **in a loop**
	6 If equal save column or immediately Return column **in a loop**
	7 Return column number or −1
	Loop(s) terminate when element with value = 1 found
	
	Max 7 marks if function heading, including return type, and ending is incorrect or incomplete
		8

6(c)	`FUNCTION GetCentreCol(ThisRow : INTEGER) RETURNS INTEGER`
	`  DECLARE StartCol, EndCol, CentreCol : INTEGER`
	
	`  StartCol ← SearchInRow(ThisRow, 1)`
	`  IF StartCol = -1 THEN`
	`  CentreCol ← StartCol`
	`  ELSE`
	`  EndCol ← SearchInRow(ThisRow, 1280)`
	`  CentreCol ← INT((StartCol + EndCol)/2)`
	`  ENDIF`
	`  RETURN CentreCol`
	`ENDFUNCTION`
	
	Mark as follows:
	
	1 Declaration of local `INTEGER` for return value
	2 Use `SearchInRow()` with correct parameters and check for `-1`
	3 Use `SearchInRow(ThisRow, 1)` and `SearchInRow(ThisRow, 1280)`
	4 Calculate centre column
	5 Use of `INT()` function // use of `DIV`
	6 Return –1 or centre value
	
	Max 5 marks if function heading, including return type, and ending is incorrect or incomplete
		6

Examiner report missing
9618_w21_qp_21 Question 6

1(a)		**4**

Example value	Explanation	Variable name	Data type
`"Wong"`	The preferred name of the member joining the football club	**`MemberName`**	**`STRING`**
`FALSE`	A value to indicate whether an existing member of the club lives at the same address	**`FamilyMember`**	**`BOOLEAN`**
`19/02/1983`	When the member joined the football club	**`StartDate`**	**`DATE`**
`1345`	The number of points a member has earned. Members of the club earn points for different activities.	**`Points`**	**`INTEGER`**

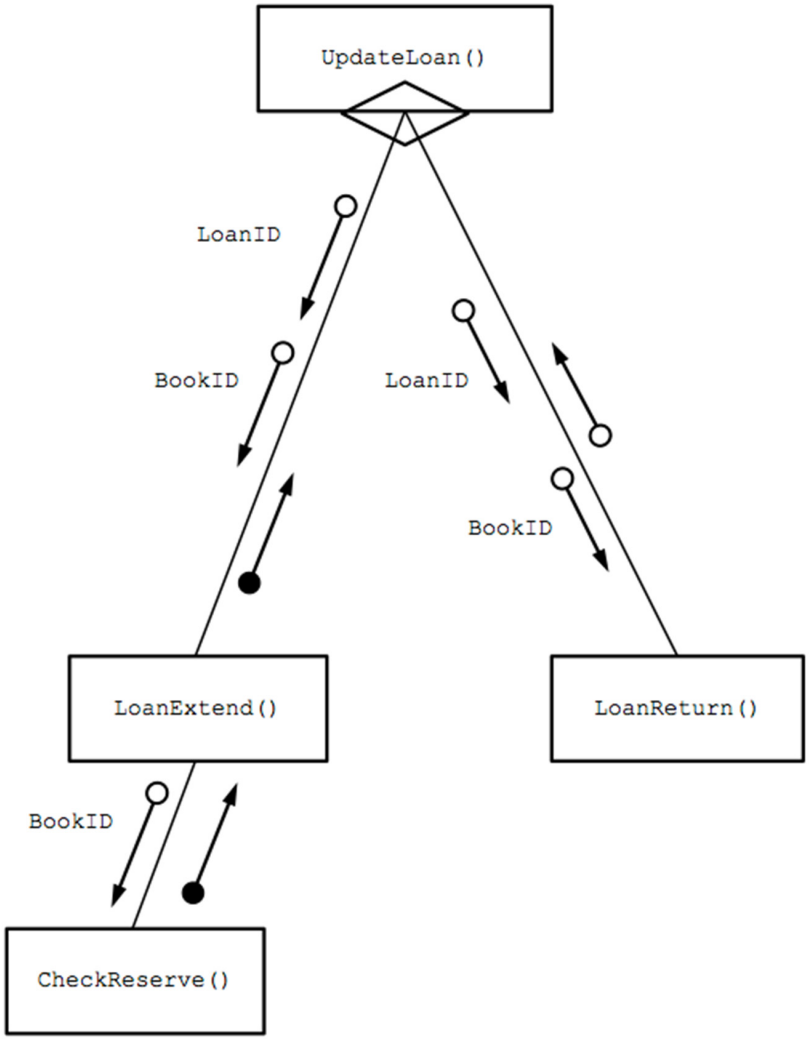
One mark for each appropriate variable name plus data type

1(b)		**5**

Statement	Error
`Result ← 2 & 4`	Should be arithmetic operator (not &) // 2 and 4 should be CHAR / STRING
`SubString ← MID("pseudocode", 4, 1)`	NO ERROR
`IF x = 3 OR 4 THEN`	Not Boolean values / incorrect operator // Condition incorrect
`Result ← Status AND INT(X/2)`	`INT(X/2)` doesn't evaluate to a boolean value / incorrect operator
`Message ← "Done" + LENGTH(MyString)`	Can't add string to number / `"Done"` is not a number

One mark for each row

Examiner report missing
9618_s21_qp_23 Question 1

2(a)	 One mark for: 1 All four boxes correctly labelled and positioned 2 Selection diamond (only on `UpdateLoan` and no iteration arrows) 3 Parameters to and from `LoanExtend()` 4 Parameters to and from `CheckReserve()` 5 Parameters to and from `LoanReturn()`	**5**

2(b)	PROCEDURE LoanReturn (LoanID, BookID : STRING, BYREF Fine : REAL) One mark for each underlined part	**2**

Examiner report missing
9618_s21_qp_23 Question 2

3(a)(i)	One mark per bullet point: • Data from the arrays is written to the files at the end of the day / before the program is terminated / computer is switched off • Data can then be read from the files at the start of the next day and written to / stored in the arrays • No need to (re-)enter the data manually // only need to enter data once **Note: Max 2 marks**	**2**

3(b)	``` PROCEDURE Preview (ThisFile : STRING) DECLARE LineNum : INTEGER DECLARE ThisLine : STRING OPENFILE ThisFile FOR READ IF EOF(ThisFile) THEN OUTPUT "Warning Message" ELSE LineNum ← 1 WHILE LineNum < 6 AND NOT EOF(ThisFile) READFILE Thisfile, ThisLine OUTPUT ThisLine LineNum ← LineNum + 1 ENDWHILE ENDIF CLOSEFILE ThisFile ENDPROCEDURE ``` Marks as follows: 1 Procedure heading (including parameter) and ending 2 File `OPEN` and subsequently `CLOSE` 3 Check if file is empty **and** output a warning message if it is 4 Conditional Loop 5 Output line (including blank lines) and read next line **in a loop**	**5**

Examiner report missing
9618_s21_qp_23 Question 3

4(a)		**5**

Name	Flag	Index	NewName	ThisChar
"∇in∇a∇∇Cup"				
	TRUE	1	""	
				'∇'
			"∇"	
		2		'i'
	FALSE		"∇I"	
		3		'n'
			"∇In"	
		4		'∇'
	TRUE		"∇In∇"	
		5		'a'
	FALSE		"∇In∇A"	
		6		'∇'
	TRUE		"∇In∇A∇"	
		7		'∇'
			"∇In∇A∇∇"	
		8		'C'
	FALSE		"∇In∇A∇∇C"	
		9		'u'
			"∇In∇A∇∇Cu"	
		10		'p'
			"∇In∇A∇∇Cup"	
	FALSE	11	"∇In∇A∇∇Cup"	'p'

Mark as follows:

- One mark for each of columns 2 to 5 (condone missing final 11)
- One mark for final row all correct (including final 11)

4(b)	Loop structure: A count-controlled loop Justification: The number of iterations is known One mark per point	**2**

4(c)(i)	A couple of solutions:
	
	`24  ThisChar ← LCASE(MID(Name, Index, 1)`
	
	ALTERNATIVE:
	
	`31  NewName ← NewName & LCASE(ThisChar)`
	
	Ignore line number
		1

Examiner report missing
9618_s21_qp_23 Question 4

5	``` PROCEDURE Sort() DECLARE Temp : INTEGER DECLARE NoSwaps : BOOLEAN DECLARE Boundary, Row, Col : INTEGER Boundary ← 999 REPEAT NoSwaps ← TRUE FOR Row ← 1 TO Boundary IF Result[Row, 2] > Result[Row + 1, 2] THEN FOR Col ← 1 TO 2 Temp ← Result [Row, Col] Result [Row, Col] ← Result [Row + 1, Col] Result [Row + 1, Col] ← Temp NEXT Col NoSwaps ← FALSE ENDIF NEXT J Boundary ← Boundary - 1 UNTIL NoSwaps = TRUE ENDPROCEDURE ``` Mark as follows: 1 Outer loop 2 Inner loop 3 Correct comparison **in a loop** 4 Correct swap of col1 array elements **in a loop** 5 Correct swap of col2 array elements **in a loop** (via loop or separate statements) 6 `'NoSwap'` mechanism: Conditional outer loop including flag reset 7 `'NoSwap'` mechanism: Set flag in inner loop to indicate swap 8 Reducing Boundary **in the <u>outer</u> loop**	**8**

Examiner report missing
9618_s21_qp_23 Question 5

7(a)	
	``
	FUNCTION GetStart (WordNum : INTEGER) RETURNS INTEGER
	  DECLARE Index, ThisPos, NumFound : INTEGER
	  DECLARE ThisChar : Char
	
	  CONSTANT SPACECHAR = ' '
	
	  Index ← -1
	  Numfound ← 0
	  ThisPos ← 1
	
	  IF WordNum = 1 THEN // if looking for word 1...
	     Index ← 1 // Word 1 always starts at index
	                      // position 1
	  ELSE
	                       // Otherwise start counting spaces...
	     WHILE ThisPos <= LENGTH(FNString) AND Index = -1
	        ThisChar ← MID(FNString, ThisPos, 1)
	        IF ThisChar = SPACECHAR THEN
	           NumFound ← NumFound + 1
	           IF NumFound = WordNum - 1 THEN
	              Index ← ThisPos + 1 // the start of the
	                                  // required word
	           ENDIF
	        ENDIF
	        ThisPos ← ThisPos + 1
	     ENDWHILE
	  ENDIF
	RETURN Index
	
	ENDFUNCTION
	``
	
	1 mark for each of the following:
	
	1 Function heading, including return type and function end
	2 Loop counting spaces until word found or end of `FNString`
	3 extract a character from `FNString` **in a loop**
	4 compare with `SPACECHAR` **and** increment count if equal **in a loop**
	5 compare count with `WordNum - 1` (depending on initialisation value) **in a loop**
	6 if equal then set flag or `Index` to `ThisPos + 1` **in a loop**
	7 Return Index (correctly in all cases / following a reasonable attempt)
	8 Works for special case when looking for word 1
	
	Note: Max 7 marks
		7

7(c)		**5**

```
FUNCTION GetWord (Index : INTEGER) RETURNS STRING

  DECLARE NextWord : STRING
  DECLARE Done : BOOLEAN
  DECLARE ThisChar : CHAR
  DECLARE Index : INTEGER

  CONSTANT SPACECHAR = ' '

  NextWord ← ""
  Done ← FALSE

  REPEAT
    ThisChar ← MID(FNString, Index, 1)
    IF ThisChar <> SPACECHAR THEN
      NextWord ← NextWord & ThisChar // build up NextWord
    ENDIF
    IF ThisChar = SPACECHAR OR Index = LENGTH(FNString) THEN
        Done ← TRUE
    ENDIF

    Index ← Index + 1

  UNTIL Done = TRUE

RETURN NextWord

ENDFUNCTION
```

1 mark for each of the following:

1 Conditional loop
2 Extract char from `FNString` and compare with `SPACECHAR` **in a loop**
3 Concatenate with `NextWord` if not `SPACECHAR` **in a loop**
4 Exit loop when `SPACECHAR` encountered **or** when end of `FNString` reached
5 Return `NextWord` (after reasonable attempt at forming, and must have been initialised)

Examiner report missing
9618_s21_qp_23 Question 7

1(a)(i)

Variable	Example data value	Data type
Name	"Catherine"	**STRING**
Index	100	**INTEGER**
Modified	FALSE	**BOOLEAN**
Holiday	25/12/2020	**DATE**

One mark per data type

4

1(a)(ii)

Expression	Evaluates to
Modified OR Index > 100	**FALSE**
LENGTH("Student: " & Name)	**18**
INT(Index + 2.9)	**102**
MID(Name, 1, 3)	**"Cat"**

One mark per value
Quotation marks must be present for final row and must be capital C

4

1(b)

Statement	Selection	Assignment	Iteration
Index ← Index + 1		✓	
IF Modified = TRUE THEN	✓		
ENDWHILE			✓

One mark per row

3

Examiner report missing
9618_s21_qp_22 Question 1

5(a)(i)	`DECLARE RNum : ARRAY[1:100] OF INTEGER`
	`DECLARE Index, Count : INTEGER`
	
	`Count ← 0`
	`FOR Index ← 1 TO 100`
	`  RNum[Index] ← INT(RAND(200)) + 1`
	`  IF RNum[Index] >= 66 AND RNum[Index] <= 173 THEN`
	`  Count ← Count + 1`
	`  ENDIF`
	`NEXT Index`
	`OUTPUT Count`
	
	Mark as follows:
	1 Array declaration
	2 Loop for 100 iterations
	3 Array element index 'syntax' (left-hand side of assignment expression) **in a loop**
	4 Use of `RAND()` to generate value in range (and assign to array element) **in a loop**
	5 Check if random number within range **and** if so, increment count **in a loop**
	6 Output of count (following a reasonable attempt) **after the loop**
		6

5(b)(i)	**Answer** Give a line number containing an example of an initialisation statement. — **07** Give a line number containing the start of a repeating block of code. — **09 / 10** Give a line number containing a logic statement. — **12** Give the number of parameters of function `MID()`. — **3** One mark for each row	**4**

5(b)(ii)	`IF (NextChar >= 'a') AND (NextChar <= 'z') THEN` One mark for `IF ... AND ...` One mark for **both** conditions	**2**

Examiner report missing
9618_s21_qp_22 Question 5

6	`PROCEDURE CountVowels(ThisString : STRING)`
	
	`  DECLARE Index : INTEGER`
	`  DECLARE ThisChar : CHAR`
	
	`  FOR Index ← 1 to 6`
	`  CharCount[Index] ← 0 //initialise elements`
	`  NEXT Index`
	`  Index ← 1`
	
	`  FOR Index ← 1 TO LENGTH(ThisString)`
	`  ThisChar ← LCASE(MID(ThisString, Index, 1))`
	
	`  CASE OF ThisChar`
	`  'a'  : CharCount[1] ← CharCount[1] + 1`
	`  'e'  : CharCount[2] ← CharCount[2] + 1`
	`  'i'  : CharCount[3] ← CharCount[3] + 1`
	`  'o'  : CharCount[4] ← CharCount[4] + 1`
	`  'u'  : CharCount[5] ← CharCount[5] + 1`
	`  'a' TO 'z': CharCount[6] ← CharCount[6] + 1`
	`  ENDCASE`
	`  NEXT Index`
	
	`  FOR Index ← 1 to 6`
	`  OUTPUT CharCount[Index] //output results`
	`  NEXT Index`
	
	`ENDPROCEDURE`
	
	1 mark for each of the following:
	
	1 Procedure heading (with parameter) and ending
	2 Declare local variable for `Index` as loop counter but **not** `CharCount` array
	3 Initialise elements of `CharCount` array to zero
	4 Loop through all characters in `ThisString`
	5 Use of `MID()` to extract single character
	6 Test for each vowel **and** increment associated count
	7 Test for consonants **and** increment associated count
	8 Output the results (supporting text not necessary) **after the loop**
	
	Note: Max 7 if `CharCount` not used to store count
		8

Examiner report missing
9618_s21_qp_22 Question 6

8(a)	`FUNCTION IgnoreWord (ThisWord : STRING) RETURNS BOOLEAN`
	
	`  DECLARE Found : BOOLEAN`
	`  DECLARE Index : INTEGER`
	
	`  Found ← False`
	`  Index ← 1`
	`  ThisWord ← TO_LOWER(ThisWord)`
	
	`  REPEAT`
	`  IF TO_LOWER(IgnoreList[Index]) = ThisWord THEN`
	`  Found ← TRUE`
	`  ENDIF`
	`  Index ← Index + 1`
	`  UNTIL Found = TRUE OR Index > 10`
	
	`  RETURN Found`
	
	`ENDFUNCTION`
	
	1 mark for each of the following:
	
	1 Loop through array elements
	2 Convert both strings to same case
	3 Compare array element with parameter **in a loop**
	4 Set a flag (or similar) if match found (after reasonable attempt at MP3) **in a loop**
	5 Return `TRUE` or `FALSE` in all cases
	
	Note: Max 4 if function declaration incorrect
		5

8(b)		**8**

```
Procedure GetInitials()

  DECLARE NewString, NextWord : STRING
  DECLARE ThisWordNum, Index : INTEGER

  ThisWordNum ← 0
  NewString ← ""

  REPEAT
    ThisWordNum ← ThisWordNum + 1
    Index ← GetStart(ThisWordNum)
    IF Index <> -1 THEN //if there is ThisWordNum
      NextWord  ← GetWord(Index)
      IF IgnoreWord(NextWord) = FALSE THEN
       NewString ← NewString & UCASE(LEFT(NextWord, 1))
      ENDIF
    ENDIF

  UNTIL Index = -1

  OUTPUT NewString

ENDPROCEDURE
```

1 mark for each of the following:

1 Declare `NewString` and initialise to empty string
2 Conditional loop to pick out **all** words from `FNString`
3 Evaluate result of `GetStart()` **in a loop**
4 Test result <> -1 and if not:
5 Assign result of `GetWord()` to a variable **in a loop**
6 Test result of `IgnoreWord()` **in a loop**
7 If not ignored, add the next initial letter to `NewString` **in a loop**
8 Increment `ThisWordNum` (must have been initialised) **in a loop**
9 Output `NewString` (must be all upper case) **outside loop**

Note: Max 8 marks

Examiner report missing
9618_s21_qp_22 Question 8

1(a)		**4**

Example value	Explanation	Variable name	Data type
`"Wong"`	The preferred name of the member joining the football club	**`MemberName`**	**`STRING`**
`FALSE`	A value to indicate whether an existing member of the club lives at the same address	**`FamilyMember`**	**`BOOLEAN`**
`19/02/1983`	When the member joined the football club	**`StartDate`**	**`DATE`**
`1345`	The number of points a member has earned. Members of the club earn points for different activities.	**`Points`**	**`INTEGER`**

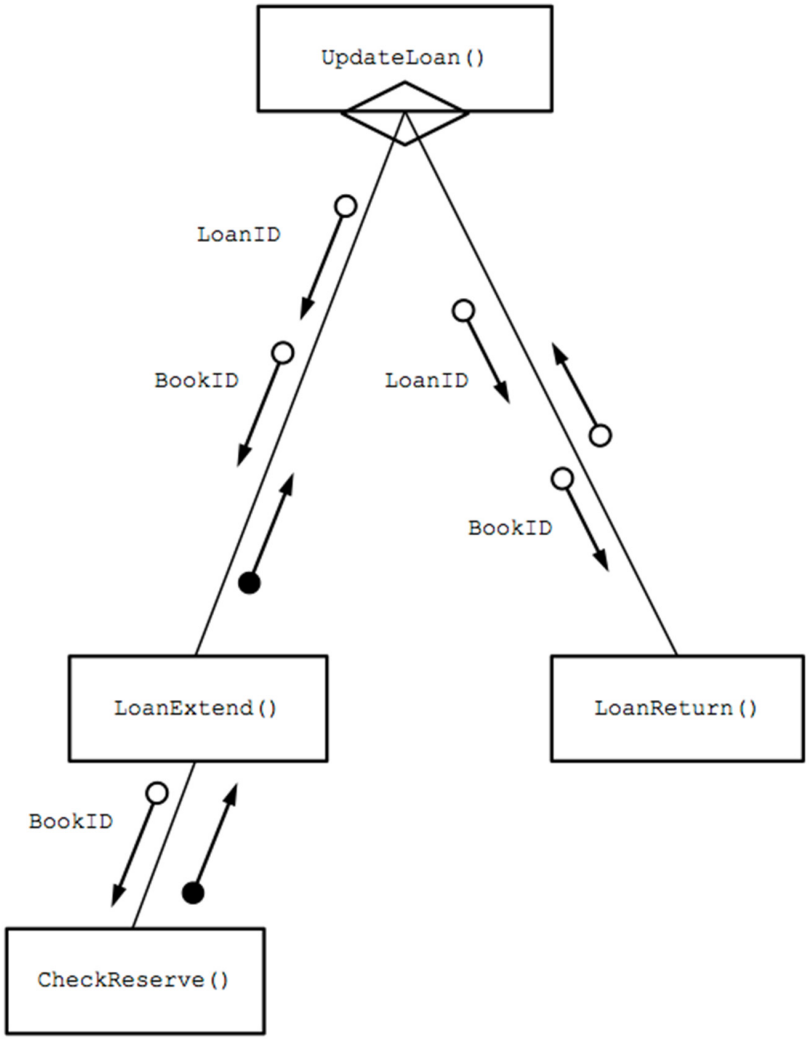
One mark for each appropriate variable name plus data type

1(b)		**5**

Statement	Error
`Result ← 2 & 4`	Should be arithmetic operator (not &) // 2 and 4 should be CHAR / STRING
`SubString ← MID("pseudocode", 4, 1)`	NO ERROR
`IF x = 3 OR 4 THEN`	Not Boolean values / incorrect operator // Condition incorrect
`Result ← Status AND INT(X/2)`	`INT(X/2)` doesn't evaluate to a boolean value / incorrect operator
`Message ← "Done" + LENGTH(MyString)`	Can't add string to number / `"Done"` is not a number

One mark for each row

Examiner report missing
9618_s21_qp_21 Question 1

2(a)	 One mark for: 1 All four boxes correctly labelled and positioned 2 Selection diamond (only on `UpdateLoan` and no iteration arrows) 3 Parameters to and from `LoanExtend()` 4 Parameters to and from `CheckReserve()` 5 Parameters to and from `LoanReturn()`	**5**

2(b)	PROCEDURE LoanReturn (LoanID, BookID : STRING, BYREF Fine : REAL) One mark for each underlined part	**2**

Examiner report missing
9618_s21_qp_21 Question 2

3(a)(i)	One mark per bullet point: • Data from the arrays is written to the files at the end of the day / before the program is terminated / computer is switched off • Data can then be read from the files at the start of the next day and written to / stored in the arrays • No need to (re-)enter the data manually // only need to enter data once **Note: Max 2 marks**	**2**

3(b)	``` PROCEDURE Preview (ThisFile : STRING) DECLARE LineNum : INTEGER DECLARE ThisLine : STRING OPENFILE ThisFile FOR READ IF EOF(ThisFile) THEN OUTPUT "Warning Message" ELSE LineNum ← 1 WHILE LineNum < 6 AND NOT EOF(ThisFile) READFILE Thisfile, ThisLine OUTPUT ThisLine LineNum ← LineNum + 1 ENDWHILE ENDIF CLOSEFILE ThisFile ENDPROCEDURE ``` Marks as follows: 1 Procedure heading (including parameter) and ending 2 File `OPEN` and subsequently `CLOSE` 3 Check if file is empty **and** output a warning message if it is 4 Conditional Loop 5 Output line (including blank lines) and read next line **in a loop**	**5**

Examiner report missing
9618_s21_qp_21 Question 3

4(a)		**5**

Name	Flag	Index	NewName	ThisChar
"∇in∇a∇∇Cup"				
	TRUE	1	""	
				'∇'
			"∇"	
		2		'i'
	FALSE		"∇I"	
		3		'n'
			"∇In"	
		4		'∇'
	TRUE		"∇In∇"	
		5		'a'
	FALSE		"∇In∇A"	
		6		'∇'
	TRUE		"∇In∇A∇"	
		7		'∇'
			"∇In∇A∇∇"	
		8		'C'
	FALSE		"∇In∇A∇∇C"	
		9		'u'
			"∇In∇A∇∇Cu"	
		10		'p'
			"∇In∇A∇∇Cup"	
	FALSE	11	"∇In∇A∇∇Cup"	'p'

Mark as follows:

- One mark for each of columns 2 to 5 (condone missing final 11)
- One mark for final row all correct (including final 11)

4(b)	Loop structure: A count-controlled loop Justification: The number of iterations is known One mark per point	**2**

4(c)(i)	A couple of solutions:
	
	`24  ThisChar ← LCASE(MID(Name, Index, 1)`
	
	ALTERNATIVE:
	
	`31  NewName ← NewName & LCASE(ThisChar)`
	
	Ignore line number
		1

Examiner report missing
9618_s21_qp_21 Question 4

5	``` PROCEDURE Sort() DECLARE Temp : INTEGER DECLARE NoSwaps : BOOLEAN DECLARE Boundary, Row, Col : INTEGER Boundary ← 999 REPEAT NoSwaps ← TRUE FOR Row ← 1 TO Boundary IF Result[Row, 2] > Result[Row + 1, 2] THEN FOR Col ← 1 TO 2 Temp ← Result [Row, Col] Result [Row, Col] ← Result [Row + 1, Col] Result [Row + 1, Col] ← Temp NEXT Col NoSwaps ← FALSE ENDIF NEXT J Boundary ← Boundary - 1 UNTIL NoSwaps = TRUE ENDPROCEDURE ``` Mark as follows: 1 Outer loop 2 Inner loop 3 Correct comparison **in a loop** 4 Correct swap of col1 array elements **in a loop** 5 Correct swap of col2 array elements **in a loop** (via loop or separate statements) 6 `'NoSwap'` mechanism: Conditional outer loop including flag reset 7 `'NoSwap'` mechanism: Set flag in inner loop to indicate swap 8 Reducing Boundary **in the <u>outer</u> loop**	**8**

Examiner report missing
9618_s21_qp_21 Question 5

7(a)	
	``
	FUNCTION GetStart (WordNum : INTEGER) RETURNS INTEGER
	  DECLARE Index, ThisPos, NumFound : INTEGER
	  DECLARE ThisChar : Char
	
	  CONSTANT SPACECHAR = ' '
	
	  Index ← -1
	  Numfound ← 0
	  ThisPos ← 1
	
	  IF WordNum = 1 THEN // if looking for word 1...
	     Index ← 1 // Word 1 always starts at index
	                      // position 1
	  ELSE
	                       // Otherwise start counting spaces...
	     WHILE ThisPos <= LENGTH(FNString) AND Index = -1
	        ThisChar ← MID(FNString, ThisPos, 1)
	        IF ThisChar = SPACECHAR THEN
	           NumFound ← NumFound + 1
	           IF NumFound = WordNum - 1 THEN
	              Index ← ThisPos + 1 // the start of the
	                                  // required word
	           ENDIF
	        ENDIF
	        ThisPos ← ThisPos + 1
	     ENDWHILE
	  ENDIF
	RETURN Index
	
	ENDFUNCTION
	``
	
	1 mark for each of the following:
	
	1 Function heading, including return type and function end
	2 Loop counting spaces until word found or end of `FNString`
	3 extract a character from `FNString` **in a loop**
	4 compare with `SPACECHAR` **and** increment count if equal **in a loop**
	5 compare count with `WordNum - 1` (depending on initialisation value) **in a loop**
	6 if equal then set flag or `Index` to `ThisPos + 1` **in a loop**
	7 Return Index (correctly in all cases / following a reasonable attempt)
	8 Works for special case when looking for word 1
	
	Note: Max 7 marks
		7

7(c)		**5**

```
FUNCTION GetWord (Index : INTEGER) RETURNS STRING

  DECLARE NextWord : STRING
  DECLARE Done : BOOLEAN
  DECLARE ThisChar : CHAR
  DECLARE Index : INTEGER

  CONSTANT SPACECHAR = ' '

  NextWord ← ""
  Done ← FALSE

  REPEAT
    ThisChar ← MID(FNString, Index, 1)
    IF ThisChar <> SPACECHAR THEN
      NextWord ← NextWord & ThisChar // build up NextWord
    ENDIF
    IF ThisChar = SPACECHAR OR Index = LENGTH(FNString) THEN
        Done ← TRUE
    ENDIF

    Index ← Index + 1

  UNTIL Done = TRUE

RETURN NextWord

ENDFUNCTION
```

1 mark for each of the following:

1 Conditional loop
2 Extract char from `FNString` and compare with `SPACECHAR` **in a loop**
3 Concatenate with `NextWord` if not `SPACECHAR` **in a loop**
4 Exit loop when `SPACECHAR` encountered **or** when end of `FNString` reached
5 Return `NextWord` (after reasonable attempt at forming, and must have been initialised)

Examiner report missing
9618_s21_qp_21 Question 7